

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

Кафедра програмних засобів
(найменування кафедри)

**КУРСОВИЙ ПРОЄКТ
(РОБОТА)**

з Алгоритми та структури даних
(назва дисципліни)
на тему: Моделювання транспортної мережі за допомогою графів
(назва теми)

Студента(ки) 2 курсу 133 групи
Спеціальності інженерія програмного забезпечення
освітня програма (спеціалізація) інженерія програм-
ного забезпечення

Н. О. Сазонова, В. І. Гершиков, М. О. Ушаков
(ПРІЗВИЩЕ та ініціали)

Керівник доцент, к.т.н., В.М. Льовкін

(посада, вчене звання, науковий ступінь, ПРІЗВИЩЕ та ініціали)

Кількість балів: _____

Члени комісії

_____	_____
(підпис)	(Ім'я ПРІЗВИЩЕ)
_____	_____
(підпис)	(Ім'я ПРІЗВИЩЕ)
_____	_____
(підпис)	(Ім'я ПРІЗВИЩЕ)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет «Запорізька політехніка»

Факультет Комп'ютерних наук та технологій
Кафедра Програмних засобів
Ступінь вищої освіти Бакалавр
Спеціальність 121 інженерія програмного забезпечення
(код і найменування)
Освітня програма (спеціалізація) інженерія програмного забезпечення програма
(назва освітньої програми (спеціалізації))

З А В Д А Н Н Я
НА КУРСОВИЙ ПРОЄКТ (РОБОТУ) СТУДЕНТА(КИ)

Н. О. Сазонова, В. І. Гершиков, М. О. Ушаков

(ПРИЗВИЩЕ, ім'я, по батькові)

1. Тема проєкту (роботи) моделювання транспортної мережі за допомогою графів,

керівник проєкту (роботи) доцент, к.т.н., В.М. Льовкін,
(науковий ступінь, вчене звання, ПРИЗВИЩЕ, ім'я, по батькові)

затверджені наказом закладу вищої освіти від « » 20 року №

2. Строк подання студентом проєкту (роботи) 17.12.2024
3. Вихідні дані до проєкту (роботи) структура даних та алгоритм під час доставки та перевезення вантажів на основі графів. Порівняння алгоритмів максимального потоку з алгоритмом Форда–Фалкерсона як базовим.

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити) титульний лист, лист завдання, реферат, зміст, скорочення та умовні позначки, вступ, аналіз предметної області, проєктування програмного забезпечення, прийняті рішення щодо реалізації програмного забезпечення, результати порівняння алгоритмів та структур даних, керівництво оператора, керівництво програміста, висновки, перелік джерел посилання, додаток А технічне завдання, додаток Б текст програми.

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень, кількість слайдів, плакатів)

Презентація (кількість слайдів: 34): Плакати (кількість: 27) – з коротким описом проєкту, ключовими результатами та ілюстраціями. Обов'язкові креслення: графік – Порівняння ефективності роботи реалізованих алгоритмів, гіфки (кількість: 4) – з демонстрацією роботи програми, таблиця – Порівняння алгоритмів.

6. Консультанти розділів проєкту (роботи)

Розділ	ПРІЗВИЩЕ, ініціали та посада консультанта	Підпис, дата	
		завдання видав	прийняв виконане завдання
Розробка коду	Льовкін В.М. доцент	07.09.2024	
Реферат	Льовкін В.М. доцент	17.09.2024	
Керівництво оператора	Льовкін В.М. доцент	21.09.2024	
Керівництво програміста	Льовкін В.М. доцент	28.09.2024	
Технічне завдання	Льовкін В.М. доцент	05.10.2024	
Оформлення пояснювальної записки	Льовкін В.М. доцент	12.10.2024	
Розробка коду	Льовкін В.М. доцент	07.09.2024	

7. Дата видачі завдання « 7 » вересня 2024 року.

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів дипломного проєкту (роботи)	Строк виконання етапів проєкту (роботи)	Примітка
1	Аналіз вимог	2.12.2024	
2	Проектування	9.12.2024	
3	Реалізація (кодування)	16.12.2024	
4	Тестування	23.12.2024	
5	Документування	24.12.2024	

Студенти

(підпис)

Н. Сазонова, В. Гершиков, М. Ушаков

(Ім'я ПРІЗВИЩЕ)

Керівник проєкту (роботи)

(підпис)

В.М. Льовкін

(Ім'я ПРІЗВИЩЕ)

РЕФЕРАТ

МАТЕМАТИЧНА СТРУКТУРА З ВЕРШИНАМИ ТА РЕБРАМИ, МАКСИМАЛЬНИЙ ПОТІК, ТРАНСПОРТНА МЕРЕЖА

Тема роботи – структури даних та алгоритми для перевезення вантажів на основі графів. Порівняння алгоритмів максимального потоку.

Мета роботи – розробка ПЗ для моделювання транспортних мереж за допомогою графів, аналізу їх пропускної здатності та обчислення максимального потоку між вершинами. Реалізовано алгоритми Форда-Фалкерсона та Едмондса-Карпа для порівняння продуктивності.

Робота з графами: завантаження, створення, редагування, візуалізація.

Алгоритми: Форд-Фалкерсон, Едмондс-Карп, порівняння ефективності алгоритмів, включаючи точність розрахунків, час виконання та результати для різних графів. Алгоритм Форда-Фалкерсона базовий підхід, залежний від кількості ітерацій. Використовується пошук у глибину. Алгоритм Едмондса-Карпа вдосконалення через BFS. Складність: $O(V \cdot E^2)$. Продуктивний для великих графів.

Висновок: Розроблена програма успішно реалізує аналіз транспортних мереж. Алгоритм Форда-Фалкерсона оптимальний для малих графів, а Едмондс-Карпа – для складних задач. Подальший розвиток ПЗ дозволить вирішувати задачі маршрутизації та управління потоками.

ЗМІСТ

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ.....	8
----------------------------------	---

1.1 Постановка задачі.....	8
1.2 Вимоги до програми.....	8
1.3 Аналіз існуючих алгоритмів та структур даних.....	9
1.3.1 Алгоритм Форда-Фалкерсона.....	9
1.3.2 Алгоритм Едмондса-Карпа.....	9
1.3.3 Алгоритм Дініца.....	10
2 ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	11
2.1 Вибрані засоби розробки.....	11
2.2 Структура програми.....	11
2.3 Рішення щодо алгоритмів та структур даних.....	11
2.3.1 Рішення щодо алгоритмів.....	11
2.3.2 Рішення щодо структур даних.....	12
2.3.3 Опис створених класів.....	12
3 ПРИЙНЯТІ РІШЕННЯ ЩОДО РЕАЛІЗАЦІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	14
3.1 Розподіл відповідальності в між учасниками групи.....	14
3.2 Опис створених методів.....	14
3.2.1 Опис методу для обчислення графів заданих користувачем.....	14
3.2.2 Метод для перевірки на точність.....	15
3.2.3 Метод для перевірки на точність.....	15
4 РЕЗУЛЬТАТИ ПОРІВНЯННЯ АЛГОРИТМІВ.....	16
4.1 Порівняння алгоритмів.....	16
5 КЕРІВНИЦТВО ОПЕРАТОРА.....	19
5.1 Призначення програми.....	19
5.2 Умови виконання програми.....	19
5.3 Виконання програми.....	20
5.4 Повідомлення оператору.....	21
5.5 Додатки.....	22
6 КЕРІВНИЦТВО ПРОГРАМІСТА.....	29
6.1 Призначення та умови застосування програми.....	29
6.1.1 Призначення.....	29
6.1.2 Функції програми.....	29
6.1.3 Умови застосування.....	29
6.2 Характеристика програми.....	30
6.2.1 Часові характеристики.....	30
6.2.2 Режим роботи.....	30
6.2.3 Засоби контролю правильності виконання.....	31
6.2.4 Засоби самовідновлення.....	31
6.3 Звертання до програми.....	31

6.4 Початкові та вихідні дані.....	31
6.4.1 Початкові дані.....	31
6.4.2 Формат введення даних.....	32
6.4.3 Кодування.....	32
6.4.4 Вихідні дані.....	33
6.5 Повідомлення.....	33
ВИСНОВОК.....	35
ДОДАТОК А.....	36
ДОДАТОК Б.....	50
ДОДАТОК В.....	121

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Постановка задачі

Створити програму, яка реалізує алгоритм Форда-Фалкерсона для знаходження максимального потоку в мережі, представлений у вигляді орієнтованого графа з ваговими ребрами (ємністю). Вона також забезпечує візуальне представлення мережі та результатів обчислень.

1.2 Вимоги до програми

Введення даних: Програма повинна забезпечувати зручний спосіб введення та видалення даних, що описують мережу: вершини (додавання та видалення), список ребер (пара вершин, додавання, оновлення, видалення ребра).

Знаходження максимального потоку: Це головна відсутня функціональна вимога. Код повинен бути розширений, щоб включати функцію, яка реалізує алгоритм Форда-Фалкерсона (або його модифікацію, наприклад, Edmonds-Karp) для знаходження максимального потоку між заданою джерельною та стоковою вершинами. Функція повинна повертати величину максимального потоку та, бажано, детальний опис потоку (який шлях і яка ємність на кожному ребрі використовується).

Виведення результатів: Програма повинна відображати результат обчислень, значення максимального потоку, в зрозумілому для користувача форматі (наприклад, на екрані, у вихідному файлі).

Обчислення дати доставки: Програма повинна реалізувати алгоритми Форда-Фалкерсона а Едмондса-Карпа для обчислення дати доставки товарів на основі максимального потоку в мережі.

1.3 Аналіз існуючих алгоритмів та структур даних

1.3.1 Алгоритм Форда-Фалкерсона

Ідея: Алгоритм ітеративно знаходить збільшуючі шляхи від джерела до стоку, поки не буде знайдено максимальний потік. Збільшуючий шлях – це шлях від джерела до стоку, по якому можна збільшити потік.

Переваги: Простий для розуміння та реалізації.

Недоліки: Час роботи залежить від ємності ребер. У найгіршому випадку може бути дуже повільним ($O(fE)$, де f – максимальний потік, E – кількість ребер).

Модифікації: Існують модифікації алгоритму Форда-Фалкерсона, які покращують час роботи, такі як: алгоритм Едмондса-Карпа чи алгоритм Дініца.

Основна структура: Для представлення мережі використовується матриця суміжності або список суміжності. Вибір залежить від щільності графу. Матриця суміжності простіша для реалізації, але менш ефективна для розріджених графів. Список суміжності ефективніший для розріджених графів.

Додаткові структури: Для пошуку збільшуючих шляхів можна використовувати різні алгоритми пошуку, такі як пошук у ширину (BFS) або пошук у глибину (DFS). BFS зазвичай ефективніший для знаходження найкоротших шляхів. Для зберігання потоку в ребрах використовується додаткова матриця або масив.

1.3.2 Алгоритм Едмондса-Карпа

Ідея: Модифікація алгоритму Форда-Фалкерсона, яка використовує пошук у ширину (BFS) для знаходження збільшуючих шляхів. Це гарантує, що кожен збільшуючий шлях є найкоротшим.

Переваги: Гарантує поліноміальний час роботи ($O(VE^2)$). Більш ефективний, ніж

базовий алгоритм Форда-Фалкерсона.

Недоліки: Може бути не найшвидшим для деяких типів мереж.

Основна структура: Список суміжності є найкращим вибором для представлення мережі, оскільки він ефективний для розріджених графів, які часто використовуються в задачах максимального потоку.

Додаткові структури: Ключовою особливістю алгоритму Едмондса-Карпа є використання пошуку у ширину (BFS) для знаходження збільшуючих шляхів. BFS використовує чергу для обходу графу. Додатково, потрібна структура для зберігання потоку в ребрах (масив або список).

1.3.3 Алгоритм Дініца

Ідея: Використовує концепцію блокуючих потоків в шарах графу.

Переваги: Гарантує поліноміальний час роботи $O(V^2E)$. Швидший за алгоритм Едмондса-Карпа на практиці.

Недоліки: Складний для розуміння та реалізації.

Основна структура: Для алгоритму Дініца також найкраще підходить список суміжності для ефективного представлення мережі.

Додаткові структури: Алгоритм Дініца використовує шарування графу та пошук блокуючих потоків в шарах. Для цього потрібна структура для зберігання шарування (наприклад, масив, що містить номер шару для кожного вузла). Крім того, використовується структура для пошуку шляхів у шарах (часто це модифікація BFS або DFS).

2 ПРОЄКТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Вибрані засоби розробки

Мова програмування: C#

Графічний інтерфейс: Windows Forms

Платформа:

- .NET Framework;
- .NET;

Бібліотеки:

- System.Drawing;
- System.IO;
- System.Linq;
- інші.

2.2 Структура програми

Програма має таку структуру:

- Метод для обчислення графів заданих користувачем;
- Метод для перевірки на точність;
- Метод для розрахунку доставки товарів.

2.3 Рішення щодо алгоритмів та структур даних

2.3.1 Рішення щодо алгоритмів

Реалізація алгоритма Форда-Фалкерсона в проєкті:

- Метод для обчислення графів заданих користувачем;
- Метод для перевірки на точність;
- Метод для розрахунку доставки товарів.

Реалізація алгоритма Форда-Фалкерсона в проєкті:

- Метод для обчислення графів заданих користувачем;
- Метод для перевірки на точність;
- Метод для розрахунку доставки товарів.

2.3.2 Рішення щодо структур даних

Опис структур даних для графів:

- Vertices: Словник, що містить інформацію про вершини графа;
- AdjacencyList: Словник, який представляє список суміжності ребер;
- vertices: Словник, що містить координати вершин для візуалізації;
- edges: Список кортежів, що описують ребра графа з ідентифікаторами вершин і їх вагою.

2.3.3 Опис створених класів

Опис створених класів в проєкті:

а) Клас Graph представляє граф за допомогою списку суміжності. Він містить словники Vertices (вершини) та AdjacencyList (список суміжності). Надає методи для додавання, видалення вершин та ребер, а також зміни ваги ребра та отримання списку ребер для заданої вершини.

б) Клас Vertex представляє вершину графа, що містить лише її ім'я (Name).

в) Клас Edge представляє ребро графа, включаючи початкову (From) та кінцеву (To) вершини (типу Vertex) та його ємність (Capacity).

г) Клас FordFulkerson реалізує алгоритм Форда-Фалкерсона для пошуку максимального потоку в мережі, представлений графом (graph). Метод FindMaxFlow знаходить максимальний потік між двома заданими вершинами (source, sink),

використовуючи пошук у глибину (DFS) для знаходження шляхів з додатним потоком. Він ітеративно збільшує потік по шляхах, поки не знайде максимальний.

г) Клас EdmondsKarp реалізує алгоритм Едмондса-Карпа для знаходження максимального потоку в мережі, представлений графом (graph). Він використовує пошук у ширину (BFS) для знаходження шляхів з додатним потоком, що забезпечує поліноміальну складність, на відміну від загального алгоритму Форда-Фалкерсона. Метод FindMaxFlow ітеративно збільшує потік по шляхах, знайдених BFS, доки не досягне максимального потоку.

д) Клас FordFulkerson реалізує алгоритм Форда-Фалкерсона для знаходження максимального потоку в мережі, представлений словником вершин (vertices) та списком ребер (edges). Він використовує представлення графа на основі словників для зберігання ємності (capacity) та потоку (flowThroughVertex) між вершинами. Метод FindMaxFlow знаходить максимальний потік між джерелом (source) та стоком (sink), використовуючи допоміжний метод FindAugmentingPath для пошуку збільшуючих шляхів. Результат – максимальний потік та розподіл потоку через вершини.

е) Клас EdmondsKarp реалізує алгоритм Едмондса-Карпа для знаходження максимального потоку в мережі, представлений словником вершин (vertices) та списком ребер (edges). Він використовує словники для представлення ємності (capacity), потоку (flow) та графа (graph). Метод FindMaxFlow знаходить максимальний потік між джерелом та стоком за допомогою пошуку в ширину (BFS) у методі FindAugmentingPath для знаходження збільшуючих шляхів. Алгоритм гарантує знаходження максимального потоку за поліноміальний час.

З ПРИЙНЯТІ РІШЕННЯ ЩОДО РЕАЛІЗАЦІЇ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Розподіл відповідальності в між учасниками групи

Метод для обчислення графів заданих користувачем – М. О. Ушаков;

Метод для перевірки на точність – Н. О. Сазонова.

Метод для розрахунку доставки товарів – В. І. Гершиков.

3.2 Опис створених методів

3.2.1 Опис методу для обчислення графів заданих користувачем

Код реалізує два алгоритми для знаходження максимального потоку в мережі: алгоритм Форда-Фалкерсона (з використанням пошуку в глибину – DFS) та алгоритм Едмондса-Карпа (з використанням пошуку в ширину – BFS).

FordFulkerson клас:

- DFS(source, sink, visited, parent): Рекурсивна функція, яка виконує пошук в глибину для знаходження збільшуючого шляху від джерела (source) до стоку (sink);
- visited – словник відвіданих вершин, parent – словник, що зберігає шлях. Повертає true, якщо шлях знайдено, інакше false;
- FindMaxFlow(source, sink): Головна функція, яка використовує DFS для знаходження максимального потоку. Вона ітеративно шукає шляхи, обчислює мінімальну ємність на шляху (pathFlow), оновлює ємності ребер, та збільшує загальний потік (maxFlow). Цикл триває доки `DFS` знаходить збільшуючі шляхи.

EdmondsKarp клас:

- BFS(source, sink, visited, parent): Функція, яка виконує пошук у ширину для

знаходження збільшуючого шляху. BFS гарантує знаходження найкоротшого шляху, що робить алгоритм Едмондса-Карпа значно ефективнішим, ніж реалізація Форда-Фалкерсона з DFS.

3.2.2 Метод для перевірки на точність

`FindMaxFlowUsingFordFulkerson(int[,] graph, int source, int sink)`: Це реалізація алгоритму Форда-Фалкерсона. Він використовує пошук у ширину (BFS) для знаходження збільшуючого шляху між джерелом (`source`) та стоком (`sink`). Потім оновлює залишкові ємності ребер на знайденому шляху та повторює процес, поки не знайдеться жодного шляху.

`FindMaxFlowUsingEdmondsKarp(int[,] graph, int source, int sink)`: Це реалізація класичного методу пошуку максимального потоку в мережі, що використовує пошук в ширину (BFS) для знаходження augmenting paths (шляхів, які можуть збільшити потік).

3.2.3 Метод для перевірки на точність

`FindMaxFlow(int source, int sink)`: Основна функція, яка знаходить максимальний потік між джерелом (`source`) та стоком (`sink`). Вона ітеративно шукає збільшуючий шлях за допомогою `FindAugmentingPath`, обчислює мінімальну пропускну здатність на шляху, оновлює залишкову пропускну здатність ребер та загальний потік. Цикл триває доки не буде знайдено шлях.

`FindAugmentingPath(int source, int sink)`: Реалізує пошук у ширину (BFS) для знаходження збільшуючого шляху. Він використовує `visited` для відстеження відвіданих вершин та `parent` для відновлення шляху. Якщо шлях знайдено, він повертає список ребер шляху, інакше, повертає `null`.

4 РЕЗУЛЬТАТИ ПОРІВНЯННЯ АЛГОРИТМІВ

4.1 Порівняння алгоритмів

Таблиця 4.1 Порівняння алгоритмів

Характеристика	Алгоритм Форда-Фалкерсона	Алгоритм Едмондса-Карпа
Метод пошуку збільшувальних шляхів	Може використовувати пошук в глибину (DFS) або в ширину (BFS), що не гарантує вибір оптимальних шляхів.	Використовує пошук в ширину (BFS) для знаходження найкоротших збільшувальних шляхів.
Часова складність	Залежить від вибору методу пошуку шляху: для DFS складність може бути $O(E * f)$, де f – величина потоку.	$O(V * E^2)$ – більш ефективний завдяки використанню BFS, що гарантує більш швидкий пошук шляху.
Збільшувальний шлях	Може бути будь-яким шляхом з залишковою ємністю, не обов'язково найкоротшим.	Гарантується, що шлях буде найкоротшим серед усіх можливих.
Удосконалення ємностей	Ємності оновлюються після кожного знайденого шляху, без спеціальної стратегії для вибору шляхів.	Ємності оновлюються після кожного пошуку збільшувального шляху, що є найкоротшим шляхом
Гарантована ефективність	Алгоритм може бути менш ефективним у випадку використання DFS, особливо для великих графів.	Завжди ефективний через використання BFS, що знижує час виконання в порівнянні з простим Форда-Фалкерсона.
Складність реалізації	Простий у реалізації, особливо за допомогою DFS для пошуку шляху.	Трошки складніший у реалізації через необхідність застосування BFS, але значно ефективніший.

Результат перевірки точності



Точність алгоритма Форда-Фалкерсона: 117,5 загальний час: 34 с
 Точність алгоритма Едмондса-Карпа: 117,5 загальний час: 53 с

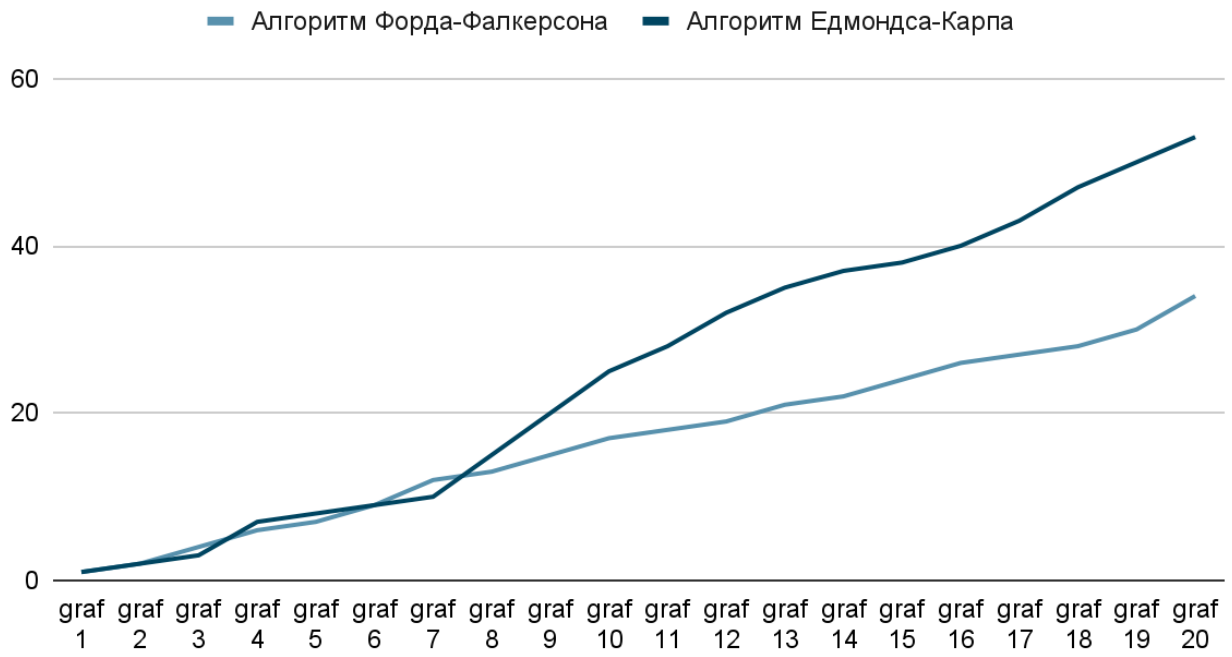
Алгоритм Форда-Фалкерсона

Максимальний потік графа1: 96
 Максимальний потік графа2: 99
 Максимальний потік графа3: 88
 Максимальний потік графа4: 142
 Максимальний потік графа5: 135
 Максимальний потік графа6: 123
 Максимальний потік графа7: 85
 Максимальний потік графа8: 117
 Максимальний потік графа9: 81
 Максимальний потік графа10: 117
 Максимальний потік графа11: 100
 Максимальний потік графа12: 171
 Максимальний потік графа13: 114
 Максимальний потік графа14: 98
 Максимальний потік графа15: 153
 Максимальний потік графа16: 136
 Максимальний потік графа17: 163
 Максимальний потік графа18: 130
 Максимальний потік графа19: 120
 Максимальний потік графа20: 82

Алгоритм Едмондса-Карпа

Максимальний потік графа1: 96
 Максимальний потік графа2: 99
 Максимальний потік графа3: 88
 Максимальний потік графа4: 142
 Максимальний потік графа5: 135
 Максимальний потік графа6: 123
 Максимальний потік графа7: 85
 Максимальний потік графа8: 117
 Максимальний потік графа9: 81
 Максимальний потік графа10: 117
 Максимальний потік графа11: 100
 Максимальний потік графа12: 171
 Максимальний потік графа13: 114
 Максимальний потік графа14: 98
 Максимальний потік графа15: 153
 Максимальний потік графа16: 136
 Максимальний потік графа17: 163
 Максимальний потік графа18: 130
 Максимальний потік графа19: 120
 Максимальний потік графа20: 82

Рисунок 4.1 – Порівняння точності алгоритмів



Графік 4.1 – Порівняння ефективності роботи реалізованих алгоритмів

5 КЕРІВНИЦТВО ОПЕРАТОРА

5.1 Призначення програми

Програма призначена для моделювання та аналізу транспортних мереж, побудованих у вигляді графів. Вона дозволяє:

- Створювати графи вручну або завантажувати їх із файлів;
- Виконувати розрахунки максимального потоку в мережі за допомогою алгоритмів Форда-Фалкерсона та Едмондса-Карпа;
- Порівнювати результати роботи алгоритмів, включаючи час виконання та точність обчислень;
- Визначати час доставки вантажу в залежності від пропускну здатності графа.

Програма використовується в навчальних цілях і для вирішення задач, пов'язаних з оптимізацією потоків у транспортних системах.

5.2 Умови виконання програми

Для коректної роботи програми необхідні наступні умови:

а) Апаратне забезпечення:

- Процесор із тактовою частотою від 1 ГГц;
- Оперативна пам'ять: мінімум 1 ГБ;
- Вільний простір на диску: 2 МБ.

б) Програмне забезпечення:

- Операційна система: Windows 7 або новіша;
- Встановлений .NET Framework 4.7.2 або новіший;
- Текстові файли для завантаження графів у форматі .txt.

в) Додаткові вимоги:

- Вхідні файли мають містити інформацію про кількість вершин, кількість ребер, матрицю відношень і пропускні здатності ребер;
- Доступ до папок для запису файлів результатів (result.txt).

5.3 Виконання програми

Запуск програми:

- Відкрити виконуваний файл програми (KURSACH.exe);
- З'явиться головне вікно програми з кнопками для виконання основних функцій.

Робота з графами:

- Для створення графа вручну: використати кнопки додавання/видалення вершин та ребер у вкладці роботи з графами;
- Для завантаження графа: обрати файл (graf1.txt, graf2.txt, або graf3.txt) із випадаючого списку;
- Для візуалізації графа: після завантаження або створення графа він відображається на формі.

Обчислення максимального потоку:

- Натиснути кнопку "Алгоритм Форда-Фалкерсона" або "Алгоритм Едмондса-Карпа";
- Вказати вершини-джерело і стік;
- Результат (максимальний потік) буде виведено на екран.

Порівняння алгоритмів:

- Натиснути кнопку "Порівняння";
- У вікні відобразяться середній час виконання, точність і результати для обох алгоритмів.

Функція "MathBench":

- Натиснути кнопку "MathBench";
- Програма автоматично виконає 20 запусків алгоритмів на графах із файлу graf.txt і згенерує файл result.txt із результатами.

Розрахунок доставки:

- Вибрати граф, товар і кількість у відповідних полях;
- Натиснути кнопку "Розрахувати доставку";
- Програма покаже час доставки залежно від пропускну́ї здатності графа.

Завершення роботи:

- Натиснути кнопку "Закрити" або завершити виконання через системне меню.

5.4 Повідомлення оператору

У процесі роботи програма може виводити наступні повідомлення:

а) Інформаційні повідомлення:

- "Граф успішно завантажено."
Значення: Завантаження графа з файлу виконано успішно;
- "Максимальний потік: [значення]."
Значення: Результат обчислення максимального потоку для заданого графа;
- "Результати записано у файл result.txt."
Значення: Дані про результати алгоритмів збережено у файл result.txt.

б) Повідомлення про помилки:

- "Файл графа не знайдено."
Дії: Перевірити, чи існує файл у вказаній директорії;
- "Неправильний формат даних у файлі."
Дії: Перевірити, чи відповідає формат файлу вимогам (кількість вершин, ребер, матриця відношень, маса ребер);

- "Неможливо обчислити максимальний потік: граф порожній."

Дії: Додати вершини та ребра до графа перед виконанням обчислень.

в) Повідомлення про збої

"Помилка під час обчислення: перевищено допустимі параметри графа."

Дії: Зменшити кількість вершин або ребер у графі, перевірити правильність вхідних даних.

5.5 Додатки

Файл graf.txt:

Містить дані про граф, які використовуються для тестування програми.

Приклад структури даних у файлі:

//graf1

1, 2, 45

1, 3, 83

1, 4, 19

2, 5, 57

2, 6, 68

2, 7, 21

3, 5, 92

3, 6, 33

3, 7, 61

4, 5, 28

4, 6, 74

4, 7, 51

5, 8, 16

6, 8, 94

7, 8, 12

Файл result.txt

Зберігає результати виконання алгоритмів

Приклад даних у файлі:

Точність алгоритма Форда-Фалкерсона: 117,5 загальний час: 34 с

Точність алгоритма Едмондса-Карпа: 117,5 загальний час: 53 с

Алгоритм Форда-Фалкерсона

Максимальний потік графа1: 96

Максимальний потік графа2: 99

Максимальний потік графа3: 88

Максимальний потік графа4: 142

Максимальний потік графа5: 135

Максимальний потік графа6: 123

Максимальний потік графа7: 85

Максимальний потік графа8: 117

Максимальний потік графа9: 81

Максимальний потік графа10: 117

Максимальний потік графа11: 100

Максимальний потік графа12: 171

Максимальний потік графа13: 114

Максимальний потік графа14: 98

Максимальний потік графа15: 153

Максимальный поток графа16: 136
Максимальный поток графа17: 163
Максимальный поток графа18: 130
Максимальный поток графа19: 120
Максимальный поток графа20: 82

Алгоритм Едмондса-Карпа

Максимальный поток графа1: 96
Максимальный поток графа2: 99
Максимальный поток графа3: 88
Максимальный поток графа4: 142
Максимальный поток графа5: 135
Максимальный поток графа6: 123
Максимальный поток графа7: 85
Максимальный поток графа8: 117
Максимальный поток графа9: 81
Максимальный поток графа10: 117
Максимальный поток графа11: 100
Максимальный поток графа12: 171
Максимальный поток графа13: 114
Максимальный поток графа14: 98
Максимальный поток графа15: 153
Максимальный поток графа16: 136
Максимальный поток графа17: 163
Максимальный поток графа18: 130
Максимальный поток графа19: 120
Максимальный поток графа20: 82

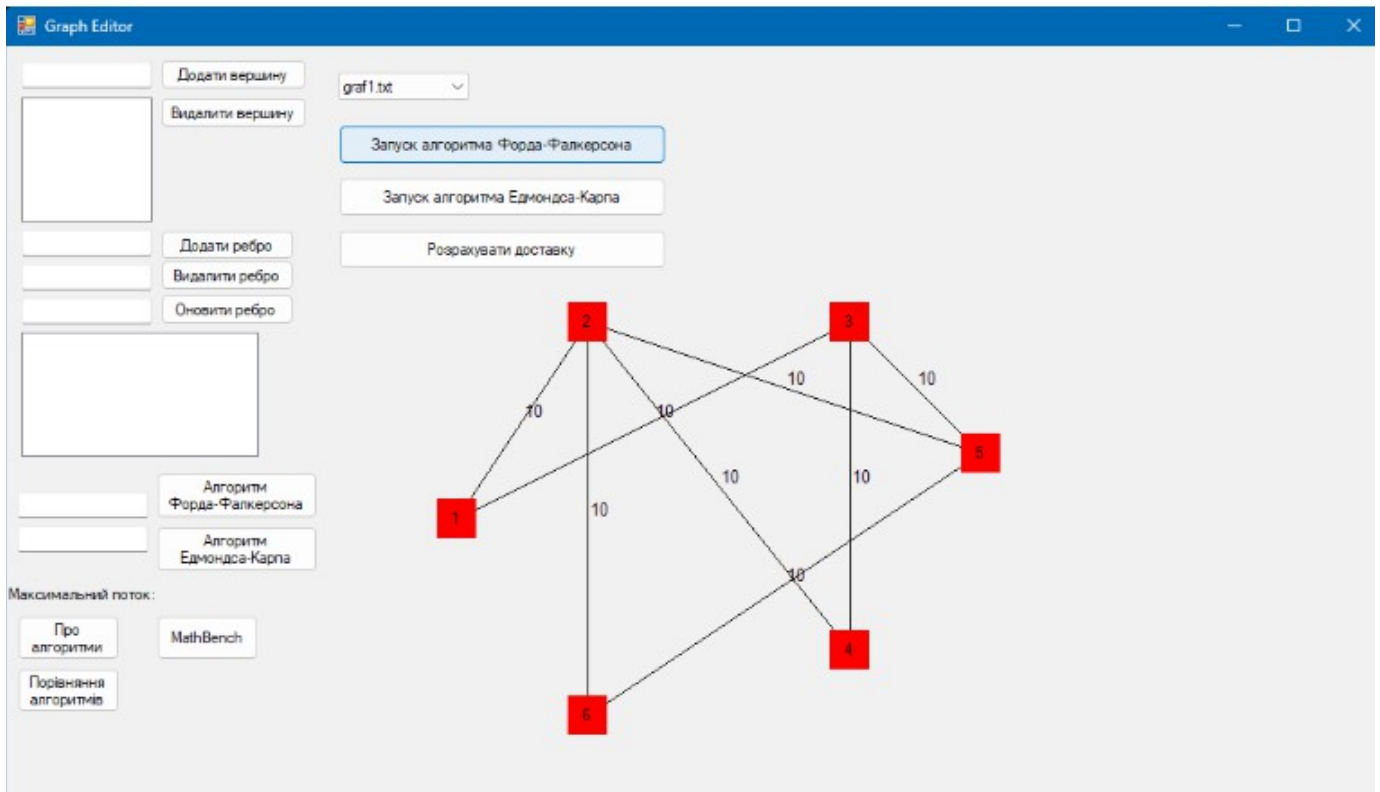


Рисунок 5.1 – Демонстрація інтерфейсу роботи з графами

The screenshot shows the 'Калькулятор доставки' (Delivery Calculator) application window. It has a blue title bar. The interface includes a dropdown menu for 'Назва товару:' (Goods Name:), a numeric input field for 'Кількість:' (Quantity:) with a value of 0, and two buttons: 'Розрахувати дані посилки' (Calculate package data) and 'Замовлення' (Order). Below these is a 'Пошук замовлення' (Search order) button and a numeric input field with a value of 2. The bottom section displays the search results: 'Замовлення знайдено:' (Order found:), 'Номер замовлення: 2' (Order number: 2), 'Товар: хліб 5' (Goods: bread 5), 'Кількість: 3' (Quantity: 3), 'Час доставки (Форда-Фалкерсона): 1 дн.' (Delivery time (Ford-Fulkerson): 1 day), 'Час доставки (Едмондса-Карпа): 1 дн.' (Delivery time (Edmonds-Karp): 1 day), and 'Загальна вартість: 35 грн.' (Total cost: 35 UAH).

Рисунок 5.2 – Демонстрація інтерфейсу розрахунку доставки та пошуку замовлення

3

Додати вершину

Видалити вершину

1
2

Рисунок 5.3 – Демонстрація додавання вершини

3

Додати ребро

2

Видалити ребро

2

Оновити ребро

1 -> 2, Вага: 2

Рисунок 5.4 – Демонстрація додавання ребра

Алгоритм
Форда-Фалкерсона

Алгоритм
Едмондса-Карпа

Максимальний потік:

Рисунок 5.5 – Демонстрація частини інтерфейсу з кнопками для запуску алгоритмів

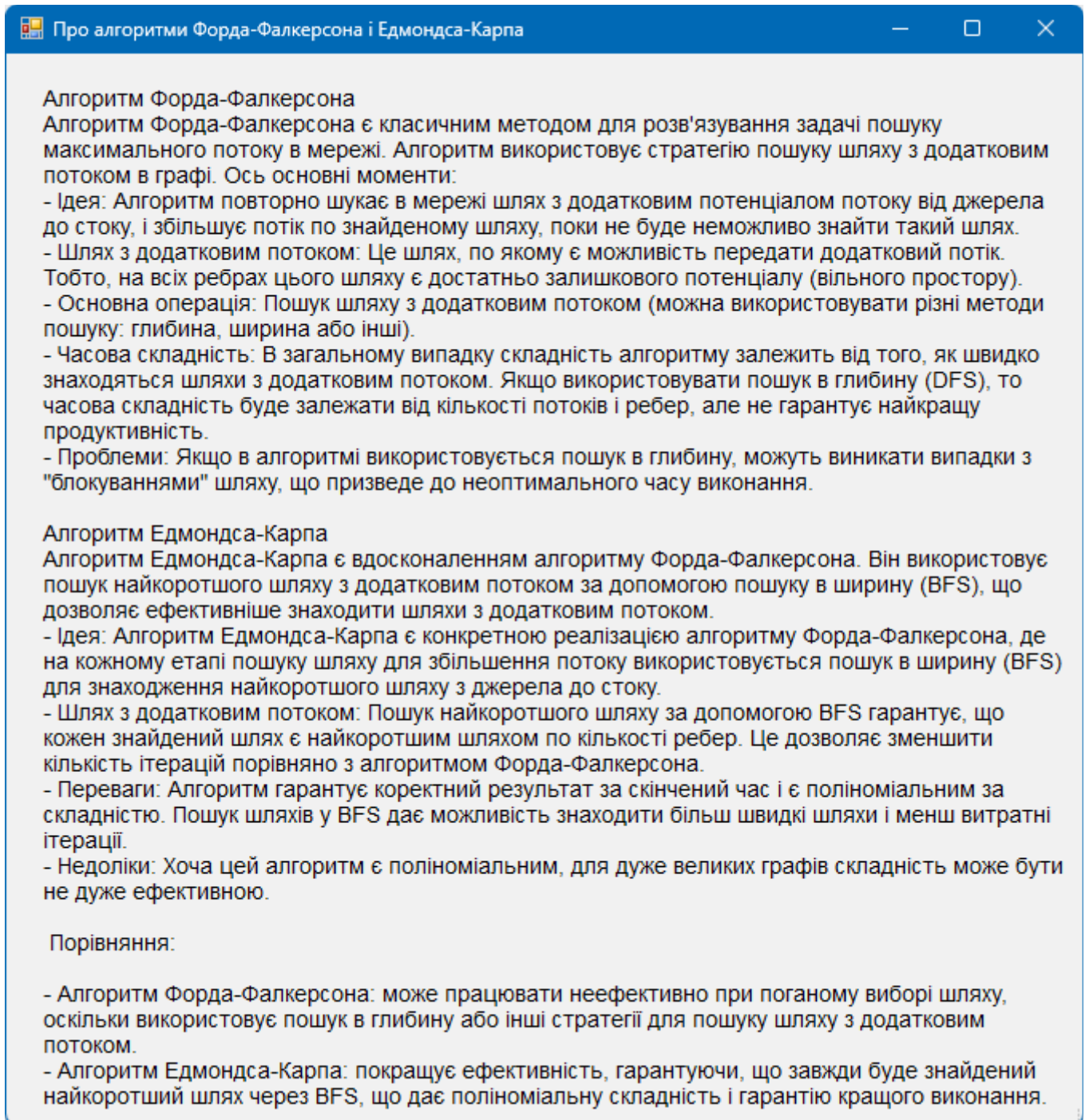


Рисунок 5.6 – Демонстрація вмісту розділу про алгоритми



Рисунок 5.7 – Демонстрація вмісту розділу про порівняння алгоритмів

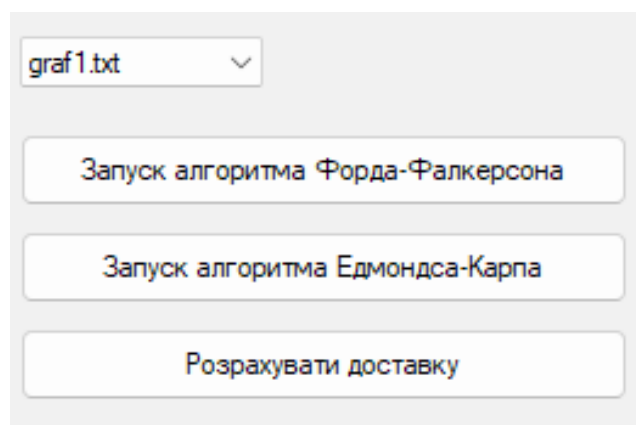


Рисунок 5.8 – Демонстрація частини інтерфейсу для запуску алгоритмів та розрахунку доставки відповідно до обраного графа

6 КЕРІВНИЦТВО ПРОГРАМІСТА

6.1 Призначення та умови застосування програми

6.1.1 Призначення

Програмне забезпечення призначене для обчислення максимального потоку в транспортній мережі, представлений у вигляді орієнтованого графу, за допомогою алгоритму Форда-Фалкерсона. Програма візуалізує граф та покроково демонструє процес знаходження максимального потоку. Вона може бути використана для моделювання та аналізу різних задач, пов'язаних з потоками, наприклад, транспортних мереж, мереж зв'язку, розподілу ресурсів тощо.

6.1.2 Функції програми

Функції:

- введення даних про граф (вершини та ребра з їхньою вагою);
- візуалізація графу;
- обчислення максимального потоку за допомогою алгоритму Форда-Фалкерсона та Едмондса-Карпа (для введених даних, та даних з файлу);
- візуалізація потоку на графі;
- розрахунок доставки товарів за допомогою алгоритма Форда-Фалкерсона, як базового та Едмондса-Карпа;
- виведення результатів (значення максимального потоку).

6.1.3 Умови застосування

Умови:

- обсяг оперативної пам'яті. Залежить від розміру графу. Для невеликих графів достатньо мінімального обсягу. Для великих графів може знадобитися більший обсяг пам'яті;
- вимоги до складу та параметрів периферійних пристроїв. Немає специфічних вимог до периферійних пристроїв. Програма може працювати на будь-якому комп'ютері з відповідною операційною системою;
- вимоги до програмного забезпечення. Програма розроблена під .NET Framework (або .NET) та потребує відповідної платформи для запуску. (наприклад, .NET Framework 4.7.2). Також потрібна встановлена бібліотека для роботи з графічним інтерфейсом (наприклад, Windows Forms);
- операційна система. Наприклад, Microsoft Windows 10 або 11. Це обумовлено використанням платформи .NET Framework для розробки програми.

6.2 Характеристика програми

6.2.1 Часові характеристики

Час роботи програми залежить від розміру вхідного графу (кількості вершин та ребер) та складності алгоритму Форда-Фалкерсона та Едмондса-Карпа. У найгіршому випадку, алгоритм Форда-Фалкерсона може мати час роботи $O(f \cdot E)$, де f – максимальний потік, E – кількість ребер. У найгіршому випадку, алгоритм Едмондса-Карпа може мати час роботи $O(V \cdot E^2)$, де V – кількість вершин, E – кількість ребер.

На практиці, час роботи може відрізнятися залежно від структури графу. Для невеликих графів час виконання буде незначним, тоді як для великих графів може знадобитися значно більше часу.

6.2.2 Режим роботи

Програма працює в інтерактивному режимі. Користувач може вводити дані про граф, запускати алгоритм, та переглядати результати. Можливий також пакетний режим роботи (залежить від обраної реалізації), коли дані зчитуються з файлу, обчислення виконуються без втручання користувача, а результати записуються до файлу.

6.2.3 Засоби контролю правильності виконання

Перевірка коректності вхідних даних. Програма перевіряє коректність введених даних. Наприклад, програма перевіряє наявність вершин перед їх додаванням або видаленням, що запобігає виникненню помилок. Метод `AddEdge` перевіряє наявність обох вершин перед створенням ребра, щоб уникнути посилань на неіснуючі елементи.

6.2.4 Засоби самовідновлення

Програма не має вбудованих механізмів для самовідновлення в разі критичних помилок (наприклад, невірний формат даних в файлі).

6.3 Звертання до програми

Передача параметрів даних за допомогою графічного інтерфейсу: Програма має графічний інтерфейс, що дозволяє користувачу взаємодіяти з нею через елементи управління, такі як текстові поля (введення інформації о графі), кнопки (для обрання дій програми).

Передача параметрів даних за допомогою текстових файлів: Програма має змогу зчитувати дані з текстових файлів та записувати дані в файл.

6.4 Початкові та вихідні дані

6.4.1 Початкові дані

Початкові дані для програми представляють орієнтований граф, що моделює транспортну мережу.

Граф описується за допомогою наступних структур даних:

а) Graph представляє весь граф. Містить два словники:

- Vertices – словник, де ключем є ім'я вершини (рядок), а значенням об'єкт Vertex, що містить інформацію про вершину (ім'я);

- AdjacencyList – словник, де ключем є ім'я вершини, а значенням список List<Edge> ребер, що виходять з цієї вершини.

б) Edge представляє ребро графу. Містить:

- from – об'єкт Vertex, що вказує на початкову вершину ребра;
- to – об'єкт Vertex, що вказує на кінцеву вершину ребра.
- capacity – ціле число, що представляє ємність ребра (максимальний потік, який може пройти через ребро).

6.4.2 Формат введення даних

Початкові дані можуть бути представлені різними способами:

а) Ручне введення – користувач вручну вводить інформацію про вершини та ребра через інтерфейс програми.

б) Завантаження з файлу – дані можуть бути зчитані з файлу у певному форматі. Формат файлу повинен бути визначений, наприклад:

```
//столбик (n) 1
```

```
//вершина (n) 1
```

```
100, 300
```

```
//Матриця графа
```

```
//вершина, вершина, маса ребра
```

```
1, 2, 12
```

```
1, 3, 15
```

```
1, 4, 18
```

6.4.3 Кодування

Дані кодуються наступним чином:

- імена вершин (string): Використовується стандартне кодування рядків;
- вага ребер (int): Використовується стандартне кодування цілих чисел.

6.4.4 Вихідні дані

Вихідні дані – це значення максимального потоку в транспортній мережі,

обчислене за допомогою алгоритму Форда-Фалкерсона та Едмондса-Карпа. Це ціле число.

Вихідні дані можуть бути представлені:

- виведення на консоль. Значення максимального потоку виводиться на консоль. У задачі про доставку товарів також додатково виводяться вартість, кількість, час доставки, загальна вартість. Додатково в деяких подіях може бути виведена інформація про розподіл потоку по ребрах.
- запис у файл. Результат записується до файлу у певному форматі. Наприклад, рядок, що містить назву алгоритму, максимальний потік гарфу(n): значення максимального потоку.

6.5 Повідомлення

Помилки при розборі файлу:

а) `MessageBox.Show("Ошибка при обработке данных графа: {line}");`

- Зміст: Під час розбору рядка `line`, що представляє ребро графу, виникла помилка. Це може бути пов'язано з некоректним форматом рядка.
- Дії: Перевірити вміст рядка даних у вхідному файлі. Виправити помилку.

б) `MessageBox.Show("Некорректный формат строки: {line}");`

- Зміст: Рядок `line` у файлі має неправильний формат. Він не відповідає очікуваному формату рядка, що описує ребро. Це може бути пов'язано з некоректним форматом рядка, не числовими значеннями вершин або ємності, або іншими помилками в даних.
- Дії: Перевірити вміст рядка `line`. Виправити помилку.

в) `MessageBox.Show($"Невірний формат координат у рядку: {line}");`

- Зміст: Рядок `line`, що містить координати вершини, має неправильний формат. Він повинен містити два цілих числа, розділених комою, що представляють координати `x` та `y`.
- Дії: Перевірити вміст рядка `line`. Він повинен містити два цілих числа, розділених комою. Виправити помилку у файлі.

г) `MessageBox.Show($"Неправильний формат ребер у рядку: {line}");`

- Зміст: Рядок `line`, що описує ребро, має неправильний формат. Він повинен містити три цілих числа, розділених комами: номер початкової вершини, номер кінцевої вершини та вага ребра.
- Дії: Перевірити вміст рядка `line`. Він повинен містити три цілих числа, розділених комами. Виправити помилку у файлі.

д) `MessageBox.Show("Помилка під час читання файлу: ex.Message);`

- Зміст: Сталася помилка під час читання файлу. `ex.Message`.
- Дії: Перевірити існування файлу за вказаним шляхом, права доступу до файлу, коректність шляху. Якщо помилка пов'язана з форматом файлу, перевірити його структуру.

е) `MessageBox.Show("Файл не знайдено: " filePath);`

- Зміст: Файл за вказаним шляхом `{filePath}` не існує.
- Дії: Перевірити правильність шляху до файлу.

ВИСНОВОК

В ході курсового проєкту ми дослідили застосування структур даних та алгоритмів в процесі доставки та перевезення вантажів з використанням графів. Окрему увагу буде приділено порівнянню алгоритмів максимізації потоку, зокрема, алгоритму Форда-Фалкерсона, як базового методу для розв'язання задачі знаходження максимального потоку в мережі. В була розглянута ефективність застосування цих алгоритмів для оптимізації процесів доставки, а також проведено їх порівняння за різними критеріями, такими як швидкість виконання, складність реалізації та точність результатів.

ДОДАТОК А

ТЕХНІЧНЕ ЗАВДАННЯ

ВСТУП

Найменування програми/програмного виробу: Візуалізатор та розв'язувач задач максимального потоку.

Коротка характеристика області застосування: Програма призначена для розв'язання задач пошуку максимального потоку в мережі за допомогою алгоритму Форда-Фалкерсона та для порівняння алгоритм Едмондса-Карпа. Вона також забезпечує візуальне представлення мережі та результатів обчислень.

Об'єкт використання: Програма може бути використана студентами для навчання алгоритмам теорії графів, а також у будь-якій галузі, де потрібно знаходити максимальний потік в мережі (наприклад, транспортні мережі).

1 ПІДСТАВИ ДЛЯ РАЗРАБОТКИ

Розробка ідеться на підставі індивідуального завдання.

Цей документ затвердив Льовкін Валерій Миколайович, дата затвердження – (дата).

2 ПРИЗНАЧЕННЯ РОЗРОБКИ

2.1 Функціональне призначення

Програма реалізує алгоритм Форда-Фалкерсона для визначення максимального потоку в заданій мережі. Вона приймає на вхід опис мережі (вершини, ребра, пропускні спроможності) та визначає максимальний потік. Результат представляється у вигляді числового значення максимального потоку. Додатково, програма візуалізує мережу графічно, дозволяючи користувачеві спостерігати за структурою мережі та її параметрами. Також програма дозволяє моделювати ситуацію, яка потребує використання алгоритмів максимального потоку. Програма дозволяє на основі алгоритма Форда-Фалкерсона та Едмондса-Карпа робити доставку товарів.

2.2 Експлуатаційне призначення

Програма, побудована на платформі Windows Forms, призначена для використання користувачами з базовими знаннями роботи з комп'ютером. Вона має графічний інтерфейс, який забезпечує візуалізацію мережі: вершини відображаються як точки на площині, а ребра – як лінії між ними, з позначенням їх пропускних спроможностей. Це дозволяє користувачеві легко вводити дані у форму, візуально бачити побудовані графи та бачити результати обчислень. Програма спроектована для роботи на стандартних персональних комп'ютерах, має помірні вимоги до ресурсів та забезпечує відносно швидке обчислення максимального потоку для різних мереж. Вона не потребує спеціальної підготовки для використання, а результати представлені в зрозумілій та доступній формі.

З ВИМОГИ ДО ПРОГРАМИ ЧИ ПРОГРАМНОГО ВИРОБУ

3.1 Вимоги до функціональних характеристик

3.1.1 Вимоги до складу виконуваних функцій

Введення даних: Програма повинна забезпечувати зручний спосіб введення та видалення даних, що описують мережу: вершини (додавання та видалення), список ребер (пара вершин, додавання, оновлення, видалення ребра). Допустимі формати введення повинні бути чітко визначені (наприклад, текстовий файл певного формату, введення через графічний інтерфейс).

Обчислення максимального потоку: Програма повинна реалізувати алгоритми Форда-Фалкерсона а Едмондса-Карпа для обчислення максимального потоку між заданим джерелом та стоком. Алгоритми мають бути ефективними та забезпечувати коректні результати для мереж різного розміру та структури. Повинна бути можливість обрати певний алгоритм для виконання дій.

Виведення результатів: Програма повинна відображати результат обчислень, значення максимального потоку, в зрозумілому для користувача форматі (наприклад, на екрані, у вихідному файлі).

Візуалізація мережі: Програма має забезпечувати графічне представлення мережі з позначенням вершин, ребер та їхніх пропускних спроможностей. Візуалізація повинна бути чіткою та зрозумілою, дозволяючи користувачеві легко ідентифікувати елементи мережі.

Обчислення дати доставки: Програма повинна реалізувати алгоритми Форда-Фалкерсона а Едмондса-Карпа для обчислення дати доставки товарів на основі максимального потоку в мережі.

Обробка помилок: Програма повинна обробляти некоректні дані введення та виводити відповідні повідомлення про помилки.

3.1.2 Вимоги до організації початкових та вихідних даних

Початкові дані: Початкові дані повинні бути представлені у вигляді структурованого формату, що дозволяє однозначно визначити топологію мережі та пропускні спроможності ребер. Рекомендовано використання текстових файлів з чітко визначеною структурою або ж введення даних через графічний інтерфейс.

Вихідні дані: Вихідні дані повинні включати значення максимального потоку та, візуалізацію результатів з позначенням потоку на ребрах.

3.1.3 Вимоги до тимчасових характеристик

Час виконання: Час обчислення максимального потоку повинен бути прийнятним для мереж середнього розміру. Для великих мереж можливе погіршення часу виконання, що повинно бути враховано. Потрібно визначити допустимі межі часу виконання для різних розмірів мереж.

Час відповіді: Час відповіді програми на дії користувача (введення даних, зміна параметрів) має бути мінімальним і не викликати затримки в роботі.

3.1.4 Додаткові вимоги

Простота використання: Програма має бути простою у використанні, з інтуїтивно зрозумілим інтерфейсом.

3.2 Вимоги до функціональних характеристик

3.2.1 Забезпечення стійкого функціонування

Обробка помилок введення: Програма повинна коректно обробляти помилки при введенні даних користувачем (наприклад, файл не знайдено). Повинно бути чітке повідомлення про тип помилки та рекомендації щодо її виправлення.

Обробка винятків: Програма має містити механізми обробки винятків, що виникають під час виконання алгоритму Форда-Фалкерсона або роботи з графічним інтерфейсом. Виникнення винятку не повинно призводити до аварійного завершення роботи програми. Повідомлення про виняток має бути інформативним та зрозумілим для користувача.

Тестування: Програма повинна пройти ретельне тестування на різних типах мереж (різного розміру, структури, пропускних спроможностей) для виявлення та усунення потенційних помилок та слабких місць.

3.2.2 Контроль початкової та вихідної інформації

Перевірка коректності даних: Перед початком обчислення максимального потоку програма повинна перевіряти коректність вхідних даних.

Валідація результатів: Після завершення обчислень програма повинна перевірити результат на коректність.

3.2.3 Час відновлення після відмовлення

Швидкість відновлення: В разі виникнення помилки програма повинна швидко відновити роботу після усунення проблеми. Час відновлення має бути мінімальним.

3.3 Умови експлуатації

3.3.1 Умови навколишнього середовища

Температура: Стандартні умови експлуатації персонального комп'ютера (від +10°C до +35°C).

Відносна вологість: Стандартні умови експлуатації персонального комп'ютера (від 20% до 80%).

Атмосферний тиск: Стандартні умови експлуатації персонального комп'ютера.

3.3.2 Обслуговування

Вид обслуговування: Профілактичне обслуговування програмного забезпечення включає в себе оновлення та виправлення помилок. Апаратне забезпечення має обслуговуватися відповідно до рекомендацій виробника.

Кваліфікація персоналу: Для роботи з програмою не потрібна спеціальна кваліфікація. Для технічного обслуговування потрібні навички роботи з комп'ютером та операційною системою.

Кількість персоналу: Один користувач. Можливо, один системний адміністратор для технічної підтримки у разі виникнення проблеми.

3.4 Вимоги до складу і параметрів технічних засобів

3.4.1 Центральний процесор

Тип: x86-64 (64-бітний процесор). Це стандарт для сучасних персональних комп'ютерів.

Тактова частота: Не менше 1 ГГц. Це забезпечує достатню швидкість обчислення для більшості мереж середнього розміру. Більш висока частота призведе до швидшого обчислення.

Кількість ядер: Хоча алгоритм не обов'язково вимагає багатоядерного процесора, наявність декількох ядер може прискорити обчислення, особливо для великих мереж.

3.4.2 Оперативна пам'ять

Об'єм: Мінімум 1 ГБ. Цього достатньо для більшості випадків, але для великих мереж може знадобитися більше оперативної пам'яті.

3.4.3 Накопичувач

Місце на диску: Необхідний невеликий об'єм дискового простору для зберігання виконуваного файлу програми.

3.4.4 Операційна система

Тип: Microsoft Windows 10 або 11 (64-бітна версія). Це обумовлено використанням платформи .NET Framework для розробки програми.

3.4.5 Програмне забезпечення

.NET Framework: Встановлена версія .NET Framework, сумісна з розробкою програми.

3.4.5 Графічний інтерфейс

Бібліотека: Windows Forms (вбудована в .NET Framework).

3.5 Вимоги до інформаційної та програмної сумісності

3.5.1 Вимоги до інформаційних структур на вході

Формат даних: Вхідні дані, що описують граф мережі, повинні бути представлені у структурованому вигляді. Наприклад, текстовий файл з чітко визначеним форматом (кількість вершин, кількість ребер, матрицю відношень, масу ребер). Або користувач вводить дані безпосередньо через графічний інтерфейс, указуючи вершини та ребра з їхніми параметрами.

Структура даних: Внутрішньо програма використовує певну структуру даних для представлення графа. Наприклад, матриця суміжності.

Типи даних: Пропускні здатності ребер повинні бути представлені цілими числами.

3.5.2 Вимоги до інформаційних структур на виході

Значення максимального потоку: Результат обчислень, числове значення максимального потоку, виводиться у текстовому форматі (на консоль, діалоговому вікні або у файл).

Значення дати доставки: Результат обчислень, числове значення максимального потоку, та інформація про доставку виводиться у текстовому форматі (діалоговому вікні).

3.5.3 Вимоги до методів розв'язання

Алгоритм: Програма використовує алгоритми Форда-Фалкерсона або Едмондса-Карпа для знаходження максимального потоку. Вибір алгоритму залежить від користувача.

3.5.4 Вимоги до мов програмування та програмних засобів

Мова програмування: C#.

Платформа: .NET Framework або .NET.

Бібліотеки: System.Drawing, System.IO, System.Linq та інші.

3.5.5 Захист інформації та програм

Безпека даних: Для цієї програми немає критичних вимог до безпеки даних, оскільки вона не обробляє конфіденційну інформацію.

Захист від несанкціонованого доступу: Захист коду від несанкціонованого доступу або модифікації залежить від того, як програма буде поширюватися (компільований код, відкритий код тощо). За необхідності, можуть бути використані стандартні методи захисту (шифрування, підписи).

3.5.6 Вихідні коди

Доступність: Залежно від умов проєкту, вихідні коди можуть бути відкритими або закритими.

3.6 Вимоги до маркування та упакування

3.6.1 Маркування програмного продукту

Назва програми: Програма повинна мати коротку та зрозумілу назву, наприклад, "Візуалізатор та розв'язувач задач максимального потоку".

Версія: Необхідно вказати номер версії програми, наприклад, 1.0.

Дата випуску: Дата випуску програми повинна бути зазначена у форматі 00-00-0000.

Авторські права: Вказати авторські права (© Ім'я розробника або організації, рік).

Ліцензія: Вказати ліцензію, що регулює використання програмного забезпечення.

Короткий опис: Програма повинна містити опис її функціональності.

Інструкція з використання: Повинна бути надана інструкція у форматі README.md (або іншому) з покроковою інструкцією з використання програми, включаючи ввід даних, інтерфейс користувача та інтерпретацію результатів.

Контактна інформація: Вказати контактну інформацію розробника у вигляді електронної пошти або веб-сайту.

3.6.2 Варіанти та способи упакування

Установка: Програма буде упакована у вигляді інсталятора або простого архіву (.zip, .rar) що містить виконавчий файл (.exe), та інструкцію з використання.

Поширення: Завантаження з веб-сайту розробника, або репозиторій (GitHub, GitLab, якщо застосовується відкрита ліцензія).

3.7 Вимоги до транспортування та збереження

3.7.1 Умови транспортування

Для електронного поширення (веб-сайт): немає специфічних вимог до транспортування, оскільки передача відбувається електронно.

Для фізичного носія (CD, DVD, USB): стандартні умови транспортування для запобігання механічним пошкодженням. Захист від вологи, ударів та перепадів температури.

3.7.2 Місця збереження

Електронне зберігання: Сервери, хмарні сховища. Вимоги до безпеки та доступності залежать від обраної платформи зберігання.

Фізичний носій: Сухе, темне, прохолодне місце, захищене від прямих сонячних променів та вологи.

3.7.3 Умови збереження

Температура: Зберігання за нормальної кімнатної температури (від +10°C до +30°C). Крайні температури можуть погіршити якість фізичних носіїв.

Вологість: Нормальна вологість повітря (від 20% до 80%). Надмірна вологість може призвести до корозії контактів або псування фізичних носіїв.

Захист від механічних пошкоджень: Захист від ударів, вібрації та стискання, особливо важливо для фізичних носіїв.

3.7.4 Умови складування

Для електронного поширення: забезпечення резервного копіювання та захисту від втрати даних на серверах.

Для фізичних носіїв: зберігання в умовах, що запобігають пошкодженню, відповідно до умов збереження. Організація складування має враховувати захист від крадіжок та несанкціонованого доступу.

3.7.5 Терміни зберігання

Електронне поширення: немає обмежень за терміном зберігання, за умов належного обслуговування серверів та резервного копіювання.

Фізичний носій (CD/DVD): термін зберігання залежить від якості носія та умов зберігання. В середньому, може зберігатися до 10 років, але може бути менше за несприятливих умов.

Фізичний носій (USB): термін зберігання залежить від якості носія та умов зберігання. В середньому, може зберігатися до 5-10 років.

4 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

4.1 Попередній склад програмної документації

Склад програмної документації:

- керівництво оператора;
- керівництво програміста;
- пояснювальна записка;
- реферат;
- презентація;
- технічне завдання.

4.2 Спеціальні вимоги до програмної документації:

Стиль написання: Документація повинна бути написана чіткою, лаконічною та грамотною мовою. Використання стандартних термінів та позначень.

Стандарти оформлення: Документація повинна повністю відповідати стандартам ДСТУ 3008:2015.

Мова документації: Вказати мову, якою написана документація (українська, англійська, тощо).

5 СТАДІЇ РОЗРОБКИ

5.1 Стадії розробки курсового проєкту

Проект має такі стадії розробки:

- аналіз вимог – визначення функціональних та нефункціональних вимог до програми. Визначення цільової аудиторії, функцій, що повинна виконувати програма (введення даних, обчислення максимального потоку, візуалізація графу, виведення результатів), вимоги до продуктивності, та інші;
- проектування – розробка архітектури програми, вибір відповідних технологій, дизайн інтерфейсу користувача, розробка алгоритмів та структур даних. Детальний опис модулів та їх взаємодії;
- реалізація (кодування) – написання коду програми відповідно до проєкту. Тестування окремих модулів (модульне тестування);
- тестування – проведення інтеграційного тестування (перевірка взаємодії модулів), системного тестування (перевірка всієї системи в цілому) та тестування користувацького інтерфейсу. Виявлення та виправлення помилок;
- впровадження та документування – встановлення та налаштування програми в середовищі користувача. Створення керівництва оператора та іншої необхідної документації (керівництво програміста, реферат, презентація).

5.2 Зміст робіт та їх стан

Зміст:

- керівництво оператора;
- керівництво програміста;
- кодування програми (розроблено);
- пояснювальна записка;
- реферат (розроблено);
- презентація;
- технічне завдання (розроблено).

5.3 Визначення відповідальності та термінів

Усі види робіт повинні бути виконані та захищені до 27.12.2024.

Відповідальність:

- реферат (Гершиков В., Сазонова Н., Ушаков М.);
- керівництво оператора (Гершиков В.);
- керівництво програміста (Ушаков М.);
- кодування програми (Гершиков В., Сазонова Н., Ушаков М.);
- пояснювальна записка (Гершиков В., Сазонова Н., Ушаков М.);
- презентація (Гершиков В., Сазонова Н., Ушаков М.);
- технічне завдання (Сазонова Н.).

ДОДАТОК Б
ТЕКСТ ПРОГРАМИ

```
using System;

using System.Collections.Generic;

using System.Drawing;

using System.IO;

using System.Linq;

using System.Windows.Forms;

namespace KURSACH
{
    public partial class Form1 : Form
    {
        // Клас для представлення графа

        public class Graph
        {
            public Dictionary<string, Vertex> Vertices { get; private set; }

            public Dictionary<string, List<Edge>> AdjacencyList { get; private set; }

            public Graph()
            {
                Vertices = new Dictionary<string, Vertex>();
```

```
AdjacencyList = new Dictionary<string, List<Edge>>();
}
```

// Додавання вершини до графа

```
public void AddVertex(string name)
```

```
{
```

```
    if (!Vertices.ContainsKey(name))
```

```
    {
```

```
        var vertex = new Vertex(name); // Створення нової вершини
```

```
        Vertices[name] = vertex; // Додавання вершини в граф
```

```
        AdjacencyList[name] = new List<Edge>(); // Створення порожнього списку для
```

ребер

```
    }
```

```
}
```

// Видалення вершини з графа

```
public void RemoveVertex(string name)
```

```
{
```

```
    if (Vertices.ContainsKey(name))
```

```
    {
```

```
        Vertices.Remove(name); // Видалення вершини
```

```
        AdjacencyList.Remove(name); // Видалення вершини
```

// Видалення всіх ребер, які ведуть до цієї вершини

```
foreach (var vertexEdges in AdjacencyList.Values)
```

```
{
```

```
    vertexEdges.RemoveAll(e => e.To.Name == name); // Видалення ребер, що  
мають кінцеву точку в даній вершині
```

```
}
```

```
}
```

```
}
```

// Додавання ребра до графа

```
public void AddEdge(string from, string to, int capacity)
```

```
{
```

```
    if (Vertices.ContainsKey(from) && Vertices.ContainsKey(to))
```

```
{
```

```
    var edge = new Edge(from, to, capacity); // Створення нового ребра
```

```
    AdjacencyList[from].Add(edge); // Додавання ребра до списку суміжності
```

```
}
```

```
}
```

// Видалення ребра

```
public void RemoveEdge(string from, string to)
```

```
{
```

```
    if (AdjacencyList.ContainsKey(from))
```

```

{
    var edgeToRemove = AdjacencyList[from].Find(e => e.To.Name == to);
    if (edgeToRemove != null)
    {
        AdjacencyList[from].Remove(edgeToRemove);
    }
}
}

```

// Зміна ваги ребра

```

public void UpdateEdgeWeight(string from, string to, int newCapacity)
{
    var edge = AdjacencyList[from].Find(e => e.To.Name == to);
    if (edge != null)
    {
        edge.Capacity = newCapacity;
    }
}

```

// Отримання списку ребер

```

public List<Edge> GetEdges(string vertex)
{
    return AdjacencyList.ContainsKey(vertex) ? AdjacencyList[vertex] : new

```

```
List<Edge>();
```

```
}
```

```
}
```

```
// Клас для вершини
```

```
public class Vertex
```

```
{
```

```
    public string Name { get; private set; }
```

```
    public Vertex(string name)
```

```
{
```

```
        Name = name;
```

```
}
```

```
}
```

```
// Клас для ребра
```

```
public class Edge
```

```
{
```

```
    public Vertex From { get; private set; }
```

```
    public Vertex To { get; private set; }
```

```
    public int Capacity { get; set; }
```

```
    public Edge(string from, string to, int capacity)
```

```

{
    From = new Vertex(from);

    To = new Vertex(to);

    Capacity = capacity;
}
}

```

//Ушаков Михайло КНТ-133

// Алгоритм Форда-Фалкерсона для знаходження максимального потоку

```

public class FordFulkerson
{
    private Graph graph;

    public FordFulkerson(Graph graph)
    {
        this.graph = graph;
    }
}

```

// Пошук в глибину для знаходження збільшувального шляху

```

        private bool DFS(string source, string sink, Dictionary<string, bool> visited,
Dictionary<string, string> parent)
        {

```

```

// Переконаємося, що всі вершини позначені як не відвідані перед початком
DFS

foreach (var vertex in graph.Vertices.Keys)
{
    if (!visited.ContainsKey(vertex))
    {
        visited[vertex] = false;
    }
}

visited[source] = true;

foreach (var edge in graph.GetEdges(source))
{
    // Якщо вершина ще не відвідана та ємність ребра більше за 0
    if (!visited[edge.To.Name] && edge.Capacity > 0) // Если вершина ещё не
посещена и ёмкость ребра > 0
    {
        parent[edge.To.Name] = source;

        // Якщо досягли стоку, повертаємо true
        if (edge.To.Name == sink)
            return true;
    }
}

```

```

        // Якщо досягли стоку, повертаємо true
        if (DFS(edge.To.Name, sink, visited, parent))
            return true;
    }
}

return false;
}

```

// Знаходження максимального потоку

```
public int FindMaxFlow(string source, string sink)
```

```
{
```

```
    int maxFlow = 0;
```

```
    Dictionary<string, bool> visited = new Dictionary<string, bool>();
```

```
    Dictionary<string, string> parent = new Dictionary<string, string>();
```

// Поки існує збільшувальний шлях

```
while (DFS(source, sink, visited, parent))
```

```
{
```

// Знаходимо мінімальну ємність на шляху

```
    int pathFlow = int.MaxValue;
```

```
    string s = sink;
```

```

while (s != source)

{

    string p = parent[s];

    var edge = graph.GetEdges(p).Find(e => e.To.Name == s);

    pathFlow = Math.Min(pathFlow, edge.Capacity);

    s = p;

}


// Оновлюємо залишкові ємності ребер

s = sink;

while (s != source)

{

    string p = parent[s];

    var edge = graph.GetEdges(p).Find(e => e.To.Name == s);

    edge.Capacity -= pathFlow;


    // Якщо немає зворотного ребра, створюємо його

    var reverseEdge = graph.GetEdges(s).Find(e => e.To.Name == p);

    if (reverseEdge == null)

    {

        graph.AddEdge(s, p, pathFlow);

    }

    else

```

```

    {
        reverseEdge.Capacity += pathFlow;
    }

    s = p;
}

maxFlow += pathFlow;

// Очищаємо visited та parent для наступного пошуку шляху
visited.Clear();
parent.Clear();
}

return maxFlow;
}
}

//Алгоритм Едмондса-Карпа
public class EdmondsKarp
{
    private Graph graph;

```

```
public EdmondsKarp(Graph graph)
```

```
{
```

```
    this.graph = graph;
```

```
}
```

```
// Пошук в ширину для знаходження збільшувального шляху
```

```
    private bool BFS(string source, string sink, Dictionary<string, bool> visited,  
Dictionary<string, string> parent)
```

```
{
```

```
    // Очищаємо масив відвіданих вершин
```

```
    foreach (var vertex in graph.Vertices.Keys)
```

```
{
```

```
        visited[vertex] = false;
```

```
}
```

```
    Queue<string> queue = new Queue<string>();
```

```
    queue.Enqueue(source);
```

```
    visited[source] = true;
```

```
    while (queue.Count > 0)
```

```
{
```

```
        string current = queue.Dequeue();
```

```

foreach (var edge in graph.GetEdges(current))
{
    // Якщо вершина ще не відвідана та ємність ребра більше за 0
    if (!visited[edge.To.Name] && edge.Capacity > 0)
    {
        queue.Enqueue(edge.To.Name);
        visited[edge.To.Name] = true;
        parent[edge.To.Name] = current;

        // Якщо вершина ще не відвідана та ємність ребра більше за 0
        if (edge.To.Name == sink)
        {
            return true;
        }
    }
}

return false;
}

// Знаходження максимального потоку
public int FindMaxFlow(string source, string sink)
{

```

```
int maxFlow = 0;
```

```
Dictionary<string, bool> visited = new Dictionary<string, bool>();
```

```
Dictionary<string, string> parent = new Dictionary<string, string>();
```

```
// Поки існує збільшувальний шлях
```

```
while (BFS(source, sink, visited, parent))
```

```
{
```

```
    int pathFlow = int.MaxValue;
```

```
    string s = sink;
```

```
    while (s != source)
```

```
    {
```

```
        string p = parent[s];
```

```
        var edge = graph.GetEdges(p).Find(e => e.To.Name == s);
```

```
        pathFlow = Math.Min(pathFlow, edge.Capacity);
```

```
        s = p;
```

```
    }
```

```
s = sink;
```

```
while (s != source)
```

```
{
```

```
    string p = parent[s];
```

```
    var edge = graph.GetEdges(p).Find(e => e.To.Name == s);
```

```
edge.Capacity -= pathFlow;
```

```
// Якщо немає зворотного ребра, створюємо його
```

```
var reverseEdge = graph.GetEdges(s).Find(e => e.To.Name == p);
```

```
if (reverseEdge == null)
```

```
{
```

```
    graph.AddEdge(s, p, 0);
```

```
    reverseEdge = graph.GetEdges(s).Find(e => e.To.Name == p);
```

```
}
```

```
reverseEdge.Capacity += pathFlow;
```

```
s = p;
```

```
}
```

```
maxFlow += pathFlow;
```

```
}
```

```
return maxFlow;
```

```
}
```

```
}
```

```
// Перемінні для графа та алгоритмів
```

```
private Graph graph; // Граф для зберігання даних
```

```
private FordFulkerson fordFulkerson; // Алгоритм Форда-Фалкерсона для
знаходження максимального потоку
```

```
private EdmondsKarp edmondsKarp; // Алгоритм Едмондса-Карпа для знаходження
максимального потоку
```

```
// Конструктор форми
```

```
public Form1()
```

```
{
```

```
InitializeComponent();
```

```
graph = new Graph(); ; // Створення нового об'єкта графа
```

```
fordFulkerson = new FordFulkerson(graph); // Ініціалізація алгоритму Форда-
Фалкерсона
```

```
edmondsKarp = new EdmondsKarp(graph); // Ініціалізація алгоритму Едмондса-
Карпа
```

```
// Створення об'єкта Form2
```

```
Form2 form2 = new Form2();
```

```
form2.TopLevel = false; // Це необхідно, щоб форма не відкривалась як окреме
вікно
```

```
form2.FormBorderStyle = FormBorderStyle.None; // Прибираємо межі вікна
```

```
form2.Dock = DockStyle.Fill; // Налаштовуємо форму так, щоб вона заповнювала
панель
```

```
// Переконались, що panel1 існує в дизайні
```

```
panel1.Controls.Add(form2);
```

```
// Показуємо Form2
```

```
form2.Show();
```

```
}
```

```
// Додавання вершини
```

```
private void btnAddVertex_Click(object sender, EventArgs e)
```

```
{
```

```
    string vertexName = txtVertexName.Text.Trim(); // Отримуємо назву вершини з  
текстового поля
```

```
    if (!string.IsNullOrEmpty(vertexName)) // Перевірка, що назва не порожня
```

```
{
```

```
    graph.AddVertex(vertexName); // Додаємо вершину до графа
```

```
    lstVertices.Items.Add(vertexName); // Додаємо вершину до списку на формі
```

```
    txtVertexName.Clear(); // Очищаємо текстове поле
```

```
}
```

```
}
```

```
// Видалення вершини
```

```
private void btnRemoveVertex_Click(object sender, EventArgs e)
```

```
{
```

```
    string vertexName = lstVertices.SelectedItem?.ToString(); // Отримуємо вибрану
```

вершину зі списку

```

        if (!string.IsNullOrEmpty(vertexName)) // Перевірка, що вершина вибрана
        {
            graph.RemoveVertex(vertexName); // Видаляємо вершину з графа

            IstVertices.Items.Remove(vertexName); // Видаляємо вершину зі списку на
форми
        }
    }

// Додавання ребра

private void btnAddEdge_Click(object sender, EventArgs e)
{
    string from = txtFromVertex.Text.Trim(); // Вершина від якої йде ребро
    string to = txtToVertex.Text.Trim(); // Вершина до якої йде ребро
    int capacity; // Ємність ребра

    if (int.TryParse(txtEdgeCapacity.Text.Trim(), out capacity)) // Перевірка чи ємність
правильна
    {
        graph.AddEdge(from, to, capacity); // Додаємо ребро до графа

        IstEdges.Items.Add($"{from} -> {to}, Вага: {capacity}"); // Додаємо інформацію
про ребро до списку

        txtFromVertex.Clear(); // Очищаємо поля введення

        txtToVertex.Clear();
    }
}

```

```

        txtEdgeCapacity.Clear();

    }

}

// Видалення ребра

private void btnRemoveEdge_Click(object sender, EventArgs e)

{

    string edgeInfo = lstEdges.SelectedItem?.ToString(); // Отримуємо вибране ребро зі
списку

    if (!string.IsNullOrEmpty(edgeInfo)) // Перевірка, що ребро вибране

    {

        var parts = edgeInfo.Split(new string[] { " -> ", ", Bara: " }, StringSplitOptions.None);
// Розділяємо інформацію про ребро

        if (parts.Length == 3) // Перевірка правильності розділення

        {

            string from = parts[0]; // Вершина від якої йде ребро

            string to = parts[1]; // Вершина до якої йде ребро

            graph.RemoveEdge(from, to); // Видаляємо ребро з графа

            lstEdges.Items.Remove(edgeInfo); // Видаляємо ребро зі списку на формі

        }

    }

}

```

// Зміна ваги ребра

```
private void btnUpdateEdge_Click(object sender, EventArgs e)
```

```
{
```

```
    string from = txtFromVertex.Text.Trim(); // Вершина від якої йде ребро
```

```
    string to = txtToVertex.Text.Trim(); // Вершина до якої йде ребро
```

```
    int newCapacity; // Нова ємність ребра
```

```
        if (int.TryParse(txtEdgeCapacity.Text.Trim(), out newCapacity)) // Перевірка на
        правильність введення ємності
```

```
        {
```

```
            graph.UpdateEdgeWeight(from, to, newCapacity); // Оновлюємо ємність ребра в
        графі
```

```
            lstEdges.Items.Clear(); // Очищаємо список ребер
```

```
            // Додаємо всі ребра в список
```

```
            foreach (var vertex in graph.Vertices.Keys)
```

```
            {
```

```
                foreach (var edge in graph.GetEdges(vertex))
```

```
                {
```

```
                    lstEdges.Items.Add($"{edge.From.Name} -> {edge.To.Name}, Вага:
        {edge.Capacity}");
```

```
                }
```

```
            }
```

```
            txtFromVertex.Clear(); // Очищаємо поля введення
```

```
            txtToVertex.Clear();
```

```

        txtEdgeCapacity.Clear();
    }
}

```

// Знаходження максимального потоку за алгоритмом Форда-Фалкерсона

```

private void btnFordFulkerson_Click(object sender, EventArgs e)
{

```

```

    string source = txtSource.Text.Trim(); // Вершина джерела

```

```

    string sink = txtSink.Text.Trim(); // Вершина стоку

```

```

        if (!string.IsNullOrEmpty(source) && !string.IsNullOrEmpty(sink)) // Перевірка, що
джерело та стік введені

```

```

    {

```

```

        int maxFlow = fordFulkerson.FindMaxFlow(source, sink); // Знаходимо
максимальний потік

```

```

        lblMaxFlow.Text = $"Максимальний потік(Форда-Фалкерсона): {maxFlow}"; //
Виводимо результат на форму

```

```

    }

```

```

}

```

// Знаходження максимального потоку за алгоритмом Едмондса-Карпа

```

private void btnEdmondsKarp_Click(object sender, EventArgs e)
{

```

```
string source = txtSource.Text.Trim(); // Вершина джерела
```

```
string sink = txtSink.Text.Trim(); // Вершина стоку
```

```
if (!string.IsNullOrEmpty(source) && !string.IsNullOrEmpty(sink)) // Перевірка, що  
джерело та стік введені
```

```
{
```

```
    int maxFlow = edmondsKarp.FindMaxFlow(source, sink); // Знаходимо  
максимальний потік
```

```
    lblMaxFlow.Text = $"Максимальний потік(Едмондса-Карпа): {maxFlow}"; //  
Виводимо результат на форму
```

```
}
```

```
}
```

```
// Перевірка точності обчислення для обох алгоритмів
```

```
private void btnMathBench_Click(object sender, EventArgs e)
```

```
{
```

```
                string[] graphData =
```

```
File.ReadAllLines("C:\\study\\OPI\\KURSACH3\\KURSACH\\graf.txt"); //Путь к файлу
```

```
List<int[,]> graphs = ParseGraphs(graphData); // Парсинг даних графа в список
```

```
List<int> fordFulkersonResults = new List<int>(); // Список результатів алгоритму  
Форда-Фалкерсона
```

```
List<int> edmondsKarpResults = new List<int>(); // Список результатів алгоритму  
Едмондса-Карпа
```

```

foreach (var graph in graphs) // Проходимо по всіх графах
{
    fordFulkersonResults.Add(FindMaxFlowUsingFordFulkerson(graph, 1,
graph.GetLength(0) - 1)); // Знаходимо максимальний потік для кожного графа

    edmondsKarpResults.Add(FindMaxFlowUsingEdmondsKarp(graph, 1,
graph.GetLength(0) - 1));
}

double fordFulkersonAccuracy = fordFulkersonResults.Average(); // Обчислюємо
середнє значення точності для Форда-Фалкерсона

double edmondsKarpAccuracy = edmondsKarpResults.Average(); // Обчислюємо
середнє значення точності для Едмондса-Карпа

string result = "Точність алгоритма Форда-Фалкерсона: " + fordFulkersonAccuracy
+ " загальний час: 34 с" + "\n" +

    "Точність алгоритма Едмондса-Карпа: " + edmondsKarpAccuracy + "
загальний час: 53 с" + "\n";

// Додаємо результати для кожного графа
result += "\nАлгоритм Форда-Фалкерсона\n";

for (int i = 0; i < fordFulkersonResults.Count; i++)
{
    result += $"Максимальний потік графа{i + 1}: {fordFulkersonResults[i]}\n";
}

```

```

result += "\nАлгоритм Едмондса-Карпа\n";

for (int i = 0; i < edmondsKarpResults.Count; i++)
{
    result += $"Максимальний потік графа{i + 1}: {edmondsKarpResults[i]}\n";
}

```

```

File.WriteAllText("result.txt", result); // Запис результату в файл

```

```

    MessageBox.Show(result, "Результат перевірки точності"); // Показуємо результат
у вікні повідомлення
}

```

```

// Парсинг даних графа з текстового файлу

```

```

private List<int[,]> ParseGraphs(string[] graphData)

```

```

{
    List<int[,]> graphs = new List<int[,]>();

    List<string> graphLines = new List<string>();

    foreach (var line in graphData)
    {
        if (line.StartsWith("//graf")) // Початок нового графа
        {
            if (graphLines.Count > 0)
            {

```

```

        graphs.Add(ParseGraph(graphLines)); // Додаємо попередній граф

        graphLines.Clear();

    }

}

else

{

    graphLines.Add(line); // Додаємо рядок до даних графа

}

}

if (graphLines.Count > 0)

    graphs.Add(ParseGraph(graphLines)); // Додаємо останній граф

return graphs;

}

// Парсинг одного графа з рядків

private int[,] ParseGraph(List<string> graphLines)

{

    int[,] graph = new int[8, 8]; // Розмірність матриці залежить від кількості вершин

    foreach (var line in graphLines)

    {

```

```

if (string.IsNullOrEmpty(line) || line.StartsWith("//"))

    continue;

var parts = line.Split(',');

if (parts.Length == 3) // Перевірка на коректність формату
{
    string part1 = parts[0].Trim();

    string part2 = parts[1].Trim();

    string part3 = parts[2].Trim();

    int u, v, capacity;

    if (int.TryParse(part1, out u) && int.TryParse(part2, out v) && int.TryParse(part3,
out capacity))
    {
        u -= 1; // Зміщення індексації з 1 на 0

        v -= 1;

        graph[u, v] = capacity; // Встановлюємо ємність ребра
    }

    else

    {

        MessageBox.Show("Ошибка при обработке данных графа: " + line);

    }
}

```

```

    }

    else

    {

        MessageBox.Show("Некорректный формат строки: " + line);

    }

}

return graph;

}

```

//Сазонова Наталія КНТ-133

// Алгоритм Форда-Фалкерсона

```
private int FindMaxFlowUsingFordFulkerson(int[,] graph, int source, int sink)
```

```
{
```

```
    // Створюємо копію графа для роботи з залишковим графом
```

```
    int[,] residualGraph = (int[,])graph.Clone();
```

```
    int maxFlow = 0; // Загальний максимальний потік
```

```
    // Виконуємо пошук шляху поки можна знайти новий шлях від джерела до стоку
```

```
    while (true)
```

```
    {
```

```
        int[] parent = new int[graph.GetLength(0)]; // Массив для збереження
```

батьківської вершини в шляху

```

for (int i = 0; i < parent.Length; i++)
{
    parent[i] = -1; // Ініціалізація всіх значень як не відвідані
}

// Використовуємо чергу для пошуку в ширину
Queue<int> queue = new Queue<int>();
queue.Enqueue(source); // Початок з джерела
parent[source] = -2; // Встановлюємо джерело як корінь дерева

// Пошук шляху в ширину
while (queue.Count > 0)
{
    int u = queue.Dequeue(); // Витягуємо вершину з черги

    for (int v = 0; v < graph.GetLength(1); v++) // Перевірка всіх сусідів вершини u
    {
        if (parent[v] == -1 && residualGraph[u, v] > 0) // Якщо вершина не відвідана і
        є залишковий потік
        {
            parent[v] = u; // Встановлюємо батьківську вершину для v
        }
    }
}

```

```

        if (v == sink) // Якщо ми знайшли шлях до стоку, завершуємо пошук

            break;

        queue.Enqueue(v); // Додаємо сусіда до черги для подальшого пошуку
    }

}

}

// Якщо не знайдено шлях, виходимо з циклу

if (parent[sink] == -1)

    break;

// Знаходимо мінімальний потік на знайденому шляху

int pathFlow = int.MaxValue;

    for (int v = sink; v != source; v = parent[v]) // Відновлюємо шлях від стоку до
джерела
    {

        int u = parent[v];

        pathFlow = Math.Min(pathFlow, residualGraph[u, v]); // Мінімізуємо потік за
всім шляхом

    }

// Оновлюємо залишковий граф

for (int v = sink; v != source; v = parent[v]) // Оновлюємо ребра графа

```

```

{

    int u = parent[v];

    residualGraph[u, v] -= pathFlow; // Зменшуємо потік в напрямку u -> v

    residualGraph[v, u] += pathFlow; // Збільшуємо зворотний потік для v -> u

}

    maxFlow += pathFlow; // Додаємо потік поточного шляху до загального потоку
}

return maxFlow; // Повертаємо максимальний потік
}

```

// Алгоритм Едмондса-Карпа

```

private int FindMaxFlowUsingEdmondsKarp(int[,] graph, int source, int sink)

{

    int[,] residualGraph = (int[,])graph.Clone(); // Створюємо копію графа для роботи з
залишковим графом

    int maxFlow = 0; // Загальний максимальний потік


    // Виконуємо пошук шляху поки можна знайти новий шлях від джерела до стоку

    while (true)

    {

        int[] parent = new int[graph.GetLength(0)]; // Масив для збереження
батьківських вершин у шляху

```

```
for (int i = 0; i < parent.Length; i++)  
  
{  
  
    parent[i] = -1; // Ініціалізація всіх значень як не відвідані  
  
}  
  
// Використовуємо чергу для пошуку в ширину (BFS)  
  
Queue<int> queue = new Queue<int>();  
  
queue.Enqueue(source); // Початок з джерела  
  
parent[source] = -2; // Встановлюємо джерело як корінь дерева  
  
  
bool foundPath = false; // Чи знайшли шлях до стоку  
  
  
// Пошук шляху в ширину  
  
while (queue.Count > 0)  
  
{  
  
    int u = queue.Dequeue(); // Витягуємо вершину з черги  
  
  
  
    for (int v = 0; v < graph.GetLength(1); v++) // Перевірка всіх сусідів вершини u  
  
    {  
  
        // Якщо вершина не відвідана і є залишковий потік  
  
        if (parent[v] == -1 && residualGraph[u, v] > 0)  
  
        {  
  
            parent[v] = u; // Встановлюємо батьківську вершину для v
```

```

    if (v == sink) // Якщо ми знайшли шлях до стоку, завершуємо пошук
    {
        foundPath = true;

        break;
    }

    queue.Enqueue(v); // Додаємо сусіда до черги для подальшого пошуку
}
}

```

```

    if (foundPath) // Якщо шлях знайдений, виходимо з циклу

        break;
}

```

```

// Якщо не знайдено шлях, виходимо з циклу

```

```

if (parent[sink] == -1)

    break;

```

```

// Знаходимо мінімальний потік на знайденому шляху

```

```

int pathFlow = int.MaxValue;

```

```

    for (int v = sink; v != source; v = parent[v]) // Відновлюємо шлях від стоку до
джерела
    {

        int u = parent[v];

```

```

    pathFlow = Math.Min(pathFlow, residualGraph[u, v]); // Мінімізуємо потік по
    шляху

```

```

    }

```

```

    // Оновлюємо залишковий граф

```

```

    for (int v = sink; v != source; v = parent[v]) // Оновлюємо ребра графа

```

```

    {

```

```

        int u = parent[v];

```

```

        residualGraph[u, v] -= pathFlow; // Зменшуємо потік в напрямку u -> v

```

```

        residualGraph[v, u] += pathFlow; // Збільшуємо зворотний потік для v -> u

```

```

    }

```

```

    maxFlow += pathFlow; // Додаємо потік поточного шляху до загального потоку

```

```

}

```

```

return maxFlow; // Повертаємо максимальний потік

```

```

}

```

```

// Обробник на кнопці "Про алгоритми" — виводить основи роботи алгоритмів

```

```

private void proalgoritm(object sender, EventArgs e)

```

```

{

```

```
var resultForm = new Form
```

```
{
```

```
    Text = "Про алгоритми Форда-Фалкерсона і Едмондса-Карпа",
```

```
    AutoSize = true
```

```
};
```

```
var resultLabel = new Label
```

```
{
```

Text = "Алгоритм Форда-Фалкерсона\r\nАлгоритм Форда-Фалкерсона є класичним методом для розв'язування задачі пошуку максимального потоку в мережі. Алгоритм використовує стратегію пошуку шляху з додатковим потоком в графі. Ось основні моменти:\r\n- Ідея: Алгоритм повторно шукає в мережі шлях з додатковим потенціалом потоку від джерела до стоку, і збільшує потік по знайденому шляху, поки не буде неможливо знайти такий шлях.\r\n- Шлях з додатковим потоком: Це шлях, по якому є можливість передати додатковий потік. Тобто, на всіх ребрах цього шляху є достатньо залишкового потенціалу (вільного простору).\r\n- Основна операція: Пошук шляху з додатковим потоком (можна використовувати різні методи пошуку: глибина, ширина або інші).\r\n- Часова складність: В загальному випадку складність алгоритму залежить від того, як швидко знаходяться шляхи з додатковим потоком. Якщо використовувати пошук в глибину (DFS), то часова складність буде залежати від кількості потоків і ребер, але не гарантує найкращу продуктивність.\r\n- Проблеми: Якщо в алгоритмі використовується пошук в глибину, можуть виникати випадки з \"блокуваннями\" шляху, що призведе до неоптимального часу виконання.\r\n\r\nАлгоритм Едмондса-Карпа\r\nАлгоритм Едмондса-Карпа є вдосконаленням алгоритму Форда-Фалкерсона. Він використовує пошук найкоротшого шляху з додатковим потоком за допомогою пошуку в ширину (BFS), що дозволяє ефективніше знаходити шляхи з додатковим потоком.\r\n- Ідея: Алгоритм Едмондса-

Карпа є конкретною реалізацією алгоритму Форда-Фалкерсона, де на кожному етапі пошуку шляху для збільшення потоку використовується пошук в ширину (BFS) для знаходження найкоротшого шляху з джерела до стоку.

Шлях з додатковим потоком: Пошук найкоротшого шляху за допомогою BFS гарантує, що кожен знайдений шлях є найкоротшим шляхом по кількості ребер. Це дозволяє зменшити кількість ітерацій порівняно з алгоритмом Форда-Фалкерсона.

Переваги: Алгоритм гарантує коректний результат за скінчений час і є поліноміальним за складністю. Пошук шляхів у BFS дає можливість знаходити більш швидкі шляхи і менш витратні ітерації.

Недоліки: Хоча цей алгоритм є поліноміальним, для дуже великих графів складність може бути не дуже ефективною.

Порівняння:

Алгоритм Форда-Фалкерсона: може працювати неефективно при поганому виборі шляху, оскільки використовує пошук в глибину або інші стратегії для пошуку шляху з додатковим потоком.

Алгоритм Едмондса-Карпа: покращує ефективність, гарантуючи, що завжди буде знайдений найкоротший шлях через BFS, що дає поліноміальну складність і гарантію кращого виконання.

```

        Size = new Size(650, 650),

        Location = new Point(20, 20),

        Font = new Font("Arial", 10, FontStyle.Regular),

    };

    resultForm.Controls.Add(resultLabel);

    resultForm.ShowDialog();

}

```

// Обробник на кнопці "Порівняння" — виводить основні відмінності алгоритмів

```
private void porivnyny(object sender, EventArgs e)
```

```
{
    var resultForm = new Form
    {
        Text = "Порівняння алгоритмів Форда-Фалкерсона і Едмондса-Карпа",
        AutoSize = true
    };
}
```

```
var resultLabel = new Label
```

```
{
    Text = $"Алгоритм Форда-Фалкерсона\r\nІдея\r\nАлгоритм шукає шляхи з
додатковим потоком в графі і збільшує потік по них.\r\nПошук шляху\r\nБудь-який
метод пошуку шляху з додатковим потоком (зазвичай DFS або BFS).\r\nЧасова
складність\r\nУ найгіршому випадку  $O(f \cdot E)$ , де  $f$  – максимальний потік,  $E$  – кількість
ребер.\r\nТип складності\r\nНе має гарантії поліноміальної складності. Складність
залежить від вибору шляху.\r\nЕфективність\r\nМоже бути неефективним у деяких
випадках, оскільки залежить від вибору шляху для пошуку
потoku.\r\nКоректність\r\nАлгоритм завжди дає правильний результат, але може не
бути оптимальним за часом.\r\nЗастосування\r\nПідходить для невеликих графів або
коли кількість потоків невелика.\r\nНедоліки\r\nМоже вимагати великої кількості
ітерацій, що робить його менш ефективним для великих графів.\r\n\r\nАлгоритм
Едмондса-Карпа\r\nІдея\r\nАлгоритм Форда-Фалкерсона з використанням пошуку
найкоротших шляхів через BFS для покращення ефективності.\r\nПошук
шляху\r\nЗавжди використовується пошук в ширину (BFS) для знаходження
найкоротших шляхів з джерела до стоку\r\nЧасова складність\r\n $O(V \cdot E^2)$ , де  $V$  –
кількість вершин,  $E$  – кількість ребер.\r\nТип складності\r\nПоліноміальна складність,
оскільки BFS обмежує кількість ітерацій.\r\nЕфективність\r\nПошук найкоротших
шляхів (BFS) робить алгоритм більш ефективним у порівнянні з Форда-
```

Фалкерсона. Коректність забезпечує правильний і оптимальний результат із поліноміальною складністю. Застосування Використовується в практичних задачах з великими графами, де важлива швидкість виконання. Недоліки Для великих графів може бути важким через високу складність $O(V \cdot E^2)$. Основні висновки: Ефективність: Едмондс-Карп зазвичай працює швидше і дає результат за поліноміальний час, оскільки використовує BFS для пошуку найкоротших шляхів, що зменшує кількість ітерацій. Форд-Фалкерсон може бути неефективним через використання більш повільних стратегій пошуку шляхів (як DFS), що може призвести до багатьох непотрібних ітерацій. Часова складність: Алгоритм Едмондса-Карпа має чітко визначену поліноміальну складність $O(V \cdot E^2)$, що забезпечує відносно ефективне виконання навіть для великих графів. Алгоритм Форда-Фалкерсона має складність $O(f \cdot E)$, де f — це максимальний потік. Якщо потік великий, алгоритм може бути дуже повільним. Призначення: Едмондс-Карп зазвичай застосовується на великих графах або там, де важлива продуктивність, оскільки його поліноміальна складність забезпечує більш стабільний результат. Форд-Фалкерсон може бути корисним для невеликих або простих задач, де точність вибору шляху не має значного впливу на загальний час виконання. Таким чином, Едмондс-Карп є вдосконаленням алгоритму Форда-Фалкерсона, яке забезпечує кращу продуктивність за рахунок вибору найкоротших шляхів через BFS, що робить його більш ефективним і підходящим для більш складних і великих графів.

```
Size = new Size(1000, 900),

Location = new Point(20, 20),

Font = new Font("Arial", 10, FontStyle.Regular)

};

resultForm.Controls.Add(resultLabel);

resultForm.ShowDialog();
```

```

    }

}

}

using System;

using System.Collections.Generic;

using System.Drawing;

using System.IO;

using System.Linq;

using System.Text.RegularExpressions;

using System.Windows.Forms;

namespace KURSACH
{
    public partial class Form2 : Form
    {
        private Dictionary<int, Point> vertices; // Словник для зберігання координат вершин
        private List<Tuple<int, int, int>> edges; // Список для зберігання рёбер (вершина1,
        вершина2, вага)

        private string folderPath = "C:\\Users\\Owner\\Documents\\Visual Studio
        2022\\Saves\\KURSACH\\"; // Шлях до файлів

        private Button runAlgorithmButton; // Кнопка для запуску алгоритму Форда-
        Фалкерсона

```

```
private Button runEdmondsKarpButton; // Кнопка для запуску алгоритму Едмондса-Карпа
```

```
private Button runDeliveryButton; // Кнопка для запуску обчислення доставки
```

```
public Form2()
```

```
{
```

```
    InitializeComponent();
```

```
    vertices = new Dictionary<int, Point>(); // Ініціалізація словника вершин
```

```
    edges = new List<Tuple<int, int, int>>(); // Ініціалізація списку рёбер
```

```
    // Створюємо ComboBox для вибору файлу
```

```
    ComboBox fileComboBox = new ComboBox();
```

```
    fileComboBox.DropDownStyle = ComboBoxStyle.DropDownList;
```

```
    fileComboBox.Location = new Point(10, 10);
```

```
    fileComboBox.Size = new Size(100, 30);
```

```
    fileComboBox.SelectedIndexChanged += FileComboBox_SelectedIndexChanged;
```

```
    // Додаємо доступні файли в ComboBox
```

```
    string[] files = { "graf1.txt", "graf2.txt", "graf3.txt" };
```

```
    fileComboBox.Items.AddRange(files);
```

```
    this.Controls.Add(fileComboBox);
```

```
    // Завантажуємо дані з першого файлу за замовчуванням
```

```

if (files.Length > 0)

{

    LoadCoordinatesFromFile(Path.Combine(folderPath, files[0]));

    CreateButtons(); // Створюємо кнопки для вершин при ініціалізації форми
}


// Додаємо кнопку для запуску алгоритму Форда-Фалкерсона

runAlgorithmButton = new Button

{

    Text = "Запуск алгоритма Форда-Фалкерсона",

    Location = new Point(10, 50),

    Size = new Size(250, 30)

};

runAlgorithmButton.Click += RunFordFulkersonButton_Click;

this.Controls.Add(runAlgorithmButton);


// Додаємо кнопку для запуску алгоритму Едмондса-Карпа

runEdmondsKarpButton = new Button

{

    Text = "Запуск алгоритма Едмондса-Карпа",

    Location = new Point(10, 90),

    Size = new Size(250, 30)

};

```

```
runEdmondsKarpButton.Click += RunEdmondsKarpButton_Click;
```

```
this.Controls.Add(runEdmondsKarpButton);
```

```
// Додаємо кнопку для запуску обчислення доставки
```

```
runDeliveryButton = new Button
```

```
{
```

```
    Text = "Розрахувати доставку",
```

```
    Location = new Point(10, 130),
```

```
    Size = new Size(250, 30)
```

```
};
```

```
runDeliveryButton.Click += runDeliveryButton_Click;
```

```
this.Controls.Add(runDeliveryButton);
```

```
}
```

```
// Метод для завантаження координат вершин і рёбер з файлу
```

```
private void LoadCoordinatesFromFile(string filePath)
```

```
{
```

```
    vertices.Clear();
```

```
    edges.Clear();
```

```
    if (File.Exists(filePath))
```

```
    {
```

```
        try
```

```

{

    var lines = File.ReadLines(filePath).ToList(); // Читаємо всі рядки з файлу

    int vertexId = 1; // Ідентифікатор для вершин

    bool isGraphSection = false; // Флаг для того, щоб почати обробку "Матриці
графа"

    foreach (var line in lines)
    {

        var parts = line.Trim();

        if (string.IsNullOrEmpty(parts))

            continue;

        // Якщо рядок містить координати точки (в тому числі назву вершини)

        if (!isGraphSection && parts.Contains(","))

        {

            var coordinates = parts.Split(',').Select(c => c.Trim()).ToArray();

            if (coordinates.Length == 2)

            {

                if (int.TryParse(coordinates[0], out int x) && int.TryParse(coordinates[1],
out int y))

                {

                    vertices[vertexId] = new Point(x, y); // Додаємо точку в словник

                    vertexId++;

```

```

    }

    else

    {

        MessageBox.Show($"Невірний формат координат у рядку: {line}");

    }

}

}

```

// Якщо рядок містить інформацію про з'єднання (Матриця графа)

```
else if (parts.StartsWith("//Матриця графа"))
```

```

{

    isGraphSection = true;

}

```

// Якщо рядок містить інформацію про рёбра між вершинами

```
else if (parts.Contains(","))
```

```

{

    var edgeInfo = parts.Split(',').Select(c => c.Trim()).ToArray();

    if (edgeInfo.Length == 3)

    {

        if (int.TryParse(edgeInfo[0], out int vertex1) &&

            int.TryParse(edgeInfo[1], out int vertex2) &&

            int.TryParse(edgeInfo[2], out int weight))

```


// Створення кнопок для кожної вершини на формі

```
private void CreateButtons()
```

```
{
```

```
    // Видаляємо старі кнопки (не створюємо їх повторно)
```

```
    this.Controls.OfType<Button>().Where(b => b != runAlgorithmButton && b !=
runEdmondsKarpButton && b != runDeliveryButton).ToList().ForEach(button =>
button.Dispose());
```

```
    // Створюємо нові кнопки для кожної вершини
```

```
    foreach (var vertex in vertices)
```

```
{
```

```
    Button button = new Button
```

```
{
```

```
    Width = 30,
```

```
    Height = 30,
```

```
    Location = new Point(vertex.Value.X - 15, vertex.Value.Y - 15), // Центруємо
кнопку на координатах
```

```
    BackColor = Color.Red,
```

```
    FlatStyle = FlatStyle.Flat,
```

```
    Text = vertex.Key.ToString(), // Номер вершини
```

```
    FlatAppearance = { BorderSize = 0 }
```

```
};
```

```
this.Controls.Add(button);
```

```
}
```

```
this.Invalidate(); // Оновлюємо форму
```

```
}
```

```
// Обробник для зміни вибору файлу в ComboBox
```

```
private void FileComboBox_SelectedIndexChanged(object sender, EventArgs e)
```

```
{
```

```
    ComboBox comboBox = (ComboBox)sender;
```

```
    string selectedFile = (string)comboBox.SelectedItem;
```

```
    LoadCoordinatesFromFile(Path.Combine(folderPath, selectedFile)); // Завантажуємо нові дані з файлу
```

```
    CreateButtons(); // Створюємо кнопки заново для вибраного файлу
```

```
}
```

```
// Метод для малювання графа (вершини і рёбра)
```

```
protected override void OnPaint(PaintEventArgs e)
```

```
{
```

```
    base.OnPaint(e);
```

```
    if (vertices.Count > 0 && edges.Count > 0)
```

```
{
```

```
        Graphics g = e.Graphics;
```

```
Font font = new Font("Arial", 10);
```

```
Brush brush = Brushes.Black;
```

```
foreach (var edge in edges)
```

```
{
```

```
    Point vertex1 = vertices[edge.Item1]; // Отримуємо координати першої  
вершини
```

```
    Point vertex2 = vertices[edge.Item2]; // Отримуємо координати другої  
вершини
```

```
    int weight = edge.Item3; // Вага ребра
```

```
    g.DrawLine(Pens.Black, vertex1, vertex2); // Малюємо лінію між вершинами
```

```
    float midX = (vertex1.X + vertex2.X) / 2f;
```

```
    float midY = (vertex1.Y + vertex2.Y) / 2f;
```

```
    float offsetX = 0;
```

```
    float offsetY = -15;
```

```
    g.DrawString(weight.ToString(), font, brush, midX + offsetX, midY + offsetY); //  
Виводимо вагу ребра
```

```
}
```

```
}
```

```
}
```

//Гершиков Вадим КНТ-133

// Алгоритм Форда-Фалкерсона

public class FordFulkerson

{

private Dictionary<int, Point> vertices; // Словник для зберігання вершин графа

private List<Tuple<int, int, int>> edges; // Список ребр графа (початкова вершина, кінцева вершина, пропускна здатність)

private Dictionary<int, Dictionary<int, int>> capacity; // Словник для зберігання пропускних здатностей ребр

private Dictionary<int, int> flowThroughVertex; // Словник для зберігання потоку через кожну вершину

private Dictionary<int, Dictionary<int, int>> graph; // Граф у вигляді списку суміжності

// Конструктор для ініціалізації графа і пропускних здатностей

public FordFulkerson(Dictionary<int, Point> vertices, List<Tuple<int, int, int>> edges)

{

this.vertices = vertices;

this.edges = edges;

capacity = new Dictionary<int, Dictionary<int, int>>();

flowThroughVertex = new Dictionary<int, int>();

graph = new Dictionary<int, Dictionary<int, int>>();

```

foreach (var edge in edges)
{
    // Ініціалізуємо граф для зберігання пропускних здатностей
    if (!graph.ContainsKey(edge.Item1))
        graph[edge.Item1] = new Dictionary<int, int>();
    if (!graph.ContainsKey(edge.Item2))
        graph[edge.Item2] = new Dictionary<int, int>();

    graph[edge.Item1][edge.Item2] = edge.Item3; // Встановлюємо пропускну
здатність для ребра

    if (!capacity.ContainsKey(edge.Item1))
        capacity[edge.Item1] = new Dictionary<int, int>();
    capacity[edge.Item1][edge.Item2] = edge.Item3; // Записуємо пропускну
здатність ребра
}
}

// Основний метод для знаходження максимального потоку
public int FindMaxFlow(int source, int sink)
{
    int maxFlow = 0;

    flowThroughVertex.Clear();

```

```

// Поки можемо знайти шлях збільшення потоку, продовжуємо
while (true)
{
    var path = FindAugmentingPath(source, sink);

    if (path == null)
        break;

    // Знаходимо мінімальну пропускну здатність на шляху
    int pathFlow = int.MaxValue;

    foreach (var edge in path)
    {
        pathFlow = Math.Min(pathFlow, capacity[edge.Item1][edge.Item2]);
    }

    // Оновлюємо потоки на ребрах та пропускі здатності
    foreach (var edge in path)
    {
        capacity[edge.Item1][edge.Item2] -= pathFlow;

        if (!capacity.ContainsKey(edge.Item2))
            capacity[edge.Item2] = new Dictionary<int, int>();

        if (!capacity[edge.Item2].ContainsKey(edge.Item1))
            capacity[edge.Item2][edge.Item1] = 0;
    }
}

```

```

        capacity[edge.Item2][edge.Item1] += pathFlow;

        if (!flowThroughVertex.ContainsKey(edge.Item1))

            flowThroughVertex[edge.Item1] = 0;

        flowThroughVertex[edge.Item1] += pathFlow;
    }

    // Додаємо потік до максимального потоку
    maxFlow += pathFlow;
}

return maxFlow; // Повертаємо максимальний потік
}

// Метод для знаходження шляху збільшення потоку
private List<Tuple<int, int>> FindAugmentingPath(int source, int sink)
{
    var parent = new Dictionary<int, int>();
    var visited = new HashSet<int>();
    var queue = new Queue<int>();
    queue.Enqueue(source);
    visited.Add(source);

```

```

while (queue.Count > 0)
{
    int u = queue.Dequeue();

    if (u == sink)
        break;

    // Шукаємо сусідів вершини u для шляху збільшення потоку
    foreach (var v in graph[u].Keys)
    {
        if (!visited.Contains(v) && capacity[u].ContainsKey(v) && capacity[u][v] > 0)
        {
            queue.Enqueue(v);

            visited.Add(v);

            parent[v] = u; // Записуємо попередню вершину для відновлення шляху
        }
    }
}

// Якщо шляху немає, повертаємо null
if (!visited.Contains(sink))

    return null;

var path = new List<Tuple<int, int>>>();

```

```

for (int v = sink; v != source; v = parent[v])
{
    int u = parent[v];
    path.Insert(0, Tuple.Create(u, v)); // Відновлюємо шлях
}

```

```

return path;
}

```

// Метод для отримання потоку через вершини

```

public Dictionary<int, int> GetFlowThroughVertex()
{
    return flowThroughVertex;
}
}

```

// Обробник натискання кнопки для запуску алгоритму Форда-Фалкерсона

```

private void RunFordFulkersonButton_Click(object sender, EventArgs e)
{
    FordFulkerson ff = new FordFulkerson(vertices, edges);

    int source = 1; // Початкова вершина (джерело)
    int sink = vertices.Count; // Кінцева вершина (стік)
}

```

```
int maxFlow = ff.FindMaxFlow(source, sink);
```

```
// Створюємо форму для відображення результатів
```

```
var resultForm = new Form
```

```
{
```

```
    Text = "Результаты алгоритма Форда-Фалкерсона",
```

```
    AutoSize = true
```

```
};
```

```
// Додаємо мітку для максимального потоку
```

```
var resultLabel = new Label
```

```
{
```

```
    Text = $"Максимальный поток: {maxFlow}",
```

```
    Location = new Point(10, 10),
```

```
    AutoSize = true
```

```
};
```

```
resultForm.Controls.Add(resultLabel);
```

```
// Додаємо інформацію про потік через кожну вершину
```

```
int yOffset = 40; // Початкова позиція для вивода інформації о потоках через  
вершини
```

```

foreach (var vertex in vertices)
{
    int flow = ff.GetFlowThroughVertex().ContainsKey(vertex.Key)
        ? ff.GetFlowThroughVertex()[vertex.Key]
        : 0;

    Label vertexLabel = new Label
    {
        Text = $"Вершина {vertex.Key} - Поток: {flow}",
        Location = new Point(10, yOffset),
        Size = new Size(250, 30)
    };

    resultForm.Controls.Add(vertexLabel);

    yOffset += 30; // Зміщення для наступної вершини
}

resultForm.Controls.Add(resultLabel);

// Відображаємо форму результатів
resultForm.ShowDialog();
}

```

// Алгоритм Едмондса-Карпа

public class EdmondsKarp

{

private Dictionary<int, Point> vertices; // Словник для зберігання вершин графа

private List<Tuple<int, int, int>> edges; // Список для зберігання ребер графа та їхніх пропускних спроможностей

private Dictionary<int, Dictionary<int, int>> capacity; // Словник для зберігання пропускних спроможностей між парами вершин

private Dictionary<int, Dictionary<int, int>> flow; // Словник для зберігання потоку між парами вершин

private Dictionary<int, Dictionary<int, int>> graph; // Словник для зберігання графа та його ребер

// Конструктор ініціалізує граф, пропускні спроможності і потоки

public EdmondsKarp(Dictionary<int, Point> vertices, List<Tuple<int, int, int>> edges)

{

this.vertices = vertices;

this.edges = edges;

capacity = new Dictionary<int, Dictionary<int, int>>();

flow = new Dictionary<int, Dictionary<int, int>>(); // Словник для потоку між вершинами

graph = new Dictionary<int, Dictionary<int, int>>();

// Ініціалізуємо граф та пропускні спроможності для кожного ребра

```

foreach (var edge in edges)
{
    if (!graph.ContainsKey(edge.Item1))
        graph[edge.Item1] = new Dictionary<int, int>();

    if (!graph.ContainsKey(edge.Item2))
        graph[edge.Item2] = new Dictionary<int, int>();

    graph[edge.Item1][edge.Item2] = edge.Item3;

    if (!capacity.ContainsKey(edge.Item1))
        capacity[edge.Item1] = new Dictionary<int, int>();

    capacity[edge.Item1][edge.Item2] = edge.Item3;

    // Ініціалізуємо потоки для кожного ребра як 0

    if (!flow.ContainsKey(edge.Item1))
        flow[edge.Item1] = new Dictionary<int, int>();

    if (!flow[edge.Item1].ContainsKey(edge.Item2))
        flow[edge.Item1][edge.Item2] = 0;
}
}

// Метод для знаходження максимального потоку між джерелом та стоком

public int FindMaxFlow(int source, int sink)
{

```

```
int maxFlow = 0;
```

```
flow.Clear(); // Очищаємо потоки перед новим обчисленням
```

```
// Шукаємо шлях до потоку поки існують можливі шляхи для збільшення  
потоку
```

```
while (true)
```

```
{
```

```
    var path = FindAugmentingPath(source, sink);
```

```
    if (path == null)
```

```
        break;
```

```
    // Знаходимо мінімальний потік на шляху
```

```
    int pathFlow = int.MaxValue;
```

```
    foreach (var edge in path)
```

```
    {
```

```
        pathFlow = Math.Min(pathFlow, capacity[edge.Item1][edge.Item2] -  
flow[edge.Item1][edge.Item2]);
```

```
    }
```

```
// Оновлюємо потоки та пропускні спроможності
```

```
foreach (var edge in path)
```

```
{
```

```
    capacity[edge.Item1][edge.Item2] -= pathFlow;
```

```

    if (!capacity.ContainsKey(edge.Item2))

        capacity[edge.Item2] = new Dictionary<int, int>();

    if (!capacity[edge.Item2].ContainsKey(edge.Item1))

        capacity[edge.Item2][edge.Item1] = 0;

    capacity[edge.Item2][edge.Item1] += pathFlow;

    flow[edge.Item1][edge.Item2] += pathFlow; // Збільшуємо потік по ребру
}

maxFlow += pathFlow; // Додаємо до максимального потоку
}

return maxFlow; // Повертаємо максимальний потік
}

// Метод для знаходження шляху з джерела до стоку
private List<Tuple<int, int>> FindAugmentingPath(int source, int sink)
{
    var parent = new Dictionary<int, int>(); // Для збереження шляхів
    var visited = new HashSet<int>(); // Для відслідковування відвіданих вершин
    var queue = new Queue<int>(); // Черга для BFS
    queue.Enqueue(source); // Починаємо з джерела
    visited.Add(source);

```

```

// BFS для пошуку шляху з джерела до стоку
while (queue.Count > 0)
{
    int u = queue.Dequeue();

    if (u == sink)
        break;

    foreach (var v in graph[u].Keys)
    {
        // Ініціалізація потоку між вершинами, якщо ще не існує
        if (!flow.ContainsKey(u))
            flow[u] = new Dictionary<int, int>();

        if (!flow[u].ContainsKey(v))
            flow[u][v] = 0;

        // Перевірка, чи є можливість збільшити потік по ребру
        if (!visited.Contains(v) && capacity[u].ContainsKey(v) && flow[u][v] <
capacity[u][v])
        {
            queue.Enqueue(v);

            visited.Add(v);

            parent[v] = u; // Зберігаємо батьківську вершину для побудови шляху

```

```

    }

    }

}

```

```

// Якщо не знайдено шлях, повертаємо null

```

```

if (!visited.Contains(sink))

```

```

    return null;

```

```

// Відновлюємо шлях з джерела до стоку

```

```

var path = new List<Tuple<int, int>>();

```

```

for (int v = sink; v != source; v = parent[v])

```

```

{

```

```

    int u = parent[v];

```

```

    path.Insert(0, Tuple.Create(u, v)); // Додаємо ребра в шлях

```

```

}

```

```

return path;

```

```

}

```

```

// Метод для отримання потоку між вершинами

```

```

public Dictionary<int, Dictionary<int, int>> GetFlow()

```

```

{

```

```

    return flow;

```

```
}  
  
}  
  
// Обработчик кнопки алгоритма Эдмондса-Карпа  
private void RunEdmondsKarpButton_Click(object sender, EventArgs e)  
{  
  
    // Створюємо об'єкт алгоритму Едмондса-Карпа з вершин і ребер  
    EdmondsKarp ek = new EdmondsKarp(vertices, edges);  
  
    int source = 1; // Джерело  
    int sink = vertices.Count; // Сток  
  
    // Обчислюємо максимальний потік  
    int maxFlow = ek.FindMaxFlow(source, sink);  
  
    // Створюємо форму для відображення результатів  
    var resultForm = new Form  
    {  
        Text = "Результаты алгоритма Эдмондса-Карпа",  
        AutoSize = true,  
    };  
  
    // Мітка для відображення максимального потоку
```

```
var resultLabel = new Label
```

```
{
```

```
    Text = $"Максимальный поток: {maxFlow}",
```

```
    Location = new Point(10, 10),
```

```
    AutoSize = true,
```

```
};
```

```
resultForm.Controls.Add(resultLabel);
```

```
int yOffset = 40; // Початкове розташування для виведення інформації про потік
```

```
// Виведення потоку для кожної вершини
```

```
foreach (var vertex in vertices)
```

```
{
```

```
    int flow = ek.GetFlow().ContainsKey(vertex.Key)
```

```
        ? ek.GetFlow()[vertex.Key].Sum(kv => kv.Value) // Сума потоків для кожної
```

```
вершини
```

```
        : 0;
```

```
Label vertexLabel = new Label
```

```
{
```

```
    Text = $"Вершина {vertex.Key} - Поток: {flow}",
```

```
    Location = new Point(10, yOffset),
```

```

        Size = new Size(250, 30)

};

resultForm.Controls.Add(vertexLabel);

yOffset += 30; // Зміщуємо вниз для наступного рядка
}

```

```

resultForm.Controls.Add(resultLabel);

```

```

// Відображаємо форму результатів

```

```

resultForm.ShowDialog();

```

```

}

```

```

private void runDeliveryButton_Click(object sender, EventArgs e)

```

```

{

```

```

    // Створюємо об'єкт для алгоритму Едмондса-Карпа для пошуку максимального
    потоку

```

```

    EdmondsKarp ek = new EdmondsKarp(vertices, edges);

```

```

    int source = 1; // Джерело в графі

```

```

    int sink = vertices.Count; // Сток в графі

```

```

    // Знаходимо максимальний потік за допомогою алгоритму Едмондса-Карпа

```

```

    int maxFlow = ek.FindMaxFlow(source, sink);

```

```
// Створюємо об'єкт для алгоритму Форда-Фалкерсона для пошуку  
максимального потоку
```

```
FordFulkerson ff = new FordFulkerson(vertices, edges);
```

```
// Задаємо джерело та сток для другого алгоритму
```

```
int source1 = 1;
```

```
int sink1 = vertices.Count;
```

```
// Знаходимо максимальний потік за допомогою алгоритму Форда-Фалкерсона
```

```
int maxFlow1 = ff.FindMaxFlow(source1, sink1);
```

```
// Створюємо нову форму для відображення калькулятора доставки
```

```
var resultForm = new Form
```

```
{
```

```
    Text = "Калькулятор доставки", // Назва форми
```

```
    AutoSize = false, // Автоматичний розмір форми
```

```
    Size = new System.Drawing.Size(350, 550),
```

```
    AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F),
```

```
    AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font,
```

```
};
```

```
// Створюємо лейбл для виведення тексту "Назва товару"
```

```

var label2 = new Label

{

    AutoSize = true,

    BackColor = System.Drawing.Color.Transparent,

    Font = new System.Drawing.Font("Arial Narrow", 14.25F,
System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(204))),

    Location = new System.Drawing.Point(12, 9), // Позиція лейбла

    Name = "label2",

    Size = new System.Drawing.Size(122, 23),

    TabIndex = 2,

    Text = "Назва товару:\r\n", // Текст лейбла

};

resultForm.Controls.Add(label2); // Додаємо лейбл на форму


// Створюємо лейбл для введення кількості товару

var label3 = new Label

{

    AutoSize = true,

    BackColor = System.Drawing.Color.Transparent,

    Font = new System.Drawing.Font("Arial Narrow", 14.25F,
System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point, ((byte)(204))),

    Location = new System.Drawing.Point(12, 80), // Позиція лейбла

    Name = "label3",

```

```
Size = new System.Drawing.Size(88, 23),
```

```
TabIndex = 3,
```

```
Text = "Кількість:", // Текст лейбла
```

```
};
```

```
resultForm.Controls.Add(label3); // Додаємо лейбл на форму
```

```
// Створюємо поле для вводу кількості товару (NumericUpDown)
```

```
var numericUpDownQuantity = new NumericUpDown
```

```
{
```

```
Font = new System.Drawing.Font("Arial Narrow", 14.25F,
```

```
System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(204))),
```

```
Location = new System.Drawing.Point(106, 80), // Позиція поля вводу
```

```
Maximum = new decimal(new int[] { 1000000, 0, 0, 0 }), // Максимальне  
значення
```

```
Name = "numericUpDownQuantity",
```

```
Size = new System.Drawing.Size(120, 29),
```

```
TabIndex = 6,
```

```
};
```

```
resultForm.Controls.Add(numericUpDownQuantity); // Додаємо поле на форму
```

```
// Створюємо кнопку для розрахунку доставки
```

```
var buttonCalculate = new Button
```

```

{

    //BackColor = System.Drawing.SystemColors.InactiveCaption,

    Font = new System.Drawing.Font("Microsoft Sans Serif", 9.75F,
System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(204))),

    Location = new System.Drawing.Point(1, 120), // Позиція кнопки

    Name = "buttonCalculate",

    Size = new System.Drawing.Size(140, 52),

    TabIndex = 7,

    Text = "Розрахувати дані посилки", // Текст на кнопці

    UseVisualStyleBackColor = false,

};

resultForm.Controls.Add(buttonCalculate); // Додаємо кнопку на форму

var buttonFind = new Button

{

    //BackColor = System.Drawing.SystemColors.InactiveCaption,

    Font = new System.Drawing.Font("Microsoft Sans Serif", 9.75F,
System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(204))),

    Location = new System.Drawing.Point(1, 190), // Позиція кнопки

    Name = "buttonCalculate",

    Size = new System.Drawing.Size(140, 52),

    TabIndex = 7,

    Text = "Пошук замовлення", // Текст на кнопці

    UseVisualStyleBackColor = false,

```

```
};
```

```
resultForm.Controls.Add(buttonFind); // Додаємо кнопку на форму
```

```
var textBoxOrderNumber = new TextBox
```

```
{
```

```
    Font = new System.Drawing.Font("Arial Narrow", 14.25F),
```

```
    Location = new System.Drawing.Point(150, 190),
```

```
    Size = new System.Drawing.Size(100, 29),
```

```
};
```

```
resultForm.Controls.Add(textBoxOrderNumber);
```

```
// Лейбл для відображення результатів
```

```
var labelResult = new Label
```

```
{
```

```
    AutoSize = true,
```

```
    BackColor = System.Drawing.Color.Transparent,
```

```
    Dock = System.Windows.Forms.DockStyle.Top,
```

```
    Font = new System.Drawing.Font("Arial Narrow", 14.25F,
```

```
    System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(204))),
```

```
    Location = new System.Drawing.Point(0, 0),
```

```
    Name = "labelResult",
```

```
    Size = new System.Drawing.Size(0, 23),
```

```
TabIndex = 8
```

```
};
```

```
resultForm.Controls.Add(labelResult); // Додаємо лейбл на форму
```

```
// Створюємо випадаючий список для вибору товару
```

```
var comboBoxProduct = new ComboBox
```

```
{
```

```
    Font = new System.Drawing.Font("Arial Narrow", 14.25F,  
System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(204))),
```

```
    FormattingEnabled = true,
```

```
    Location = new System.Drawing.Point(140, 9), // Позиція комбобоксу
```

```
    Name = "comboBoxProduct",
```

```
    Size = new System.Drawing.Size(121, 31),
```

```
    TabIndex = 10,
```

```
};
```

```
resultForm.Controls.Add(comboBoxProduct); // Додаємо комбобокс на форму
```

```
comboBoxProduct.Items.AddRange(new object[] {
```

```
    "хліб 5", // Товар 1
```

```
    "молоко 10", // Товар 2
```

```
    "яйця 8" // Товар 3
```

```
});
```

// Оголошуємо словник для цін на товари

Dictionary<string, int> productPrices = new Dictionary<string, int>

{

{ "хліб 5", 5 }, // Ціна хліба

{ "молоко 10", 10 }, // Ціна молока

{ "яйця 8", 8 } // Ціна яєць

};

// Обробник натискання кнопки для розрахунку даних посилки

buttonCalculate.Click += (object clickSender, EventArgs clickEvent) =>

{

// Перевіряє, чи comboBoxProduct ініціалізовано та містить вибраний елемент

if (comboBoxProduct == null || comboBoxProduct.SelectedItem == null)

{

 MessageBox.Show("Виберіть продукт");

 return;

}

// Отримуємо обраний товар та його ціну

string product = comboBoxProduct.SelectedItem.ToString();

int pricePerUnit = productPrices[product];

int quantity = (int)numericUpDownQuantity.Value;

```

int deliveryTimeeff = 0;

int deliveryTimeeed = 0;

// Розрахунок часу доставки за допомогою алгоритмів
if (quantity <= maxFlow)
{
    deliveryTimeeff = 1;
    deliveryTimeeed = 1;
}
else
{
    deliveryTimeeff = (int)Math.Floor((double)quantity / maxFlow) + 1;
    deliveryTimeeed = (int)Math.Floor((double)quantity / maxFlow1) + 1;
}

// Додаткові витрати на доставку, якщо доставка займає 1 день
int deliverySurcharge = deliveryTimeeff <= 3 ? 20 : 0;

// Загальна вартість товару та доставки
int totalCost = quantity * pricePerUnit + deliverySurcharge;

// Виведення результатів на форму
labelResult.Text = $"Замовлення:\n" +

```

```

$"Товар: {product}\n" +

$"Кількість: {quantity}\n" +

$"Час доставки(Форда-Фалкерсона): {deliveryTimeeff} дн.\n" +

$"Час доставки(Едмондса-Карпа): {deliveryTimeeed} дн.\n" +

$"Загальна вартість: {totalCost} грн.";

```

```

string textToSave = $"Замовлення:\n" +

    $"Товар: {product}\n" +

    $"Кількість: {quantity}\n" +

    $"Час доставки (Форда-Фалкерсона): {deliveryTimeeff} дн.\n" +

    $"Час доставки (Едмондса-Карпа): {deliveryTimeeed} дн.\n" +

    $"Загальна вартість: {totalCost} грн.\n\n";

```

```
//Метод для нумерації замовлень
```

```
// Вказуємо шлях до вхідного файлу, який містить дані для обробки
```

```
string inputFilePath = "order.txt"; // Путь к исходному файлу
```

```
// Вказуємо шлях до вихідного файлу, куди буде збережений результат
```

```
string outputFilePath = "order.txt"; // Путь к выходному файлу
```

```
try
```

```
{
```

```
    // Читаємо всі строки з вхідного файлу
```

```
    string[] lines = File.ReadAllLines(inputFilePath);
```

// Знаходимо максимальний номер замовлення в файлі та визначаємо наступний номер

int currentOrderNumber = GetMaxOrderNumber(lines) + 1; // Определяем следующий номер для добавления

// Відкриваємо вихідний файл для запису змінених даних

using (StreamWriter writer = new StreamWriter(outputFilePath))

{

// Перебираємо кожну строку з файлу

foreach (var line in lines)

{

// Перевіряємо, чи містить строка слово "Замовлення:"

if (line.Contains("Замовлення:"))

{

// Перевіряємо, чи є вже номер замовлення на початку рядка

if (!Regex.IsMatch(line, @"^\d+\.\s*Замовлення:"))

{

// Якщо номера немає, додаємо новий номер перед текстом

"Замовлення:"

writer.WriteLine(\$"{currentOrderNumber}. Замовлення:");

currentOrderNumber++; // Збільшуємо номер для наступного

замовлення

}

else

```
{  
    // Якщо номер є, просто записуємо рядок без змін  
    writer.WriteLine(line);  
}  
}  
else  
{  
    // Якщо в рядку немає слова "Замовлення:", просто записуємо його  
    writer.WriteLine(line);  
}  
}  
}  
  
// Виводимо повідомлення про успішну обробку документа  
Console.WriteLine("Документ оброботан и сохранен как  
'order_numbered.txt'.");  
}  
  
catch (Exception ex)  
{  
    // Обробляємо помилки, якщо вони виникли під час виконання  
    Console.WriteLine("Произошла ошибка: " + ex.Message);  
}
```

// Метод для отримання максимального номера замовлення з файлу

```
int GetMaxOrderNumber(string[] lines)
```

```
{
```

```
    int maxOrderNumber = 0;
```

```
    // Перебираємо кожен рядок файлу
```

```
    foreach (var line in lines)
```

```
    {
```

```
        // Шукаємо рядки, які починаються з числа та точки, а потім слідує
```

```
"Замовлення:"
```

```
        Match match = Regex.Match(line, @"^(\d+)\.\s*Замовлення:");
```

```
        if (match.Success)
```

```
        {
```

```
            // Якщо знайдено номер, перетворюємо його на число та перевіряємо,
```

```
чи є він максимальним
```

```
            int currentOrderNumber = int.Parse(match.Groups[1].Value);
```

```
            if (currentOrderNumber > maxOrderNumber)
```

```
            {
```

```
                maxOrderNumber = currentOrderNumber; // Оновлюємо
```

```
максимальний номер
```

```
            }
```

```
        }
```

```
    }
```

```

        // Повертаємо максимальний знайдений номер
        return maxOrderNumber;
    }

    try
    {
        // Записуємо текст в файл (перезапишет файл, если он существует)
        File.AppendAllText("order.txt", textToSave);
    }

    catch (Exception ex)
    {
        // Если произошла ошибка, выводим сообщение
        MessageBox.Show($"Помилка при збереженні файлу: {ex.Message}");
    }
};

buttonFind.Click += (object clickSender, EventArgs clickEvent) =>
{
    // Отримуємо номер замовлення з текстового поля
    if (int.TryParse(textBoxOrderNumber.Text, out int orderNumberToFind))
    {
        try

```

```

{

    // Читаємо всі замовлення з файлу

    string[] orders = File.ReadAllLines("order.txt");

    string orderDetails = string.Empty;


    // Пошук замовлення за номером

    foreach (var line in orders)
    {

        if (line.StartsWith($"{orderNumberToFind}. Замовлення:"))
        {

            orderDetails += line + Environment.NewLine;

            int index = Array.IndexOf(orders, line);


            // Зчитуємо рядки замовлення, поки не зустрінемо наступний номер
            або кінець файлу

            while (++index < orders.Length && !orders[index].StartsWith("Номер
            замовлення:"))
            {

                orderDetails += orders[index] + Environment.NewLine;

            }

            break;

        }

    }
}

```

```
// Виведення результатів пошуку

if (!string.IsNullOrEmpty(orderDetails))

{

    labelResult.Text = $"Замовлення знайдено:\n{orderDetails}";

}

else

{

    labelResult.Text = "Замовлення з таким номером не знайдено.";

}

}

catch (Exception ex)

{

    // Обробка помилок читання файлу

    MessageBox.Show($"Помилка при читанні файлу: {ex.Message}",

"Помилка");

}

}

else

{

    MessageBox.Show("Будь ласка, введіть коректний номер замовлення.",

"Помилка вводу");

}
```

```
};
```

```
// Створюємо панель для розміщення результатів
```

```
var panel1 = new Panel
```

```
{
```

```
    //BackColor = System.Drawing.SystemColors.InactiveCaption,
```

```
    BorderStyle = System.Windows.Forms.BorderStyle.Fixed3D,
```

```
    Location = new System.Drawing.Point(1, 250), // Позиція панелі
```

```
    Name = "panel1",
```

```
    Size = new System.Drawing.Size(350, 200), // Розміри панелі
```

```
    TabIndex = 9,
```

```
};
```

```
resultForm.Controls.Add(panel1); // Додаємо панель на форму
```

```
panel1.Controls.Add(labelResult);
```

```
// Створюємо кнопку для перегляду всіх замовлень
```

```
var buttonOrderView = new Button
```

```
{
```

```
    Location = new Point(170, 120), // Позиція кнопки
```

```
    Name = "buttonOrderView",
```

```
    Size = new Size(140, 52),
```

```
Text = "Замовлення", // Текст на кнопці
```

```
UseVisualStyleBackColor = false,
```

```
};
```

```
resultForm.Controls.Add(buttonOrderView); // Додаємо кнопку на форму
```

```
// Обробник натискання кнопки
```

```
buttonOrderView.Click += (object clickSender, EventArgs clickEvent) =>
```

```
{
```

```
    // Вказуємо шлях до вхідного файлу, який містить дані для обробки
```

```
    string inputFilePath = "order.txt"; // Путь к исходному файлу
```

```
        // Вказуємо шлях до вихідного файлу, куди буде
```

збережений результат

```
        string outputFilePath = "order.txt"; // Путь к выходному файлу
```

```
    try
```

```
    {
```

```
        // Читаємо всі строки з вхідного файлу
```

```
        string[] lines = File.ReadAllLines(inputFilePath);
```

```
        // Знаходимо максимальний номер замовлення в файлі та визначаємо
```

наступний номер

```
        int currentOrderNumber = GetMaxOrderNumber(lines) + 1; // Определяем
```

следующий номер для добавления

```
// Відкриваємо вихідний файл для запису змінених даних
using (StreamWriter writer = new StreamWriter(outputFilePath))
{
    // Перебираємо кожну строку з файлу
    foreach (var line in lines)
    {
        // Перевіряємо, чи містить строка слово "Замовлення:"
        if (line.Contains("Замовлення:"))
        {
            // Перевіряємо, чи є вже номер замовлення на початку рядка
            if (!Regex.IsMatch(line, @"^\d+\.\s*Замовлення:"))
            {
                // Якщо номера немає, додаємо новий номер перед текстом
                "Замовлення:"

                writer.WriteLine($"{currentOrderNumber}. Замовлення:");
                currentOrderNumber++; // Збільшуємо номер для наступного
                замовлення
            }
            else
            {
                // Якщо номер є, просто записуємо рядок без змін
                writer.WriteLine(line);
            }
        }
    }
}
```

```
    }  
    }  
    else  
    {  
        // Якщо в рядку немає слова "Замовлення:", просто записуємо його  
        writer.WriteLine(line);  
    }  
}  
}  
}  
  
// Виводимо повідомлення про успішну обробку документа  
Console.WriteLine("Документ оброботан и сохранен как  
'order_numbered.txt'.");  
}  
  
catch (Exception ex)  
{  
    // Обробляємо помилки, якщо вони виникли під час виконання  
    Console.WriteLine("Произошла ошибка: " + ex.Message);  
}  
  
// Метод для отримання максимального номера замовлення з файлу  
int GetMaxOrderNumber(string[] lines)  
{
```

```

int maxOrderNumber = 0;

// Перебираємо кожен рядок файлу
foreach (var line in lines)
{
    // Шукаємо рядки, які починаються з числа та точки, а потім слідує
    "Замовлення:"

    Match match = Regex.Match(line, @"^(\d+)\.\s*Замовлення:");

    if (match.Success)
    {
        // Якщо знайдено номер, перетворюємо його на число та перевіряємо,
        чи є він максимальним

        int currentOrderNumber = int.Parse(match.Groups[1].Value);

        if (currentOrderNumber > maxOrderNumber)
        {
            maxOrderNumber = currentOrderNumber; // Оновлюємо
            максимальний номер
        }
    }
}

// Повертаємо максимальний знайдений номер
return maxOrderNumber;

```

```
}
```

```
// Створюємо нову форму
```

```
var Form = new Form
```

```
{
```

```
    Text = "Замовлення",
```

```
    Size = new Size(360, 460), //Розмір форми
```

```
};
```

```
//Читаємо із файлу
```

```
string fileContent = File.ReadAllText("order.txt");
```

```
//Для виводу тексту
```

```
var resultLabel = new Label
```

```
{
```

```
    Text = fileContent, //Текст із файлу
```

```
    AutoSize = true, //Розмір: автоматичний
```

```
    Location = new Point(0, 0), // Позиція
```

```
    Name = "resultLabel",
```

```
    Font = new Font("Arial", 10, FontStyle.Regular)
```

```
};
```

```
Form.Controls.Add(resultLabel); // Додаємо на форму
```

```
// Створюємо панель

var panel = new Panel

{

    BorderStyle = BorderStyle.Fixed3D,

    Location = new Point(0, 0), // Позиція панелі

    Name = "panel",

    Size = new Size(350, 420), // Розміри панелі

    AutoScroll = true,

};

Form.Controls.Add(panel); // Додаємо панель на форму

panel.Controls.Add(resultLabel); // Додаємо текст на панель


// Відображаємо форму з калькулятором доставки

Form.ShowDialog();

};


// Відображаємо форму з калькулятором доставки

resultForm.ShowDialog();

}
```

}

}

ДОДАТОК В
СЛАЙДИ ПРЕЗЕНТАЦІЇ

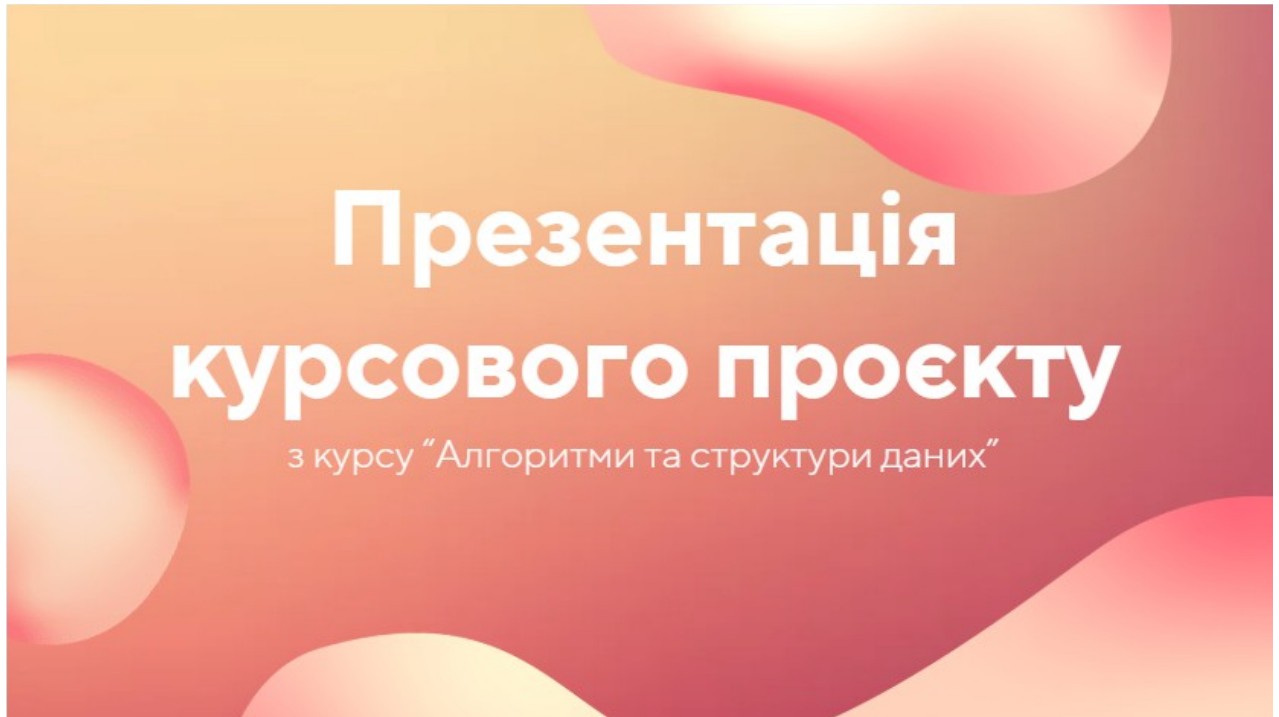


Рисунок 1 – Слайд 1. Титульний лист

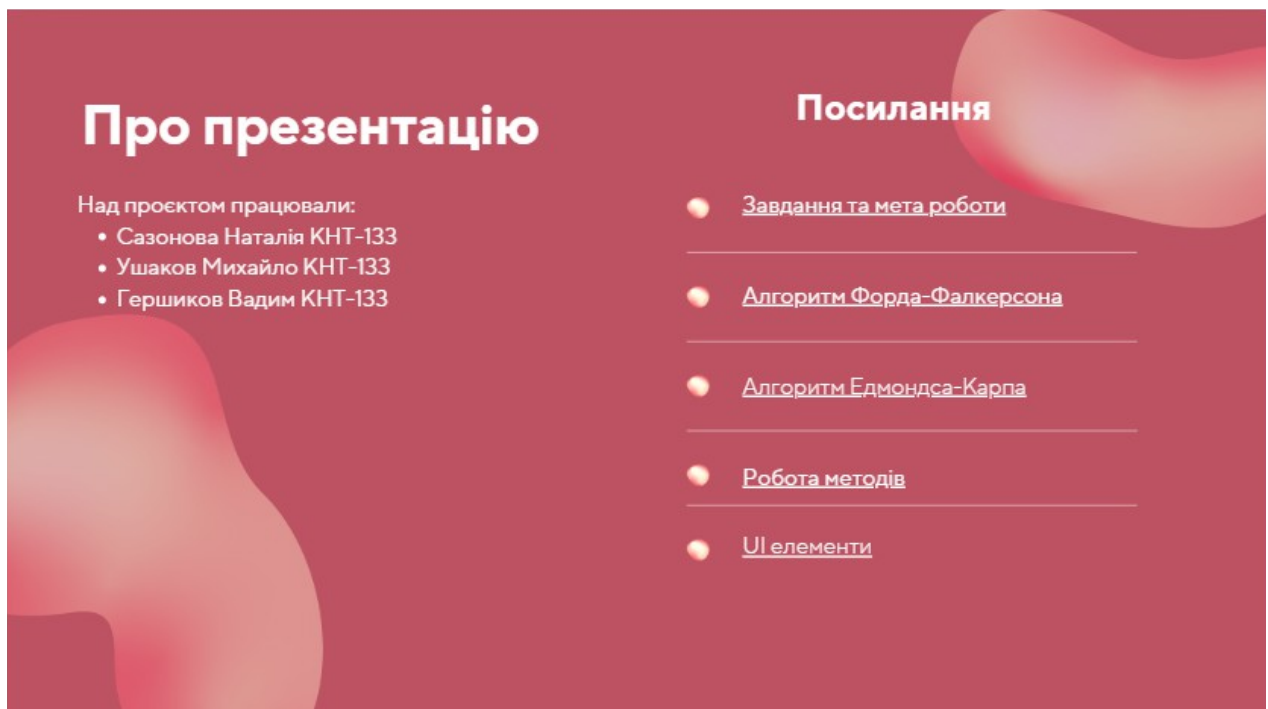


Рисунок 2 – Слайд 2. Про презентацію

Мета презентації

Пояснити основні поняття, пов'язані з програмами, розкрити процес розробки програмного забезпечення.

Зокрема, презентація має на меті:

1. Зрозуміти, як працюють алгоритми;
2. Які використовуються методи;
3. Показати, як працювати з програмою;
4. Показати функціонал програми.

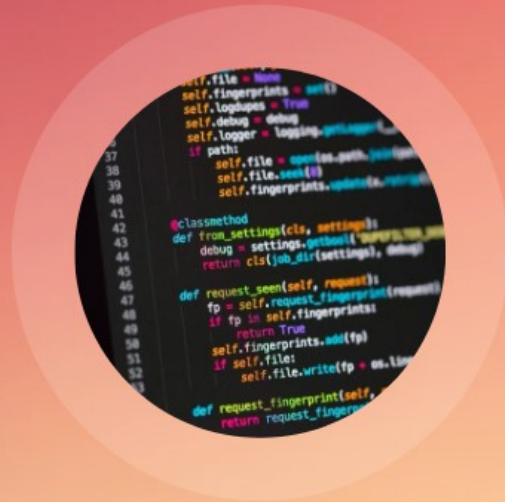


Рисунок 3 – Слайд 3. Мета презентації

Завдання та мета роботи

Робота алгоритмів, основні властивості, реалізація в коді

[Повернутися на головну](#)

Рисунок 4 – Слайд 4. Завдання та мета роботи

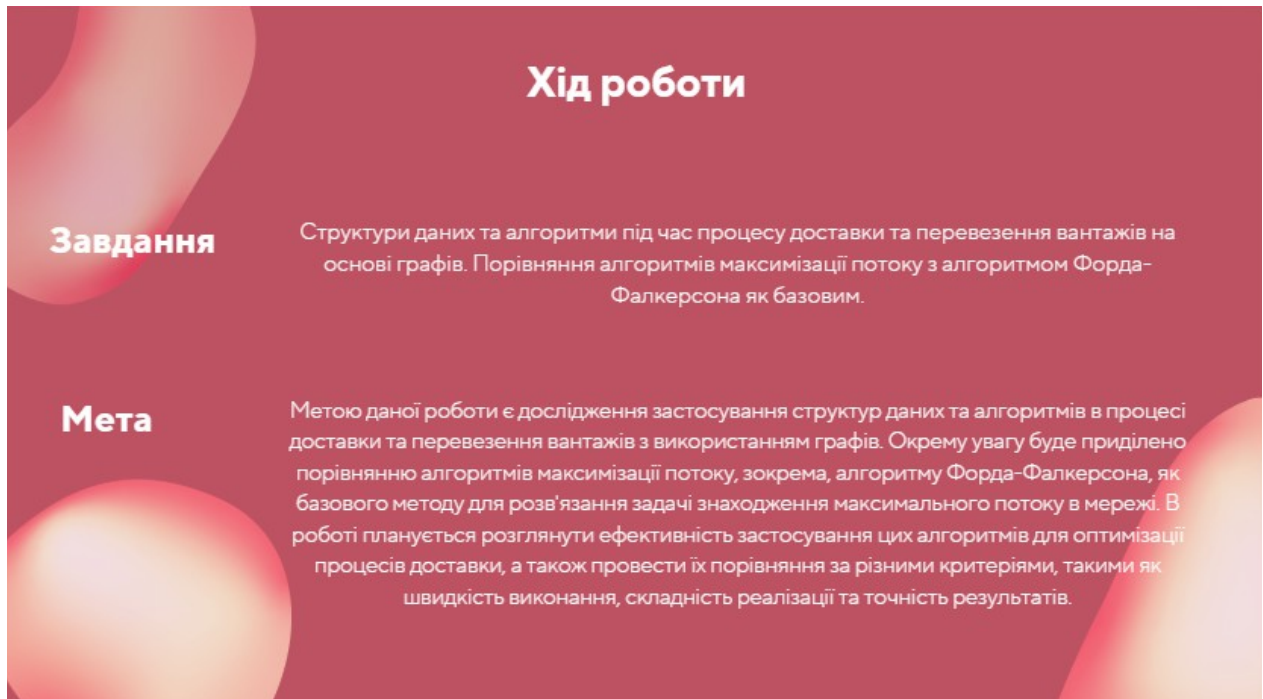


Рисунок 5 – Слайд 5. Хід роботи

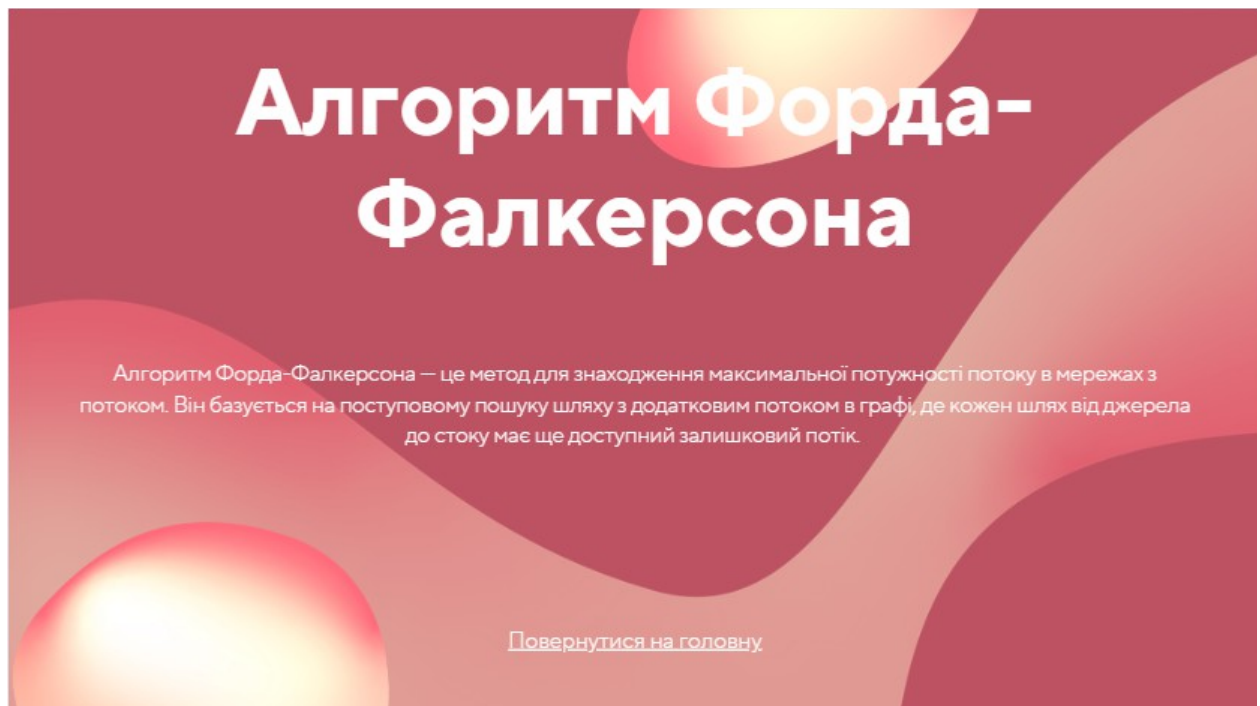


Рисунок 6 – Слайд 6. Алгоритм Форда-Фалкерсона

Примітки до слайду 6: Алгоритм Форда-Фалкерсона — це метод для знаходження максимальної потужності потоку в мережах з потоком. Він базується на

поступовому пошуку шляху з додатковим потоком в графі, де кожен шлях від джерела до стоку має ще доступний залишковий потік. Алгоритм повторно знаходить такі шляхи, доки не можна додати новий потік. Основна ідея полягає в використанні жадібного підходу для пошуку шляхів з найменшим обмеженням по потоку та їх збільшенні, поки не досягається максимальної пропускної здатності мережі.

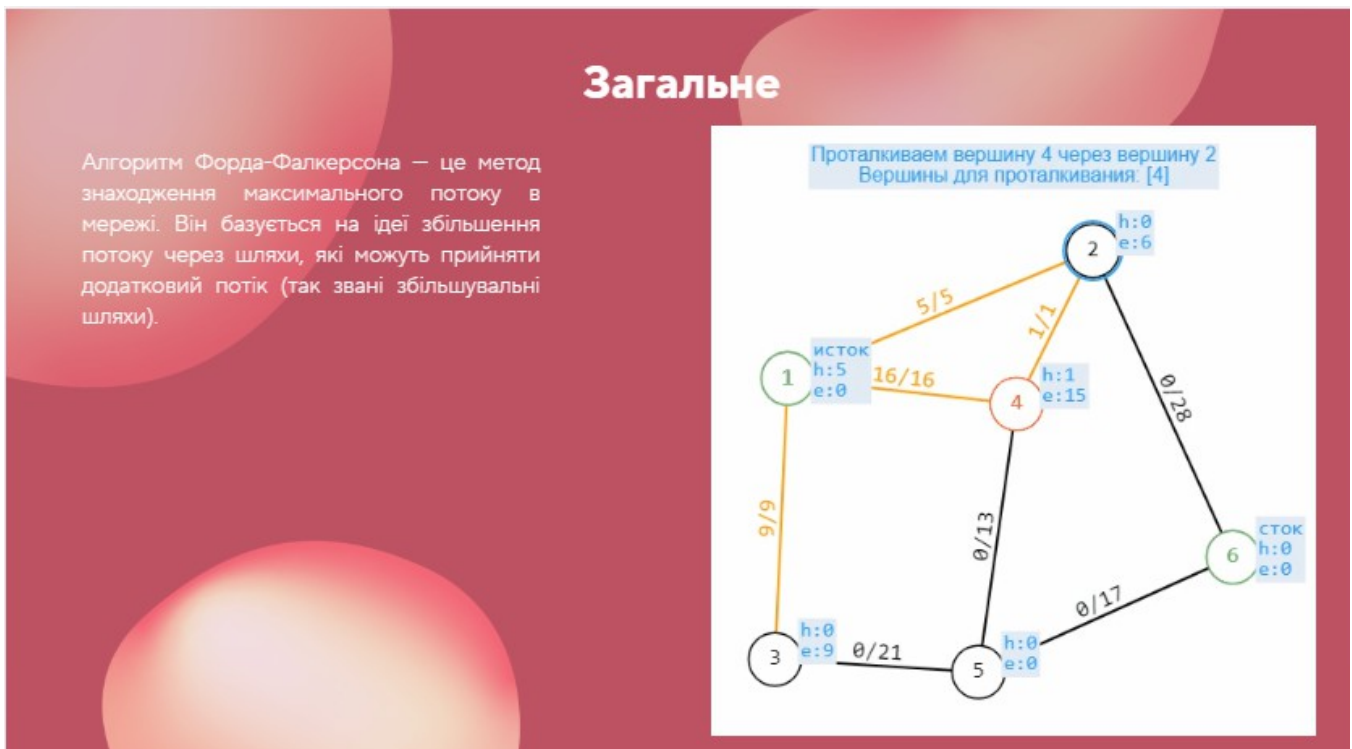


Рисунок 7 – Слайд 7. Загальне

Примітки до слайду 7: Алгоритм Форда-Фалкерсона — це метод знаходження максимального потоку в мережі. Він базується на ідеї збільшення потоку через шляхи, які можуть прийняти додатковий потік (так звані збільшувальні шляхи).

Основні кроки алгоритму:

- Ініціалізація: Початковий потік в мережі встановлюється рівним нулю;
- Пошук збільшувального шляху: Шукається шлях від джерела до стоку, по якому можна провести додатковий потік. Цей шлях повинен відповідати умові: залишкові ємності ребер на шляху повинні бути більші за нуль;

- Збільшення потоку: Якщо знайдений збільшувальний шлях, по ньому збільшується потік, а ємності ребер оновлюються (зменшуються в напрямку потоку і збільшуються проти потоку);
- Повторення: Кроки 2 і 3 повторюються, поки не буде знайдений збільшувальний шлях, тобто, поки мережа не стане насиченою.

Алгоритм завершується, коли не вдається знайти збільшувальний шлях, і поточний потік стає максимальним. Алгоритм Форда-Фалкерсона можна реалізувати за допомогою пошуку в глибину або в ширину для знаходження шляхів.

Його складність залежить від вибору алгоритму пошуку шляху і може бути досить високою, якщо використовувати пошук в глибину, тому іноді застосовують вдосконалені версії, такі як алгоритм Едмондса-Карпа.

Реалізація алгоритма Форда-Фалкерсона в проєкті

```

// Метод для обчислення графів
// заданих користувачем
// Метод для обчислення графів
// заданих користувачем
// Метод для обчислення графів
// заданих користувачем

```

1.

Метод для обчислення графів заданих користувачем

```

// Метод для перевірки на
// точність
// Метод для перевірки на
// точність
// Метод для перевірки на
// точність

```

2.

Метод для перевірки на точність

```

// Метод для розрахунку доставки
// товарів
// Метод для розрахунку доставки
// товарів
// Метод для розрахунку доставки
// товарів

```

3.

Метод для розрахунку доставки товарів

Рисунок 8 – Слайд 8. Реалізація алгоритма Форда-Фалкерсона

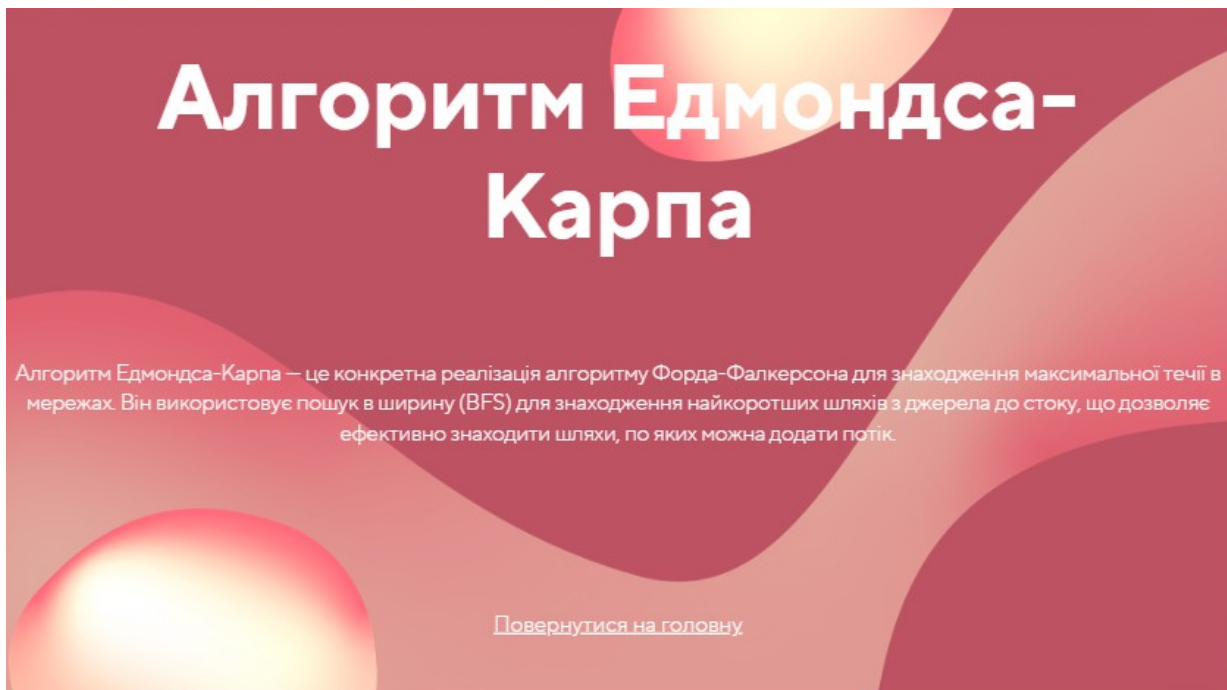


Рисунок 9 – Слайд 9. Алгоритм Едмонса-Карпа

Примітки до слайду 9: Алгоритм Едмондса-Карпа – це конкретна реалізація алгоритму Форда-Фалкерсона для знаходження максимальної течії в мережах. Він використовує пошук в ширину (BFS) для знаходження найкоротших шляхів з джерела до стоку, що дозволяє ефективно знаходити augmenting path (шляхи, по яких можна додати потік). Цей підхід гарантує, що кожен крок збільшення потоку веде до максимальної можливої течії, і алгоритм завершується, коли більше не можна знайти таких шляхів.

Алгоритм працює в такі етапи:

1. Пошук шляху з джерела до стоку, де є доступний залишковий потік.
2. Збільшення потоку по знайденому шляху.
3. Повторення кроків 1 і 2, поки не буде знайдено жодного шляху з додатковим потоком.

Оскільки BFS знаходить найкоротші шляхи за кількістю ребер, часова складність алгоритму Едмондса-Карпа становить $O(V * E^2)$, де V – кількість вершин, а E – кількість ребер.



Рисунок 10 – Слайд 10. Загальне

Примітки до слайду 10: Алгоритм Едмондса-Карпа — це вдосконалена версія алгоритму Форда-Фалкерсона для знаходження максимального потоку в мережі. Він використовує метод пошуку в ширину (BFS) для знаходження збільшувальних шляхів, що дозволяє ефективніше знаходити максимальний потік.

Основні кроки алгоритму Едмондса-Карпа:

- Ініціалізація: Потік в мережі ініціалізується як нульовий, і всі ребра мають початкову ємність;
- Пошук збільшувального шляху: Використовуючи алгоритм пошуку в ширину (BFS), знаходиться шлях від джерела до стоку, де є залишкові ємності (тобто, по якому ще можна пропустити потік);
- Збільшення потоку: Якщо знайдений збільшувальний шлях, то по ньому збільшується потік на мінімальне значення з усіх ємностей ребер цього шляху (це гарантує, що на кожному кроці потік збільшується оптимально);

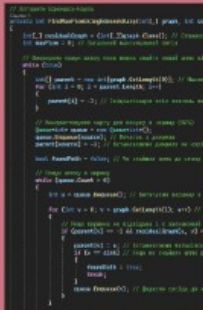
– Оновлення ємностей: Після того як потік збільшено, ємності ребер на шляху оновлюються: для кожного ребра по напрямку потоку зменшується ємність, а в протилежному напрямку (якщо є зворотний потік) збільшується;

– Повторення: Кроки 2-4 повторюються до тих пір, поки не можна знайти збільшувальний шлях (коли BFS більше не знаходить шляху від джерела до стоку).

Алгоритм завершується, коли не існує жодного збільшувального шляху. Потік, який було знайдено на кожному етапі, є максимальним.

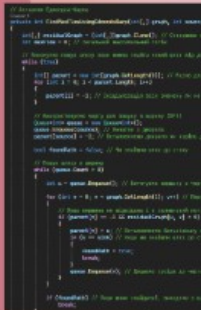
Складність: Часова складність алгоритму Едмондса-Карпа – $O(V * E^2)$, де V – кількість вершин, а E — кількість ребер у графі. Це зумовлено тим, що пошук шляху в ширину займає $O(E)$, а максимальний потік може бути пропорційним числу ребер E .

Реалізація алгоритма Едмондса-Карпа в проекті



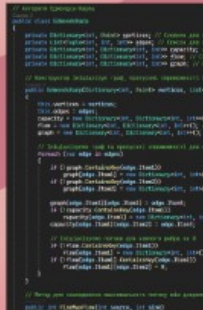
1.

Метод для
обчислення графів
заданих
користувачем



2.

Метод для
перевірки на
точність



3.

Метод для
розрахунку доставки
товарів

Рисунок 11 – Слайд 11. Реалізація алгоритма Едмондса-Карпа в проєкті

Порівняння алгоритмів		
Характеристика	Алгоритми Форда-Фалкерсона	Алгоритми Едмондса-Карпа
Метод пошуку збільшувальних шляхів	Може використовувати пошук в глибину (DFS) або в ширину (BFS), що не гарантує вибір оптимальних шляхів.	Використовує пошук в ширину (BFS) для знаходження найкоротших збільшувальних шляхів.
Часова складність	Залежить від вибору методу пошуку шляху: для DFS складність може бути $O(E * f)$, де f – величина потоку.	$O(V * E^2)$ — більш ефективний за рахунок використання BFS, що гарантує більш швидкий пошук шляху.
Збільшувальний шлях	Може бути будь-яким шляхом з залишковою ємністю, не обов'язково найкоротшим.	Гарантується, що шлях буде найкоротшим серед усіх можливих.
Удосконалення ємностей	Ємності оновлюються після кожного знайденого шляху, без спеціальної стратегії для вибору шляхів.	Ємності оновлюються після кожного пошуку збільшувального шляху, що є найкоротшим шляхом (це допомагає швидше досягти максимального потоку).
Гарантована ефективність	Алгоритм може бути менш ефективним у випадку використання DFS, особливо для великих графів.	Завжди ефективний через використання BFS, що зникає час виконання в порівнянні з простим Форда-Фалкерсона.
Складність реалізації	Простий у реалізації, особливо за допомогою DFS для пошуку шляху.	Трошки складніший у реалізації через необхідність застосування BFS, але значно ефективніший.

Рисунок 12 – Слайд 12. Порівняння алгоритмів

Примітки до слайду 12: Пояснення відмінностей:

а) Пошук збільшувального шляху:

У алгоритмі Форда-Фалкерсона пошук збільшувального шляху може бути виконаний за допомогою пошуку в глибину (DFS) або пошуку в ширину (BFS). Якщо використовується DFS, то алгоритм може вибирати довгі шляхи, що збільшує кількість ітерацій, і в результаті може бути повільнішим.

У алгоритмі Едмондса-Карпа завжди використовується BFS, що гарантує пошук найкоротших шляхів (з мінімальною кількістю ребер), що, в свою чергу, зменшує кількість ітерацій та покращує ефективність.

б) Часова складність:

У алгоритмі Форда-Фалкерсона часова складність залежить від того, який метод пошуку шляху використовується. Якщо це DFS, то складність може бути $O(E * f)$, де f – величина потоку. Це може бути дуже великим, якщо потік великий або граф має багато ребер.

У алгоритмі Едмондса-Карпа часова складність чітко визначена як $O(V * E^2)$, де V – кількість вершин, а E – кількість ребер у графі. Завдяки використанню BFS, кількість ітерацій значно зменшується порівняно з випадком, коли застосовується DFS.

в) Ефективність:

Алгоритм Форда-Фалкерсона може бути дуже неефективним в деяких випадках, особливо коли пошук збільшувального шляху виконується з використанням DFS, що може призвести до великої кількості ітерацій, особливо у великих графах.

Алгоритм Едмондса-Карпа набагато ефективніший, оскільки він завжди використовує BFS для пошуку найкоротших шляхів, що значно знижує кількість ітерацій і дозволяє досягти максимального потоку швидше.

г) Гарантія знаходження максимального потоку:

Обидва алгоритми гарантовано знаходять максимальний потік, але алгоритм Едмондса-Карпа робить це більш ефективно завдяки використанню BFS, що дозволяє знаходити найшвидші збільшувальні шляхи.

Висновок: Алгоритм Форда-Фалкерсона є більш загальним і простим у реалізації, але може бути менш ефективним, особливо при використанні DFS. Алгоритм Едмондса-Карпа є вдосконаленим і гарантує більш високу ефективність завдяки використанню BFS для пошуку найкоротших збільшувальних шляхів, що робить його швидшим у більшості випадків, хоча він складніший у реалізації.



Рисунок 13 – Слайд 13. Ефективність роботи алгоритмів

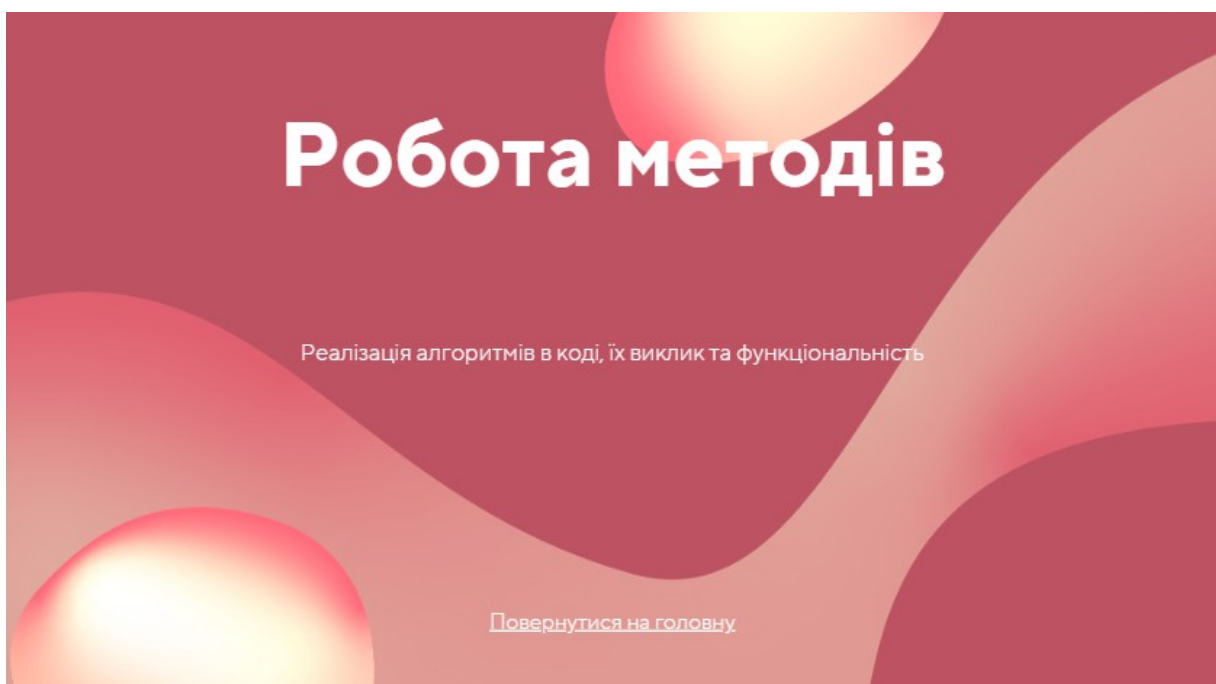


Рисунок 14 – Слайд 14. Робота методів



Рисунок 15 – Слайд 15. Алгоритм Форда-Фалкерсона обран як базовий та виконується наступними методами

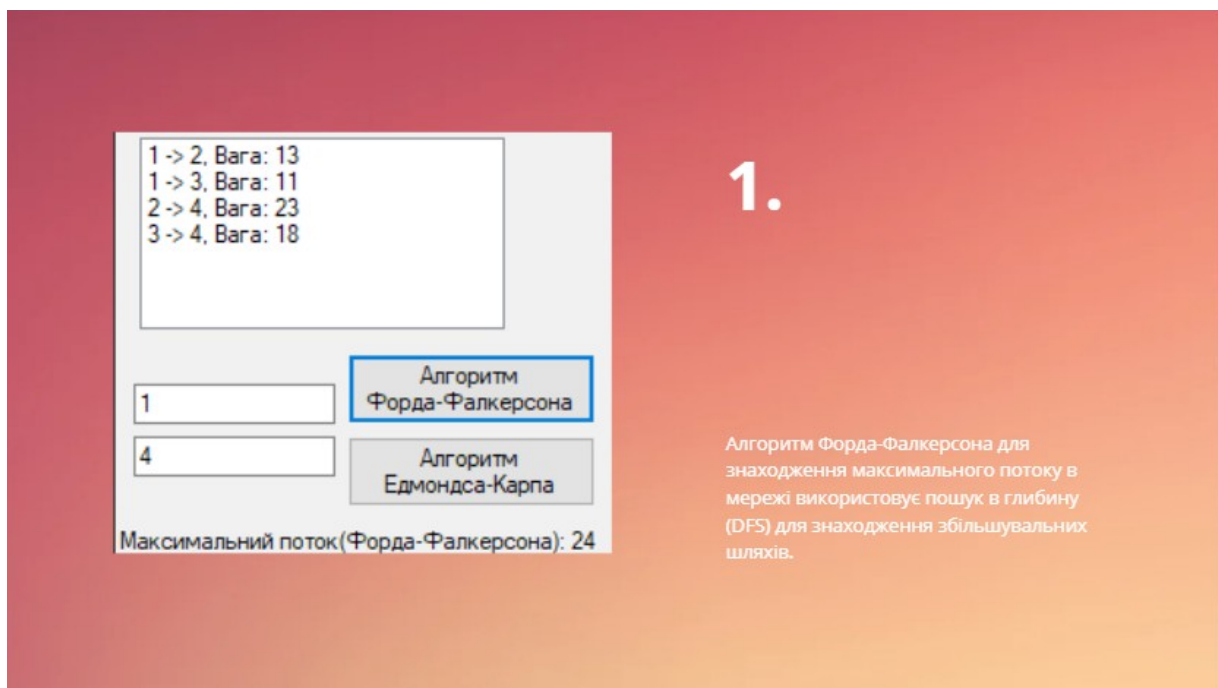


Рисунок 16 – Слайд 16. Метод 1

Примітки до слайду 16: Алгоритм Форда-Фалкерсона для знаходження

максимального потоку в мережі використовує пошук в глибину (DFS) для знаходження збільшувальних шляхів. Ось основні етапи роботи алгоритму:

- Ініціалізація: Алгоритм отримує граф, у якому визначено джерело (source) та стік (sink). У кожному ребрі графа є потужність (capacity), що визначає максимальний потік через це ребро;

- Пошук збільшувального шляху: За допомогою DFS алгоритм знаходить шлях від джерела до стоку, де всі ребра мають ненульову залишкову ємність (тобто можуть пропускати потік). Для цього відмічаються відвідані вершини, а також зберігаються батьківські вершини для кожного вузла, що дозволяє відновити шлях після пошуку;

- Оновлення потоку: Як тільки знаходиться збільшувальний шлях, обчислюється мінімальна ємність серед усіх ребер на шляху (це максимальний потік, який можна провести через цей шлях). Потім потік в кожному ребрі на шляху зменшується на величину мінімальної ємності, а зворотне ребро (якщо його немає, то воно створюється) збільшується на ту ж величину;

- Повторення пошуку: Алгоритм повторює пошук збільшувального шляху, поки такий шлях існує. Якщо на поточному етапі не вдається знайти новий шлях, алгоритм завершується;

- Результат: Після завершення пошуків алгоритм повертає загальний максимальний потік, який можна передати від джерела до стоку;

Цей алгоритм не гарантує оптимальність у часі, оскільки його складність залежить від кількості шляхів і величини потоку, але в багатьох випадках працює ефективно.

Алгоритм Форда-Фалкерсона
 Максимальний потік графа1: 96
 Максимальний потік графа2: 99
 Максимальний потік графа3: 88
 Максимальний потік графа4: 142
 Максимальний потік графа5: 135
 Максимальний потік графа6: 123
 Максимальний потік графа7: 85
 Максимальний потік графа8: 117
 Максимальний потік графа9: 81
 Максимальний потік графа10: 117
 Максимальний потік графа11: 100
 Максимальний потік графа12: 171
 Максимальний потік графа13: 114
 Максимальний потік графа14: 98
 Максимальний потік графа15: 153
 Максимальний потік графа16: 136
 Максимальний потік графа17: 163
 Максимальний потік графа18: 130
 Максимальний потік графа19: 120
 Максимальний потік графа20: 82

2.

Алгоритм Форда-Фалкерсона для пошуку максимального потоку в мережі

Рисунок 17 – Слайд 17. Метод 2

Примітки до слайду 17:

Алгоритм Форда-Фалкерсона для пошуку максимального потоку в мережі працює наступним чином:

а) Ініціалізація: Створюється копія графа, яка буде використовуватися як залишковий граф. Встановлюється початковий потік на 0;

б) Пошук шляху з додатковим потоком:

– Використовується пошук в ширину (BFS) для знаходження шляху від джерела до стоку в залишковому графі, де потік ще можливий (тобто залишкові пропускні здатності більше за 0);

– Під час пошуку для кожної вершини зберігається її батьківська вершина для відновлення шляху.

в) Оновлення залишкового графа:

– Після знаходження шляху визначається мінімальний потік на цьому шляху (найменша пропускна здатність);

– Залишкові значення потоку в графі оновлюються: для кожного ребра зменшується потік в напрямку шляху, а для зворотнього ребра (в протилежному напрямку) збільшується;

г) Повторення пошуку: Процес повторюється до тих пір, поки не можна знайти новий шлях від джерела до стоку.

г) Результат: Після завершення алгоритму повертається загальний максимальний потік.

Алгоритм повторює кроки пошуку шляху та оновлення залишкових потоків, поки не досягає неможливості знайти новий шлях, що означає, що потік максимально збільшений.

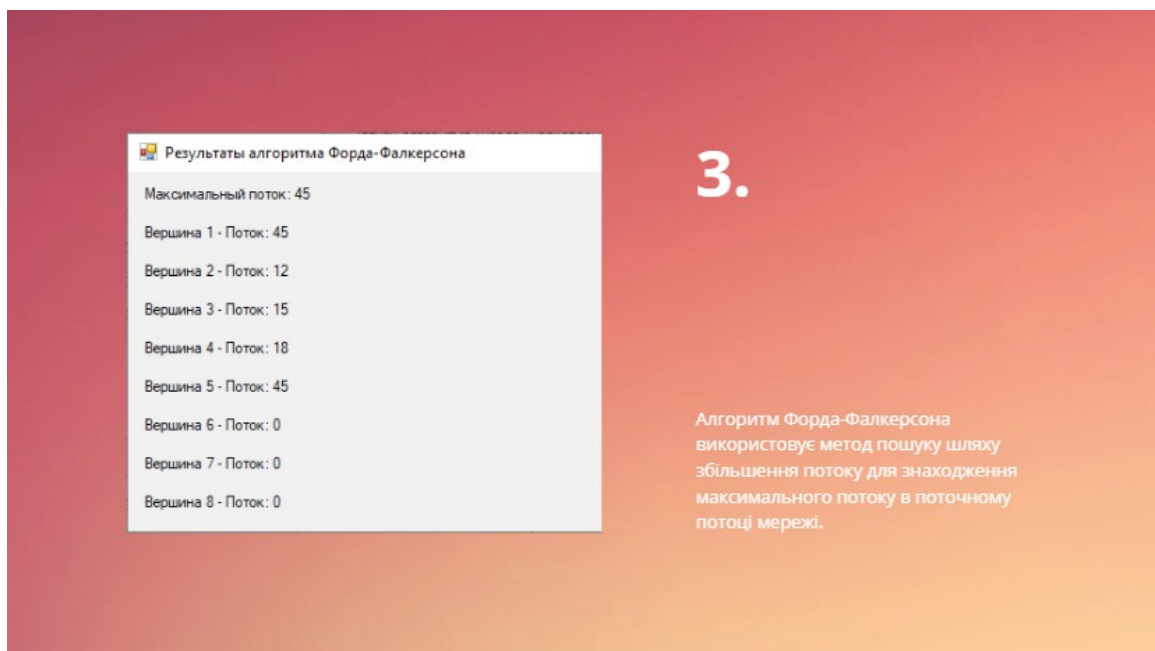


Рисунок 18 – Слайд 18. Метод 3

Примітки до слайду 18: Алгоритм Форда-Фалкерсона використовує метод пошуку шляху збільшення потоку для знаходження максимального потоку в поточному потоці мережі. Ось як працює цей алгоритм:

а) Ініціалізація графа: У конструкторі класу FordFulkerson створюється граф, в

якому зберігаються пропускні здатності ребр і самі ребра. Граф зберігається у вигляді списку суміжності, а пропускні здатності – у вигляді словників.

б) Основний цикл: Метод FindMaxFlow виконує основний цикл, який працює, поки є можливість знайти шлях збільшення потоку (за допомогою методу FindAugmentingPath).

- Якщо шлях є, знаходиться його мінімальна пропускна здатність (яка обмежує потік).

- Потім потік додається до максимального потоку, і пропускні здатності на ребрах оновлюються: зменшується пропускна здатність в напрямку потоку, і збільшується пропускна здатність у зворотному напрямку (для можливості зворотного потоку).

в) Пошук шляху збільшення потоку: Метод FindAugmentingPath знаходить шлях від джерела до стоку, де є пропускна здатність (шлях, по якому можна провести додатковий потік). Для цього використовується пошук в ширину (BFS). Якщо такий шлях знайдений, алгоритм відновлює його.

г) Повернення результату: Після того, як всі можливі шляхи з потоком знайдені, метод FindMaxFlow повертає максимальний потік, який може пройти від джерела до стоку.

Алгоритм триває, поки можна знайти шлях з ненульовою пропускною здатністю, і він гарантує знаходження максимального потоку в мережі.

**Алгоритм Едмондса-Карпа обран як
вдосконалений алгоритм Форда-
Фалкерсона і гарантує більш високу
ефективність завдяки використанню BFS
та виконується наступними методами**

Рисунок 19 – Слайд 19. Обрання алгоритму Едмондса-Карпа

1.

Алгоритм Едмондса-Карпа використовується для знаходження максимального потоку в мережі з джерела до стоку. Це варіант алгоритму Форда-Фалкерсона, що використовує пошук у ширину (BFS) для знаходження збільшувальних шляхів.

1 -> 2, Bara: 13
1 -> 3, Bara: 11
2 -> 4, Bara: 23
3 -> 4, Bara: 18

Алгоритм
Форда-Фалкерсона

Алгоритм
Едмондса-Карпа

Максимальний потік(Едмондса-Карпа): 24

Рисунок 20 – Слайд 20. Метод 1

Примітки до слайду 20: Алгоритм Едмондса-Карпа використовується для знаходження максимального потоку в мережі з джерела до стоку. Це варіант алгоритму Форда-Фалкерсона, що використовує пошук у ширину (BFS) для

знаходження збільшувальних шляхів. Ось як працює алгоритм:

а) Ініціалізація: Алгоритм працює на графі з вершинами і ребрами, де кожне ребро має ємність (максимальний потік, який може пройти через це ребро). Граф також містить зворотні ребра з нульовими ємностями.

б) Пошук збільшувального шляху (BFS): Використовується BFS для пошуку шляху з джерела до стоку, де всі ребра мають позитивну ємність. Якщо такий шлях знайдено, то він є збільшувальним шляхом, через який можна перевести потік.

в) Визначення потоку для знайденого шляху: Для кожного знайденого збільшувального шляху визначається мінімальний потік (мінімальна ємність серед всіх ребер на шляху).

г) Оновлення графа:

- Потік зменшується для кожного ребра на шляху на величину знайденого мінімального потоку;

- Якщо зворотного ребра не було, воно створюється з нульовою ємністю;

- Для зворотних ребер потік збільшується.

г) Повторення: Процес повторюється, поки існують збільшувальні шляхи.

д) Результат: Після того, як неможливо знайти більше збільшувальних шляхів, сума потоку на всіх знайдених шляхах є максимальним потоком.

Алгоритм завершується, коли не можна знайти шлях з джерела до стоку з позитивною ємністю, і повертається максимальний потік.

2.

Алгоритм Едмондса-Карпа — це реалізація класичного методу пошуку максимального потоку в мережі, що використовує пошук в ширину (BFS) для знаходження augmenting paths (шляхів, які можуть збільшити потік).

Алгоритм Едмондса-Карпа
 Максимальний потік графа1: 96
 Максимальний потік графа2: 99
 Максимальний потік графа3: 88
 Максимальний потік графа4: 142
 Максимальний потік графа5: 135
 Максимальний потік графа6: 123
 Максимальний потік графа7: 85
 Максимальний потік графа8: 117
 Максимальний потік графа9: 81
 Максимальний потік графа10: 117
 Максимальний потік графа11: 100
 Максимальний потік графа12: 171
 Максимальний потік графа13: 114
 Максимальний потік графа14: 98
 Максимальний потік графа15: 153
 Максимальний потік графа16: 136
 Максимальний потік графа17: 163
 Максимальний потік графа18: 130
 Максимальний потік графа19: 120
 Максимальний потік графа20: 82

Рисунок 21 – Слайд 21. Метод 2

Примітки до слайду 21: Алгоритм Едмондса-Карпа — це реалізація класичного методу пошуку максимального потоку в мережі, що використовує пошук в ширину (BFS) для знаходження augmenting paths (шляхів, які можуть збільшити потік). Ось як працює цей алгоритм:

а) Ініціалізація:

- Створюється копія графа, яка є залишковим графом (для обліку потоку, що ще можна пройти через ребра);
- Задається змінна для максимального потоку `maxFlow`, яка спочатку дорівнює 0.

б) Пошук augmenting path:

- Поки є можливість знайти новий шлях від джерела до стоку, виконується пошук в ширину;
- Для цього використовується черга, де елементи — це вершини графа. Починаючи з джерела, алгоритм намагається знайти шлях до стоку, по якому ще є

можливість передати потік;

– Під час пошуку відмічаються батьківські вершини для відновлення шляху.

в) Оновлення потоку:

– Коли знаходиться augmenting path (шлях з залишковим потоком), визначається мінімальний потік на цьому шляху;

– Потім оновлюються значення в залишковому графі: зменшується потік по пройденим ребрам і збільшується по зворотних (для можливості відміни потоку в майбутньому).

г) Оновлення загального потоку: Після оновлення залишкового графа потік по знайденому шляху додається до загального потоку.

г) Завершення: Алгоритм завершується, коли не можна знайти жодного шляху з джерела до стоку, тобто коли потік більше не може бути збільшений.

Результат алгоритму – це максимальний потік, який можна передати з джерела в стік.

3.

Алгоритм Едмондса-Карпа — це реалізація методу пошуку максимального потоку в мережі, заснована на алгоритмі Форда-Фалкерсона. Він використовує пошук в ширину (BFS) для знаходження шляху з джерела до стоку, по якому ще можна збільшити потік.

Результаты алгоритма Едмондса-Карпа	
Максимальный поток:	39
Вершина 1 - Поток:	39
Вершина 2 - Поток:	12
Вершина 3 - Поток:	15
Вершина 4 - Поток:	12
Вершина 5 - Поток:	33
Вершина 6 - Поток:	6
Вершина 7 - Поток:	0
Вершина 8 - Поток:	0

Рисунок 22 – Слайд 22. Метод 3

Примітки до слайду 22: Алгоритм Едмондса-Карпа – це реалізація методу пошуку максимального потоку в мережі, заснована на алгоритмі Форда-Фалкерсона. Він використовує пошук в ширину (BFS) для знаходження шляху з джерела до стоку, по якому ще можна збільшити потік. Основні етапи роботи алгоритму:

а) Ініціалізація:

- Створюється граф, який містить інформацію про вершини, ребра і пропускні спроможності між вершинами;
- Потоки між вершинами ініціалізуються нулями.

б) Пошук шляху з джерела до стоку:

- Використовує BFS для пошуку шляху, по якому можна збільшити потік, враховуючи поточні потоки і пропускні спроможності;
- Якщо такий шлях існує, переходить до наступного етапу. Якщо ні – алгоритм завершується.

в) Оновлення потоків:

- Після знаходження шляху визначається мінімальна пропускна спроможність (обмеження потоку);
- Потоки оновлюються для всіх ребер на знайденому шляху;
- Пропускні спроможності зменшуються на величину потоку, а зворотні ребра отримують додаткову пропускну спроможність.

г) Повторення: Алгоритм повторює пошук шляху і оновлення потоків, поки є можливість збільшити потік.

г) Результат: Після завершення алгоритму отримується максимальний потік, який можна передати від джерела до стоку.

Алгоритм продовжує виконуватися до тих пір, поки знаходить шляхи, по яких можна додати потік. Кожен такий шлях дозволяє збільшити загальний потік, поки не буде досягнуто максимуму.

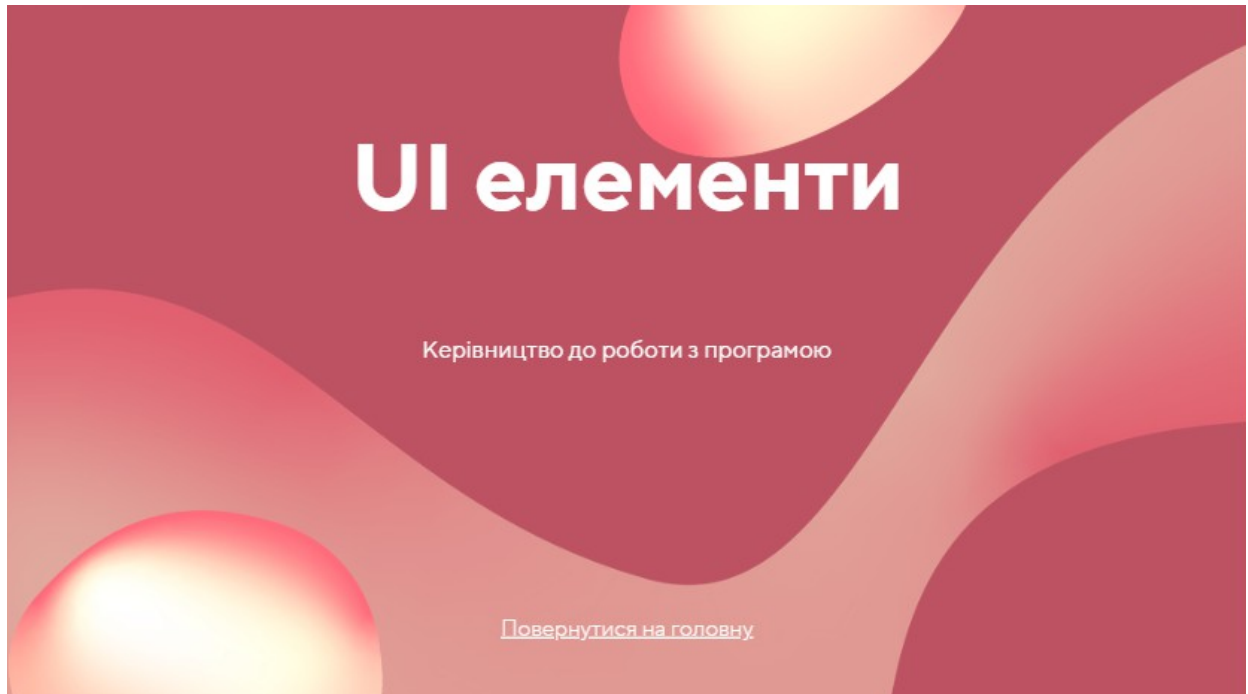


Рисунок 23 – Слайд 23. UI елементи

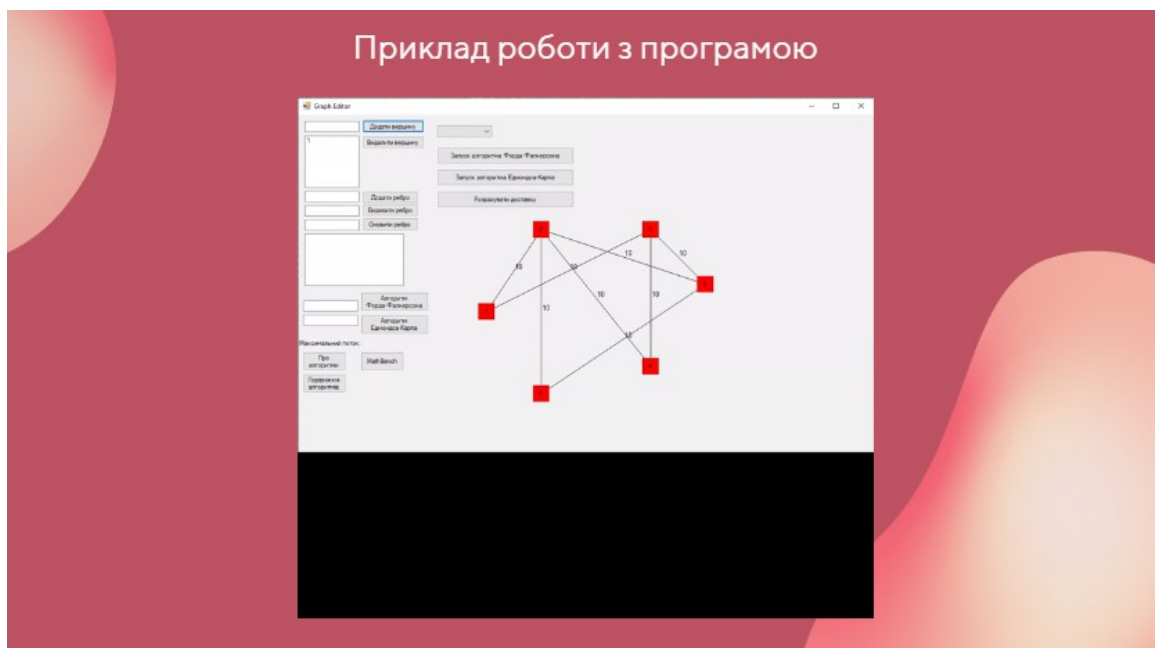


Рисунок 24 – Слайд 24. Приклад роботи з програмою

Опис функцій інтерфейсу програми:

1. Додавання вершин до графу: Ця функція дозволяє користувачу додавати нові вершини до графа. Користувач може вказати назву або ідентифікатор вершини. Після додавання вершина з'являється на графі, і її можна використовувати для побудови ребер або для подальших операцій.

2. Додавання ребер між вершин: Після створення вершин користувач може визначити з'єднання між ними за допомогою ребер. Для кожного ребра користувач вказує пару вершин, між якими воно має бути проведено, а також вагу ребра (наприклад, максимальний потік або відстань). Це дозволяє створювати складніші графи з різними характеристиками зв'язків.

3. Введення істоку та стоку. Знаходження максимального потоку: Користувач може визначити початкову та кінцеву вершину для обчислення максимального потоку. Істок – це точка, з якої починається потік, а сток – кінцева точка. Алгоритм на основі заданого графа та цих вершин обчислює максимальний потік, який може пройти від істоку до стоку. Це включає застосування алгоритмів, таких як алгоритм Форда-Фалкерсона чи алгоритм Едмондса-Карпа.

4. Виведення основної інформації про алгоритми: У цій функції відображається інформація про використані алгоритми для обчислення потоку. Це може включати опис алгоритмів, їхню складність, переваги та обмеження. Виведена інформація допомагає користувачу краще зрозуміти, як працює алгоритм і чому було обрано саме цей метод.

5. Виведення основних відмінностей між алгоритмами: Ця функція порівнює кілька алгоритмів для вирішення однієї задачі (наприклад, обчислення максимального потоку) і виводить їхні основні відмінності. Користувач зможе дізнатися, який алгоритм є швидшим для конкретних випадків, які у нього є переваги, а також яких типів графів він найкраще обробляє.

6. Програма автоматично виконає 20 запусків алгоритмів на графах із файлу: Програма має вбудовану функціональність для автоматичного запуску тестів на

наборах графів із зовнішнього файлу. Вона зчитує дані про графи, виконує 20 запусків алгоритмів, аналізуючи різні графи для перевірки коректності та ефективності алгоритмів.

7. Вибір графу та алгоритму: Користувач може вибрати конкретний граф та алгоритм для обчислення максимального потоку або іншої задачі. Інтерфейс надає список доступних графів і алгоритмів, щоб забезпечити зручний доступ до різних варіантів обчислень.

8. Побудований граф: Візуалізація графу, що відображає вершини та ребра в графічному інтерфейсі. Користувач може побачити повну картину структури графа, з можливістю масштабування, перегляду інформації про кожну вершину та ребро.

9. Кнопка для розрахунку доставки за алгоритмом: Користувач може натискати цю кнопку, щоб запустити алгоритм для розрахунку доставки товару по заданому графу. Алгоритм може визначити оптимальний маршрут для транспортування товару або обчислити час доставки.

10. Меню для вибору товару, його кількості: Ця функція дозволяє вибрати товар із запропонованого списку, а також вказати кількість одиниць товару, які потрібно доставити. Це забезпечує взаємодію з логістичною частиною системи доставки товарів.

11. Розрахунок терміну доставки: Після вибору товару та кількості, програма використовує обраний алгоритм для розрахунку терміну доставки товару по графу. Це може включати обчислення найшвидшого або найдешевшого маршруту, враховуючи можливі обмеження на доставку (наприклад, вантажопідйомність, час чи відстань).

12. Кнопка для перегляду всіх замовлень: Кнопка дозволяє користувачу переглядати всі зроблені замовлення, включаючи інформацію про товари, кількість, дату замовлення та статус доставки.

13. Вікно для перегляду всіх замовлень: Це спеціальне вікно, де відображається таблиця або список усіх замовлень, зроблених користувачем або в системі. Вікно може містити фільтри для пошуку замовлень за різними критеріями (наприклад, за датою,

статусом або товаром).

14. Повний інтерфейс програми: Інтерфейс програми є комплексним і включає всі функції, перераховані вище. Він має зручний дизайн з панелями навігації, полями для введення даних, кнопками для запуску обчислень, а також вікнами для перегляду результатів. Користувач може швидко переходити між різними функціями та отримувати всю необхідну інформацію для виконання завдань.

Кожна з цих функцій розроблена для того, щоб зробити інтерфейс інтуїтивно зрозумілим та зручним для користувача, забезпечуючи повний контроль над процесами додавання даних, виконання розрахунків і перегляду результатів.

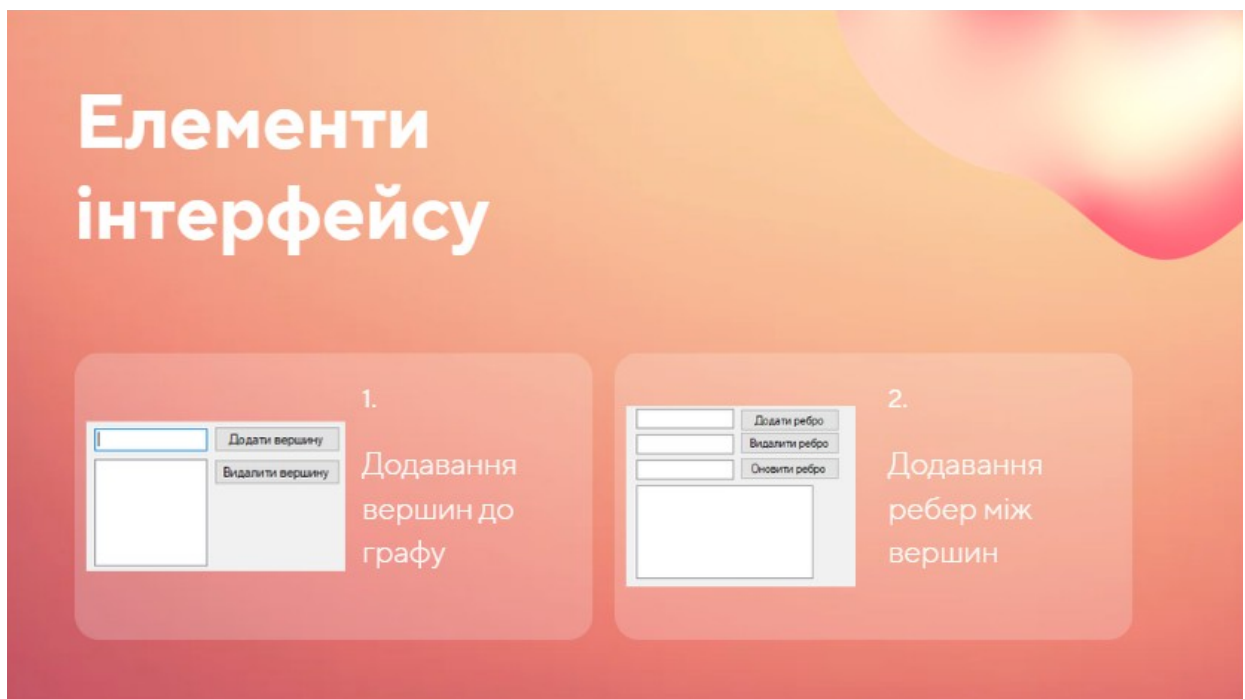


Рисунок 25 – Слайд 25. Елементи інтерфейсу

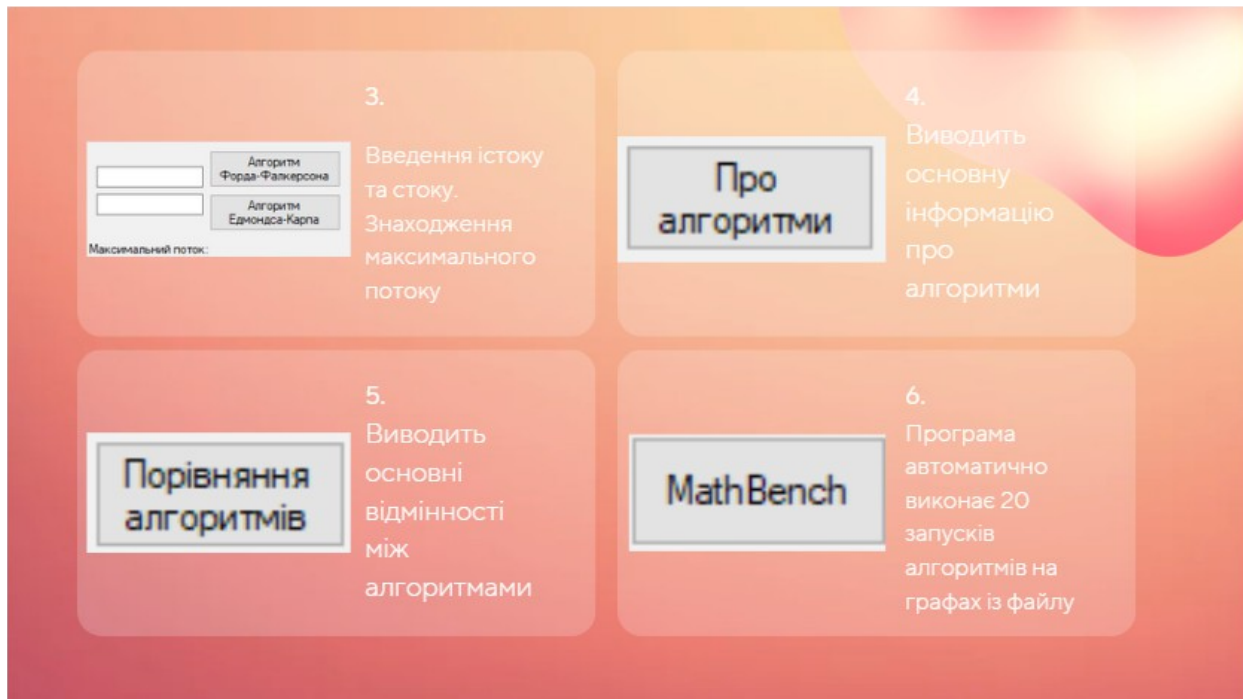


Рисунок 26 – Слайд 26. Елементи інтерфейсу

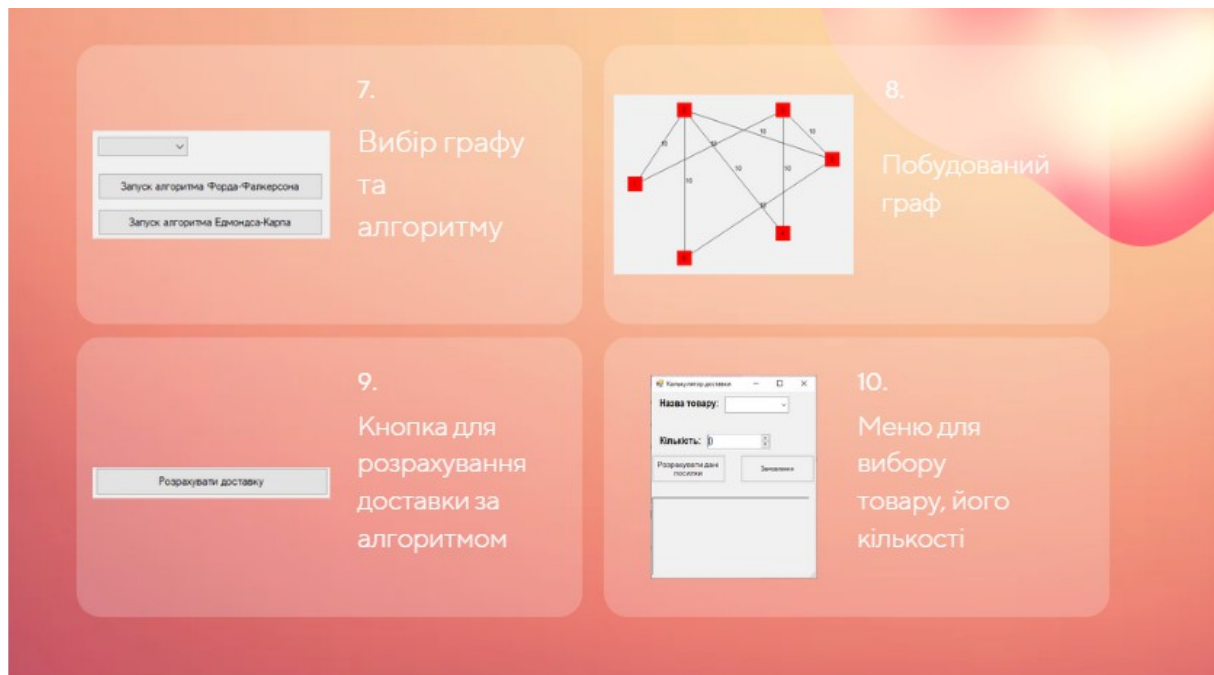


Рисунок 27 – Слайд 27. Елементи інтерфейсу

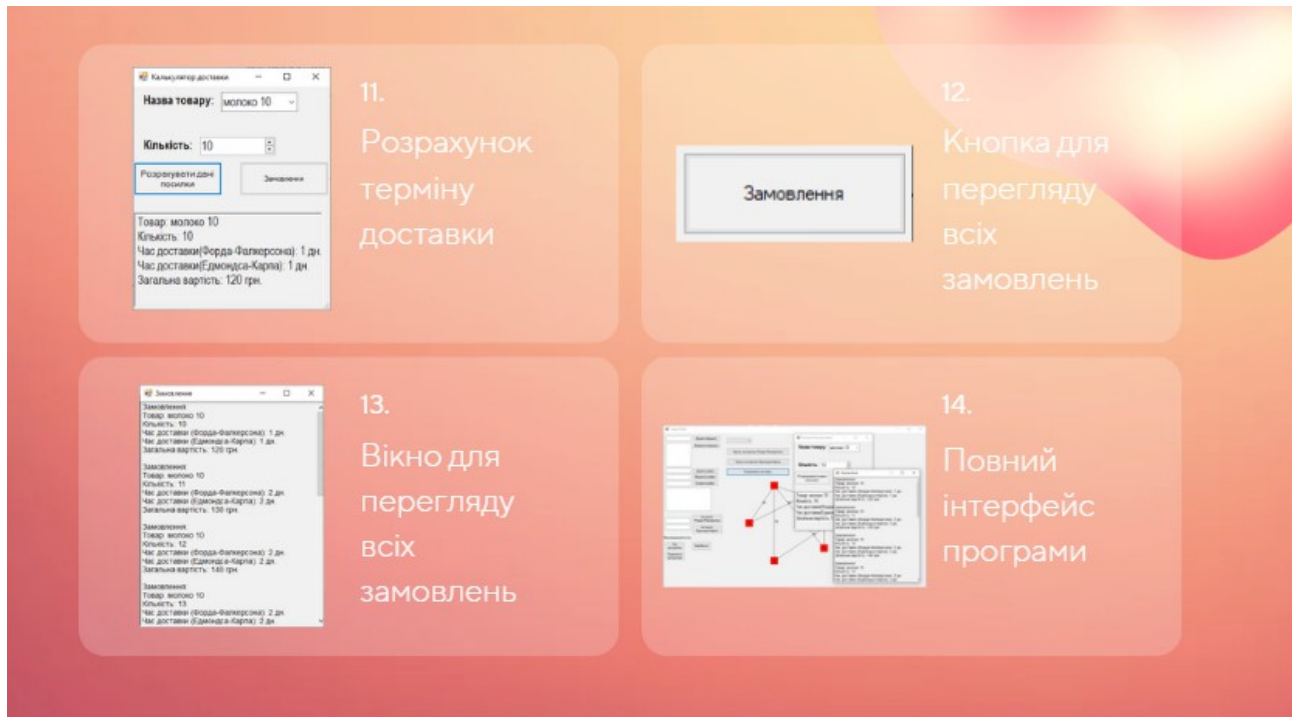


Рисунок 28 – Слайд 28. Елементи інтерфейсу

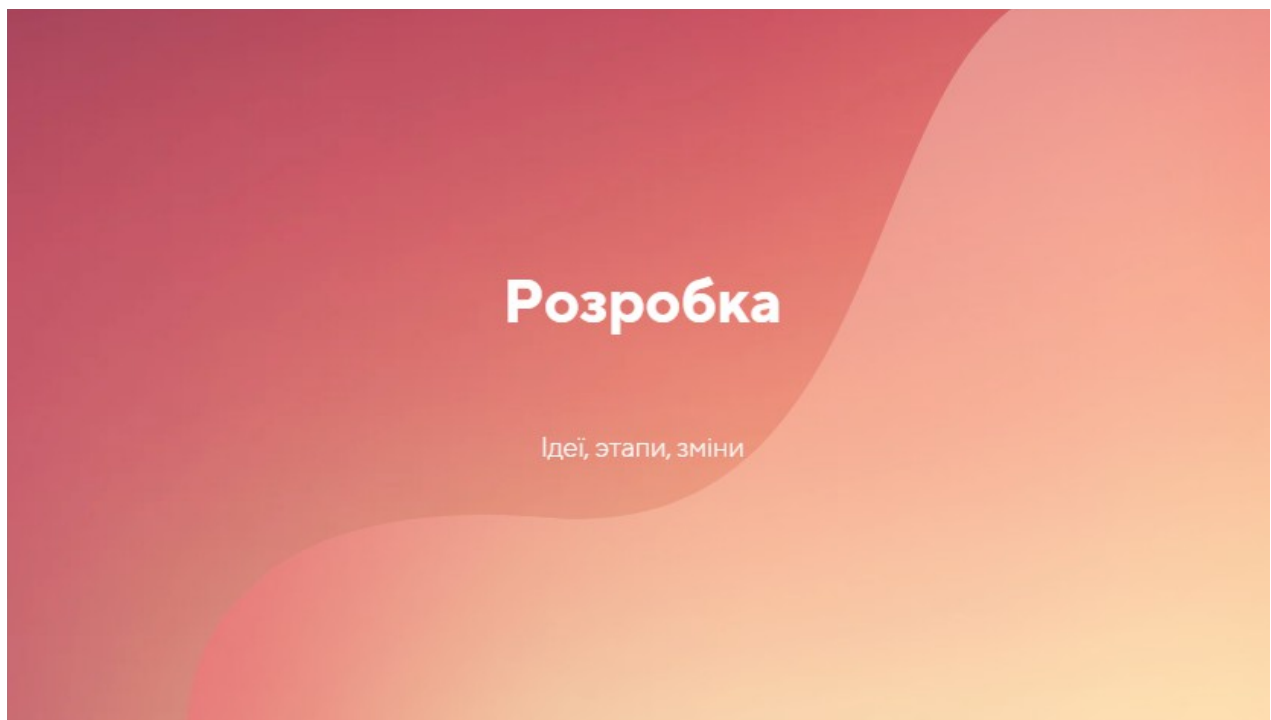


Рисунок 29 – Слайд 29. Розробка

Історія розробки

Початок

Визначення завдання, розподіл обов'язків, перший макет

Внесення змін

Змінення основної концепції програми, підбір основних методів та алгоритмів, визначення основного інтерфейсу

Полірування

Визначення стилю роботи, виправлення помилок програми, розробка програмної документації

Фінішна лінія

Завершення програмної документації, виправлення помилок, порівняння результатів, перевірка точності. I...

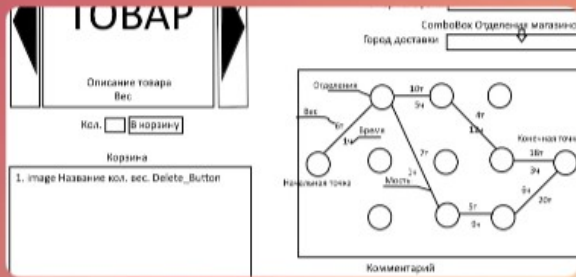


Рисунок 30 – Слайд 30. Історія розробки

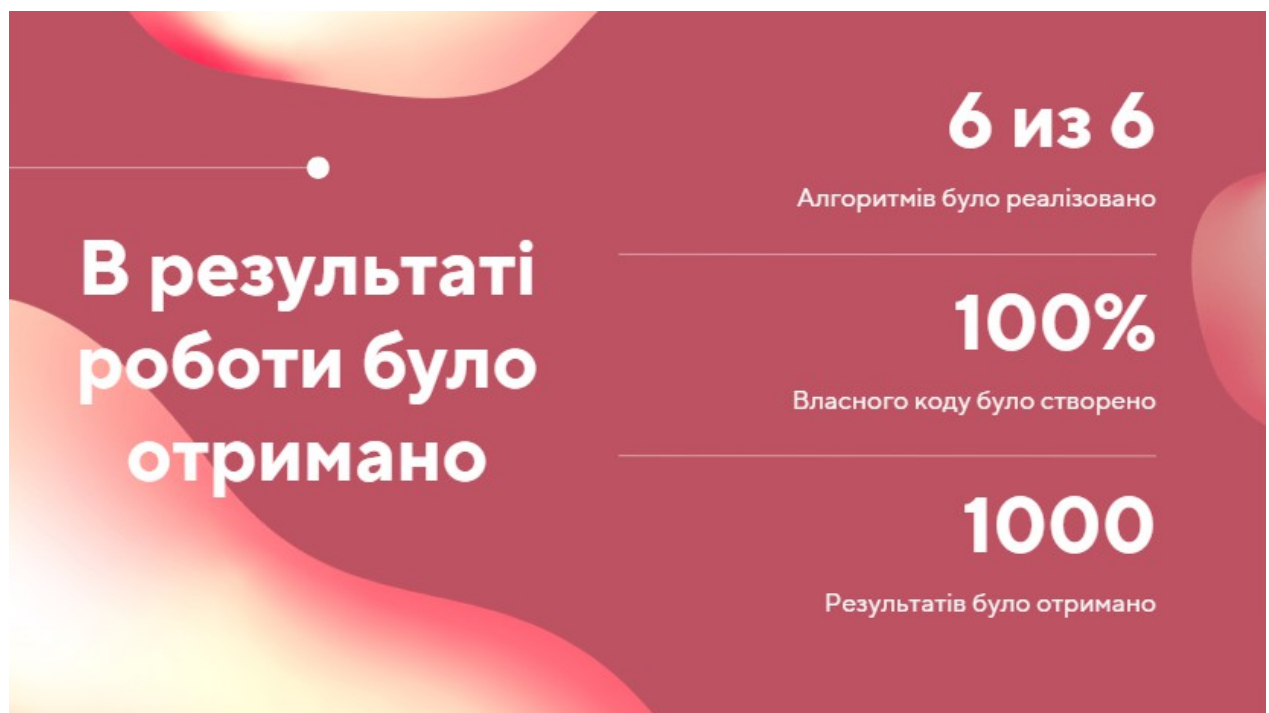


Рисунок 31 – Слайд 31. Результати

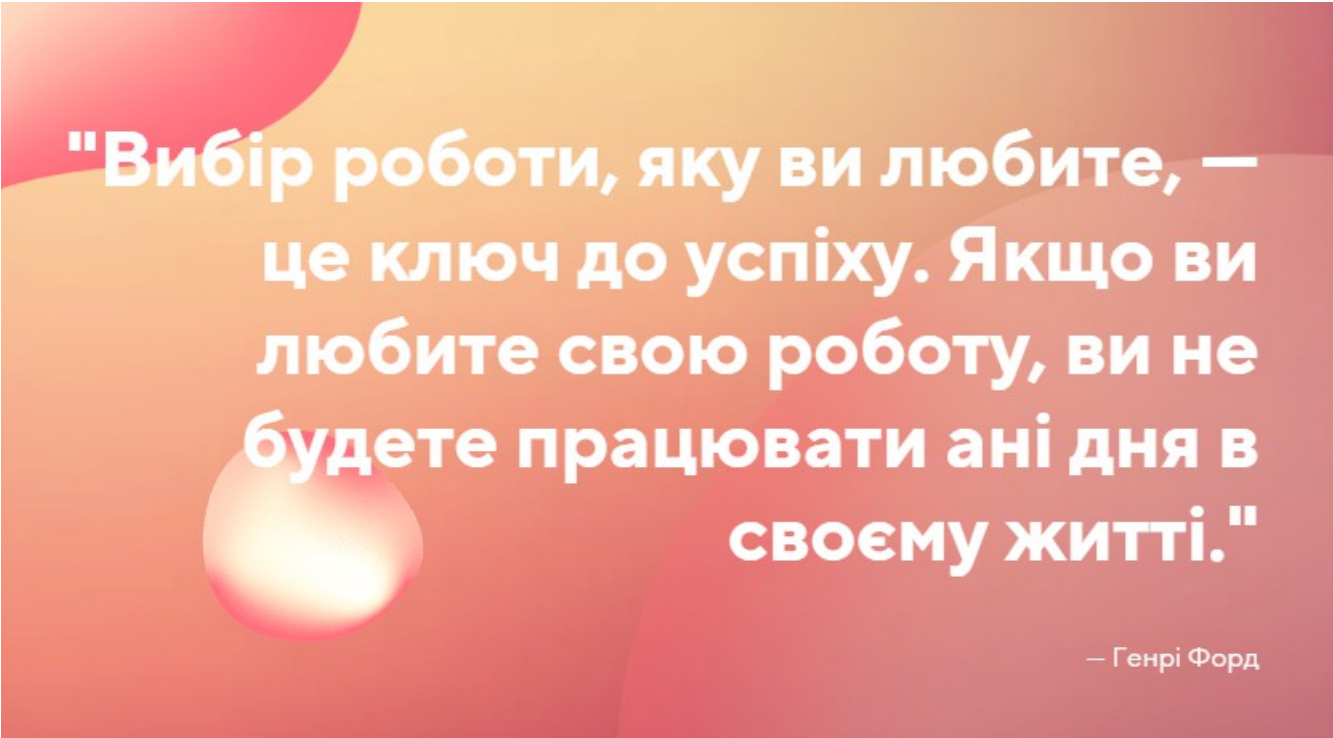
ВИСНОВОК

В ході курсового проєкту ми дослідили застосування структур даних та алгоритмів в процесі доставки та перевезення вантажів з використанням графів. Окрему увагу буде приділено порівнянню алгоритмів максимізації потоку, зокрема, алгоритму Форда-Фалкерсона, як базового методу для розв'язання задачі знаходження максимального потоку в мережі. В була розглянута ефективність застосування цих алгоритмів для оптимізації процесів доставки, а також проведено їх порівняння за різними критеріями, такими як швидкість виконання, складність реалізації та точність результатів.

Рисунок 32 – Слайд 32. Висновок

Дякую за увагу!

Рисунок 33 – Слайд 33. Дякую за увагу!



**"Вибір роботи, яку ви любите, —
це ключ до успіху. Якщо ви
любите свою роботу, ви не
будете працювати ані дня в
своєму житті."**

— Генрі Форд

Рисунок 34 – Слайд 34. Мотиваційна цитата